
PDF/SOLUTIONS™

exensio®

**Exensio® –Test
SwifTest**

End–User Manual

version 7.4.1.3 and Higher



CONTENTS

88-180301-D6

PDF Solutions, Inc. Confidential Information

NOTICE

This document has been developed and created by PDF Solutions[®], Inc.

The software programs described in this document are copyrighted and are confidential information and proprietary products of PDF Solutions, Inc. This Manual is furnished to authorized users of PDF Solutions products, and is intended solely to facilitate the use of PDF Solutions tools and products, as specified in written agreements. The recipient of this document, by its retention and use, agrees to hold this document and its content in strict confidence and to protect the same from loss, theft or unauthorized use. No part of this publication may be reproduced, stored in a retrieval system, translated, transcribed, or transmitted, in any form, or by any means without prior explicit written permission from PDF Solutions, Inc. This Notice is an integral part of this document and shall not be removed or altered.

While every precaution has been taken in the preparation of this document PDF Solutions assume no responsibility for errors or omissions, or for damages resulting from the use of information contained in this document or from the use of programs and source code that may accompany it, if/as applicable. In no event shall PDF Solutions be liable for any loss of profit or any other commercial damage caused or alleged to have been caused directly or indirectly by this document.

PDF Solutions, Inc. reserves the right to make changes in specifications and other information contained in this publication without prior notice. The reader should in all cases consult PDF Solutions to determine whether any such changes have been made.

Download Disclaimer

Through this documentation you can link to other websites which are not under the control of PDF Solutions. We have no control over the nature, content and availability of those sites. In no event shall we, PDF Solutions, be liable for any loss or damage including without limitation, indirect or consequential loss or damage, or any loss or damage whatsoever arising from loss of data or profits arising out of or in connection with the use of any external website sites.

Trademarks

Characterization Vehicle[®], Circuit Surfer[®], CV[®], dataPOWER[®] and dataPOWER[®], Exensio[®], maestria[®], ModelWare[®], pdFasTest[®], PDF Solutions[®], Yield Ramp Simulator[®], and YRS[®] are registered trademarks of PDF Solutions, Inc. or its subsidiaries; and ALPS[™], Advanced Monitoring Executive[™], AMX[™], Design-to-Silicon-Yield[™], dataPOWER[®] VSF[™], dP-bitMAP[™], dP-Defect[™], dP-Mining[™], dP-SSA[™], dP-Variability Analysis[™], dP-WorkFlow[™], DTC[™], Dynamic Test Controller[™], Exensio ALPS[™], Exensio –Control[™], Exensio –Test[™], Exensio –Test DTC[™], Exensio –Hosted[™], Exensio –Yield[™], pdBRIX[™], pdCV[™], SwifTest[™], Template[™], and YA-FDC[™] are our common law trademarks.

The following logos are trademarks or registered trademarks of PDF Solutions:

PDF/SOLUTIONS[™]

exensio[®]

maestria[®]

Copyright[©] 1991-2021 PDF Solutions, Inc. All Rights Reserved.

PDF Solutions, Inc.

2858 De La Cruz Blvd., Santa Clara, CA 95050 USA

Tel: (408) 280-7900 ~ Fax: (408) 280-7915

email: info@pdf.com (general), doc@pdf.com (documentation),

dpsupport@pdf.com (support)

web: www.pdf.com

PDF Solutions, Inc. Confidential Information

CONTENTS

Notice	2
--------------	---

CHAPTER 1

SwiftTest Overview	11
Overview of SwiftTest Product Group	11
SwiftTest-AMX	11
SwiftTest-TTO	12
SwiftTest-FOX	12
SwiftTest-MAX	12
SwiftTest-DTI	13
SwiftTest-PAT	13
Remote Command Interface	13
Exensio –Test	14

CHAPTER 2

SwiftTest MAX – Triggers	15
Triggers and Trigger Actions	15
Introduction	15
Trigger Profile Entry Example	16
Profile XML Specification	16
Process Rule Triggers and Examples	17
BinCountTrigger	17
BinPercentTrigger	19
ConsecutiveBinTrigger	22
ParametricTestSpcTrigger	25
ParametricTestStatisticTrigger	28
SiteToSiteParametricTestStatisticDeltaTrigger	32
ParametricTestYieldTrigger	37
EcidTrigger	40
SiteToSiteBinPercentDeltaTrigger	43
YieldTrigger	46
RecoveryYieldTrigger	48
YieldTriggerByTouchdown	51

SiteToSiteYieldTrigger	53
DeviceFrequencyTrigger	55
LotBeginTrigger	56
LotEndTrigger	57
LotValidationTrigger	58
SegmentBeginTrigger	59
SegmentEndTrigger	60
TimeFrequencyTrigger	61
Exception Rule Triggers and Examples	62
BadDeviceGoodBinTrigger	62
GoodDeviceBadBinTrigger	64
DeviceTestTimeTrigger	66
TestCountTrigger	70
BLostSiteTrigger	73
LotSetupTrigger	74
MixedLotLimitTrigger	75
LimitExceptionTrigger	77
LimitFailGoodBinTrigger	80
RetestLimitTrigger	83
MeasurementExceptionTrigger	85
UnitsPerHourLimitTrigger	89
Trigger Actions	91
CreateBinningAlertReport	91
CreateBinningReport	92
CreateDynamicTestReport	93
CreateEventReport	94
CreateGenericReport	95
CreatellogReport	96
CreateLotStatusReport	97
CreateLotSummaryReport	98
CreateMidLotReport	99
CreateOutlierDetectionReport	100
CreateOutlierEventReport	101
CreatePDLReport	102
CreateSamplingReport	103
ExecuteProxyCommand	104

ExecuteSystemCommand.....	105
VisualAlert	105
WriteMessageToLog	106
Reporting Output Strategies	107
File Output Strategy	107
Socket Output Strategy.....	108
FTP Output Strategy.....	109
Email Output Strategy	110
Notification Output Strategy	111
Examples	112
Example 1	112
Example 2	114
Custom Trigger and Trigger Actions	115
Reports.....	116
Heartbeat.xml.....	116
DieResults Reports.....	117

CHAPTER 3

SwiftTest Commands	119
Remote Command Interface.....	119
AbortLot	119
AddRemoteSystemEventListener	119
BeginLotMonitorSession	120
BeginParametricDataSession	120
BeginSamplingSession.....	120
BeginSocketLotMonitorSession.....	121
BeginSwiftTestProxyConnection	121
BinSummary.....	121
DisableTestRunTriggers.....	122
EndSamplingSession	122
EndSwifttestProxyConnection	122
ForwardMessage	122
GetActiveTriggers	123
GetDeviceProfile	123
GetHistory	123

GetLibDirectory	124
GetLogFile.....	124
GetLogFileList	124
GetLotDetails	124
GetOpenDataSourceList.....	124
GetSettings	125
LicenseInfo	125
OpenLots	125
OpenNewStdDataConduitConnection	126
RemoveRemoteSystemEventListener	126
SetCapture	127
SetDebug	127
SetLotClass.....	127
Shutdown.....	127
ShutdownNow	128
SiteSummary	128
Stop.....	128
SystemStatus	128
TriggerReset	129
Command-Line Interface.....	130
swifttest Usage	130
Usage Examples.....	131

CHAPTER 4

SwiftTest FOX	135
Introduction to SwiftTest-FOX	135
During Test Run	135
Test-Run End.....	135
Wafer End.....	135
Outlier Detection Algorithm Catalog	136
Adjacent Rank Delta	136
General ARD Process.....	136
ARD over Interquartile Range Process (IQR).....	137
Interpercentile Range - IPR.....	140
General Interpercentile Range (IPR) Procedure	141

Principal Component Analysis	142
Principal Component Outliers	144
Issues With PCA-based Production Test	146
Defect Resolution	148
Single Regression Residual.....	151
Part Average Testing (PAT) With FOX	155
Overview – PAT Within FOX	155
AEC PAT	155
Box Plot.....	155
Robust Sigma	155
PAT By Site	155
STDF Update Example	156
MPR Example	157
During Test Outlier Detection.....	158
Profile Example.....	159
Profile XML Specification	159
Adjacent Rank Delta	160
Profile Example.....	160
Parameters	160
Interpercentile Range	161
Profile Example.....	161
Parameters	161
Single Regression Residual.....	162
Profile Example.....	162
Parameters	162
Test-Run End Outlier Detection.....	163
Profile Example.....	163
Profile XML Specification	164
Single Regression Residual.....	165
Profile Example.....	165
Parameters	165
Wafer End Outlier Detection.....	166
Profile Example.....	167
Profile XML Specification	167
Principal Component Analysis	168
Profile Example.....	168

Parameters	168
Interpercentile Range	169
Profile Example	169
Parameters	169
 CHAPTER 5	
SwiftTest TTR	171
Overview	171
Test Groups	172
Profile Example	173
Profile XML Specification	173
Sampling Conditions	174
InStatisticalControl	174
FailureCount	175
Sampling Strategies	177
EveryNthTestRun	177
Examples	178
Example 1	178
Example 2	179
Example 3	180
Example 4	182
Selecting Test Groups for Sampling	183
Recommendations – Gathering Data and Test Times	184
Baseline Settings	185
Test Program Dependencies	185
Correlation and Target DPPM	186
Customizing Sampling	187
Creating Custom Test Time Reduction Conditions	187
Creating Custom Test Time Reduction Strategies	188

CHAPTER 6

SwiftTest Part Average Testing (PAT)	189
Overview	189
Dynamic PAT vs. Static PAT	189
Static PAT	189

Dynamic PAT.....	190
PAT Methodology	190
AEC Method	190
Robust Method	191
Device Profiles	192
Device Profile Example	193
Device Profile XML Specification.....	194
Standard AEC	194
Robust Sigma.....	194
Box Plot	194
Device Profile Example	195
PATTest Elements.....	196
PATMethod Elements	196
How Devices are Binned.....	196

CHAPTER 7

AutoUpdate.....	197
UpdateSwiftTest Overview	197
AutoUpdate Data Flow and Timing	198
How UpdateSwiftest Works	199
UpdateSwiftest Commands.....	199
UpdateSwiftest Process.....	199
SwiftestUpdate Directory Structure	200
SwiftestUpdate Configuration	201
Update_logging.xml.....	202
“UpdateFiles.zip” File Contents	202
Restoring the Previous Version of SwiftTest	203

CHAPTER 8

Appendix — Development and Integration.....	205
Remote Command Interface.....	205
Command Catalog	205
AbortLot	205
AddRemoteSystemEventListener	206
BeginLotMonitorSession	206

BeginSamplingSession	207
BeginSocketLotMonitorSession.....	207
BeginSwiftTestProxyConnection	207
BinSummary	208
DisableTestRunTriggers	208
EndSamplingSession	208
EndSwifttestProxyConnection	208
ForwardMessage	209
GetDeviceProfile	209
GetHistory	209
GetLogFile.....	210
GetLogFileList	210
GetLotDetails	210
GetOpenDataSourceList.....	210
GetSettings	211
LicenseInfo	211
OpenLots	212
OpenNewStdDataConduitConnection	212
RemoveRemoteSystemEventListener	213
SetCapture	213
SetDebug	213
SetLotClass.....	214
Shutdown.....	214
ShutdownNow	214
SiteSummary	215
Stop.....	215
SystemStatus	215

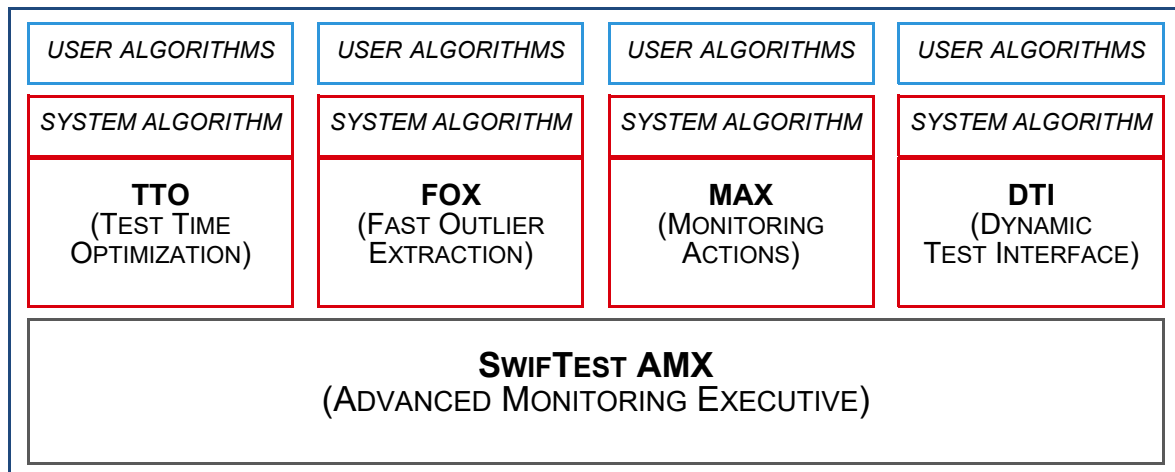
CHAPTER 1

SWIFTEST OVERVIEW

OVERVIEW OF SWIFTEST PRODUCT GROUP

The SwiftTest system, part of the Exensio –Test offering, consists of several optionally licensed modules for performing a wide variety of on-line adaptive test functions. SwiftTest allows users to add their own algorithms for controlling test flows, outlier extraction or other on-line test strategies.

AMX (Advanced Monitoring Executive) is the base functionality for SwiftTest; the other optional features will not work without it. The major optional licensed SwiftTest features are: TTO (Test Time Optimization – sampling), FOX (Fast Outlier Extraction), MAX/Triggers (Monitoring Actions), DTI (Dynamic Test Interface (custom algorithms)) and PAT (Part Average Testing - limits calculation at baseline).



THIS DIAGRAM ILLUSTRATES THE PRODUCT CONFIGURATIONS AVAILABLE FOR THIS ON-LINE SUITE OF SOFTWARE.

SwiftTest-AMX

SwiftTest-AMX — The Advanced Monitoring Executive is a pre-requisite module for all of the SwiftTest product options. As a standalone module it is used to capture data in real-time from the tester and transmit it to a database system like the Exensio database or Exensio –Test TDS. SwiftTest-AMX is documented in the *Exensio® –Test SwiftTest Advanced Monitoring Executive™ (AMX) Installation and Configuration* manual that is made available with this manual.

Standard functions included with SwiftTest-AMX are:

LDP — The Live Data Port accepts test data in various formats including STDF and translates it into a common format used by SwiftTest for its adaptive test functions.

Data Writer — The output port that sends test data to databases for systems like Exensio-Yield and Exensio –Test TDS. Reports that go directly to Exensio –Test TDS or Exensio-Yield include: iLog, PDL, MidLot, Event, ForwardMessage, DieResult.

This is handled through RPG, the report generator that prepares and emails alerts and lot summaries, based on the user-defined output strategy — just choose your output strategy: Socket, FTP, Email, File.

DCI — The SwiftTest Command interface that allows signaling between an external, off-line application and SwiftTest. This is useful for feed-forward or feed-back adaptive test algorithms.

SwiftTest

SwiftTest-TTO

The Test Time Optimization module. This product performs real-time, test flow optimization to reduce cost of test. It adaptively increases or decreases test coverage based upon the quality and yield of the current Device Under Test (DUT) and its associated lot. Flow optimization for a specific test may be controlled by the statistical performance of another test or group of tests. It is effective on both parametric tests as well as functional patterns. It is effective at both wafer probe and final (packaged) test. SwiftTest also has the capability for users to easily program their own optimization algorithms.

SwiftTest-FOX

FOX performs on-line as well as post-processing outlier detection based upon system algorithms and also algorithms from licensed libraries. This tool is effective for increasing device reliability or for reducing the cost of burn-in and may be used at both wafer probe and final (packaged) test. SwiftTest-FOX also includes the capability for users to easily program their own outlier detection and rejection algorithms. See *“SwiftTest FOX,”* pg. 135.

SwiftTest-MAX

Applies real-time test monitoring actions in the form of configurable triggers, trigger actions and alerts. Sophisticated monitoring actions can be easily scripted to optimize test and retest strategies. SwiftTest can be configured to monitor specific conditions such as yields, site-to-site variance, hardware and software bins, counts, statistics, trends, etc. See *“SwiftTest MAX – Triggers,”* pg. 15.

SwiftTest-DTI

The Dynamic Test Interface used to support a variety of miscellaneous adaptive test algorithms that cannot be categorized specifically under monitoring, test time optimization or outlier detection. Examples include Dynamic Site-site Calibration (DSC) which can automatically correct certain forms of site-to-site variation on the fly and thus recover test yield losses. Users can develop their own algorithms that plug into this interface.

SwiftTest-PAT

PAT is a statistically based method for removing parts with abnormal characteristics. SwiftTest's PAT uses accepted methodology and takes into account process variation from one wafer to the next or one test setup to the next. SwiftTest takes a baseline number of measurements, then calculates and applies new PAT limits based on the current wafer/lot being tested, then assigns "PAT-bins" to the outliers. The system allows you to implement PAT in real-time for both packaged parts and wafers, and as a post-process for wafers.

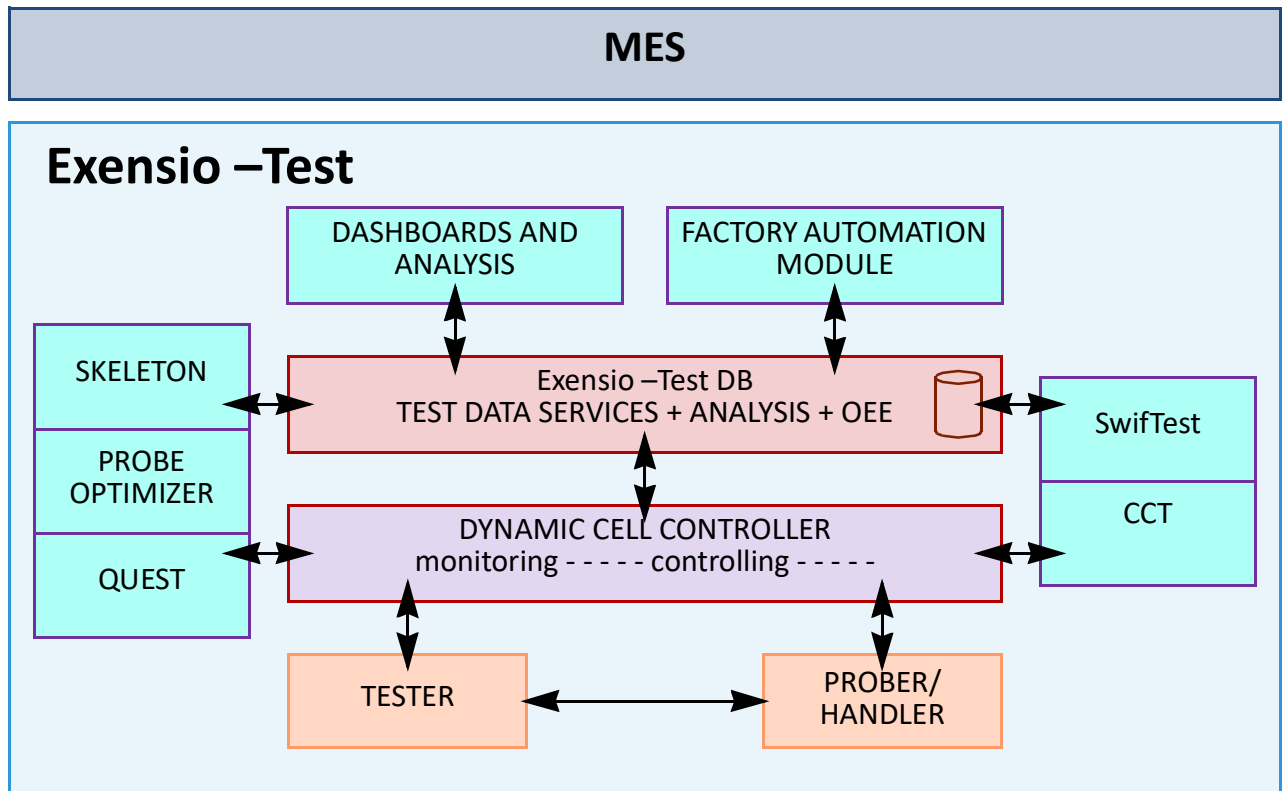
See "[Part Average Testing \(PAT\) With FOX](#)," pg. 155, regarding PAT within the FOX tool and "[SwiftTest Part Average Testing \(PAT\)](#)," pg. 189, regarding PAT as a stand-alone tool.

Remote Command Interface

SwiftTest includes a set of remote commands that can be used when integrating with external systems. Examples of available commands include Abort Lot, Begin Lot Monitoring Session, Begin Sampling Session, See "[SwiftTest Commands](#)," pg. 119.

EXENSIO –TEST

SwiftTest is part of Exensio –Test, PDF Solutions’ comprehensive test solution including real-time adaptive test, OEE analysis, test/prober cell control, and other capabilities to drive test productivity and quality.



EXENSIO –TEST COMPONENTS AND SYSTEM DIAGRAM

CHAPTER 2

SWIFTTEST MAX – TRIGGERS

TRIGGERS AND TRIGGER ACTIONS

INTRODUCTION

Incorporating triggers and trigger actions into the test process is an innovative feature of SwiftTest-MAX. Normally tied to lot-processing events, triggers signal SwiftTest-MAX to perform a task or trigger action.

Triggers enable users to isolate and identify specific anomalies and to take appropriate action. Simply put, it finds the problem then solves it. This can be particularly beneficial, for example, when reports indicate above normal and/or recurring failure of devices traced to a specific manufacturer.

First, define the problem. When writing the device profile, include any or all conditions that require attention. Test results, tester state, lot and wafer events, yield, and bin results are some examples of trigger conditions.

Second, define the solution: In the same device profile, define the required action. Sending alerts, stopping sampling, and writing reports are examples of trigger actions.

Think of triggers as thresholds for action. Triggers are how quality controllers figuratively draw a line in the sand. Conditions that exceed the threshold (cross the line) activate or *trigger* certain actions. Exact conditions and actions depend on the design of the trigger. This allows ultimate flexibility in the events that can set off a trigger and the actions that take place after the trigger has fired.

Conditions under which the action is to occur and the action itself are an integral part of the design for some triggers. For other triggers, the related concepts of thresholds and trigger actions define either the condition or the action. Each trigger defined in this document will include specific examples.

Trigger Profile Entry Example

```
<Feature class="com.pintail.swifttest.feature.trigger.TriggerFeatureProfile"
enabled="true">
  <TriggerMethodSet>
    <TriggerMethod type="LotEndTrigger" enabled="true">
      <TriggerActions>
        <TriggerAction type='ExecuteSystemCommand'>
          <Command commandPath="/bin/ls.exe" parameters="-la"
            outputPath="/usr/" />
        </TriggerAction>
      </TriggerActions>
    </TriggerMethod>
  </TriggerMethodSet>
</Feature>
```

The above profile trigger entry sets up a system command executed by a LotEnd event from SwiftTest-MAX.

Profile XML Specification

Element	type	Default	Validation	Description
Feature	tag			Begin trigger feature entry
Class	text			The name of the trigger Feature Class
TriggerMethodSet	tag			Begin trigger method entries
TriggerMethod	tag			Begin a single trigger method entry
type	text			Name of trigger method
TriggerActions	tag			Begin trigger action entries to be executed when this trigger is fired
TriggerAction	tag			Begin a single trigger action entry
type	text			Name of trigger action
Command	tag			Optional command executed by trigger action
commandPath	text			Path to the command to be executed
parameters	text			Parameters to be passed to the command on the command line
outputPath	text			The location where output from the command will be sent

TRIGGER PROFILE TAG AND ATTRIBUTE NAMES.

PROCESS RULE TRIGGERS AND EXAMPLES

BinCountTrigger

The **BinCountTrigger** fires when the specified bin or set of bins counted exceeds the threshold defined in the count parameter.

BinCountTrigger Element Specifications:

Element	type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration.
type	text	BinCountTrigger		The type of trigger being configured
name	text			User-defined trigger name
enabled	bool	false	true/false	Trigger is enabled
baselineEnabled	bool	true	true/false	Require baseline devices tested before arming trigger
baseline	int	1		Number of devices tested before arming trigger
count	int	100		Total bin count of all configured Hardware and Software bins required to fire the trigger. This count resets at each arm/re-arm point.
windowSize	int	0		Length of the rolling window to compute bin counts. A value of 0 means there is no window, include all devices. Setting the windowSize attribute will define the size of a rolling window of data used to evaluate this trigger.
bySite	bool	false	true/false	(Optional.) Setting this triggers bySite attribute to true will treat each site population independently.
resetEnabled	bool	false		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
resetResultCount	int	20		Minimum number of devices tested before trigger is rearmed. SwiftTest-MAX will test the number of devices defined in the baseline/resetResultCount parameter before arming/rearming the trigger.
maxResetCount	int	-1		Maximum number of times this trigger can re-arm. A value of -1 means no maximum; the trigger can re-arm indefinitely.
Bins	tag			Begin bin definition section, multiple bins may be defined
Bin	tag			Define a single watched bin
type	text	"Hardware" / "Software"	case insensitive	Hardware or software bin
number	int			Number of each hardware or software bin to monitor
ExBins	tag			Define bins to be excluded
type	text	"Hardware" / "Software"	case insensitive	Hardware or software bin
bin number	int			Number of each hardware or software bin to be excluded; one line per bin

Element	type	Default	Validation	Description
DisabledSiteList	tag			Begin list of sites to be disabled
DisabledSite	int			Site number to be disabled.
message	text			Trigger firing message to pass to Exensio database/SC
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag			Begin a single trigger action configuration

BinCountTrigger Profile Example

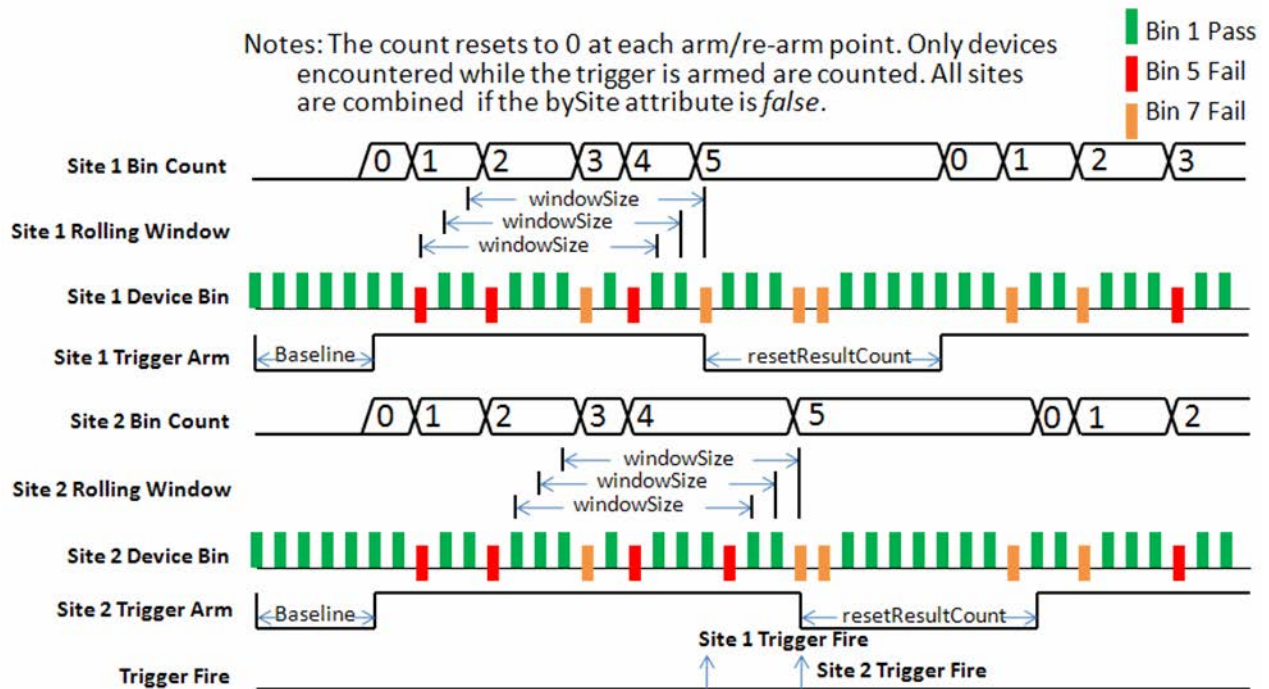
```

<TriggerMethod type="BinCountTrigger" name="MyBinCountTrigger: HBIN3"
enabled="true" triggerLevel="testrun" baselineEnabled="true" baseline="6"
count="5" windowSize="11" bySite="true" resetEnabled="true"
resetResultCount="10" maxResetCount="1" deviceGroup="all" >
  <Bins>
    <Bin type="Software" number="5"/>
    <Bin type="Software" number="7"/>
  </Bins>
  <ExBins>
    <Bin number="2050" type="software"/>
    <Bin number="2080" type="software"/>
  </ExBins>
  <DisabledSiteList>
    <DisabledSite number="0" />
    <DisabledSite number="2" />
  </DisabledSiteList>
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog" message="### write message
      trigger action log test message ###"/>
  </TriggerActions>
</TriggerMethod>

```



The system will not report bins if the count is zero.



TIMING DIAGRAM FOR BINCOUNTTRIGGER PROFILE EXAMPLE

BinPercentTrigger

The **BinPercentTrigger** fires when the percentage of parts in a watched bin or group exceeds the upper and/or lower control limits as defined by the **minimumPercent** and **maximumPercent** parameters.

Bin PercentTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
type	text	BinPercentTrigger		The type of trigger being configured
name	text			User-defined trigger name
enabled	bool	false	true/false	Trigger is enabled
baselineEnabled	bool	true	true/false	Require baseline devices tested before arming trigger
baseline	int	1		Number of devices tested before arming trigger

Element	Type	Default	Validation	Description
windowSize	int	0		Length of the rolling window to compute bin counts. A value of 0 means there is no window, include all devices. Setting the windowSize attribute will define the size of a rolling window of data used to evaluate this trigger.
bySite	bool	false	true/false	(Optional.) Setting this triggers bySite attribute to true will treat each site population independently.
maximumPercent	float	100		Maximum bin percent threshold
minimumPercent	float	0		Minimum bin percent threshold
resetEnabled	bool	false		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
resetResultCount	int	20		Minimum number of devices tested before trigger is re-armed. SwiftTest-MAX will test the number of devices defined in the baseline/resetResultCount parameter before arming/rearming the trigger.
maxResetCount	int	-1		Maximum number of times this trigger can re-arm. A value of -1 means no maximum, the trigger can re-arm indefinitely.
Bins	tag			Begin bin definition section, multiple bins can be defined
Bin	tag			Define a single watched bin
type	text	"Hardware" / "Software"	case insensitive	Hardware or software bin
bin number	int			Number of each hardware or software bin to monitor
ExBins	tag			Define bins to be excluded
type	text	"Hardware" / "Software"	case insensitive	Hardware or software bin
bin number	int			Number of each hardware or software bin to be excluded; one line per bin
DisabledSiteList	tag			Begin list of sites to be disabled
DisabledSite	int			Site number to be disabled.
message	text			Trigger firing message to pass to Exensio database/SC
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag			Begin a single trigger action configuration

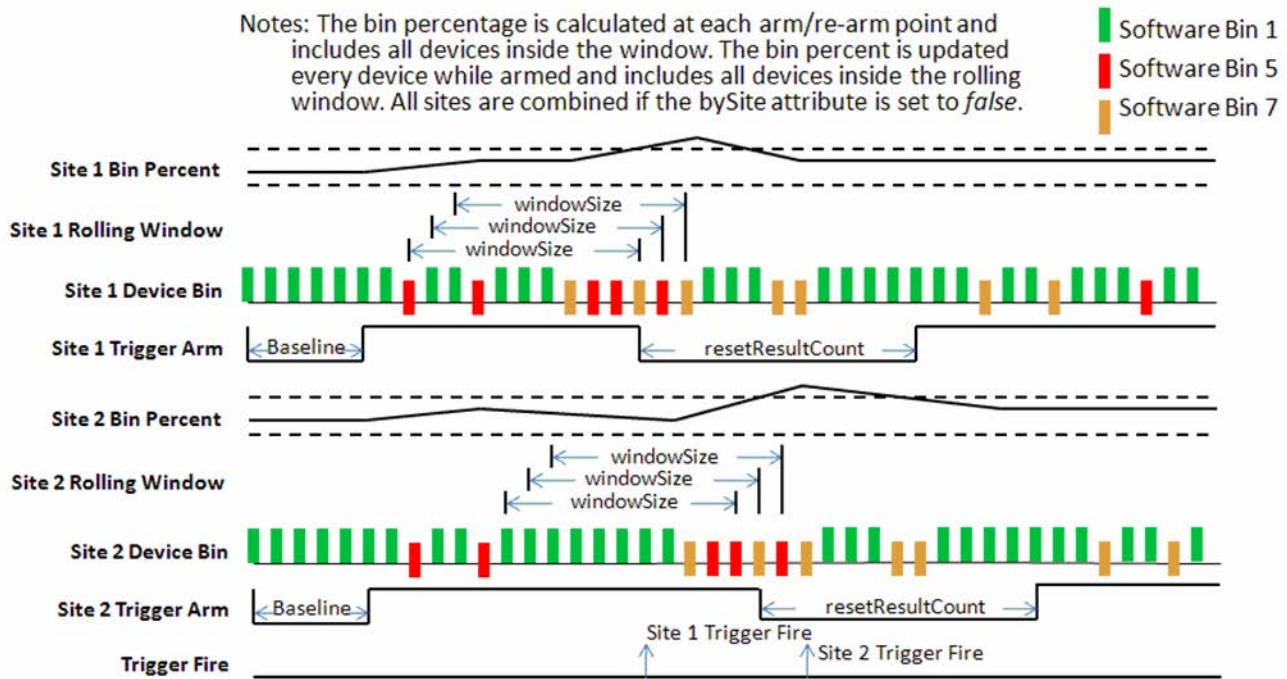
BinPercentTrigger Profile Example

```
<TriggerMethod type="BinPercentTrigger" name="MyBinPercentTrigger"
enabled="true" triggerLevel="testrun" baselineEnabled="true" baseline="50"
windowSize="11" bySite="true" maximumPercent="10" resetEnabled="true"
resetResultCount="10" maxResetCount="1" deviceGroup="all">
  <Bins>
    <Bin type="Software" number="5"/>
    <Bin type="Software" number="7"/>
  </Bins>
  <ExBins><<
    <Bin number="2050" type="software"/>
    <Bin number="2080" type="software"/>
  </ExBins>
  <DisabledSiteList>
    <DisabledSite number="0" />
    <DisabledSite number="2" />
  </DisabledSiteList>
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog"
      message="### write message trigger action log test message ###">

    <TriggerAction>
  </TriggerActions>
</TriggerMethod>
```



The system will not report bins if the percent is zero.



TIMING DIAGRAM FOR BINPERCENTTRIGGER PROFILE EXAMPLE

ConsecutiveBinTrigger

The **ConsecutiveBinTrigger** fires when the number of consecutive parts in a watched bin or group of bins exceeds the threshold defined in the **maxFrequency** parameter.

ConsecutiveBinTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
type	text	ConsecutiveBinTrigger		The type of trigger being configured
name	text			User-defined trigger name
enabled	bool	false	true/false	Trigger is enabled
baselineEnabled	bool	true	true/false	Require baseline devices tested before arming trigger
baseline	int	1		Number of devices tested before arming trigger
bySite	bool	false	true/false	(Optional.) Setting this triggers bySite attribute to true will treat each site population independently.
maxFrequency	int	20		Number of consecutive bins to cause trigger to fire

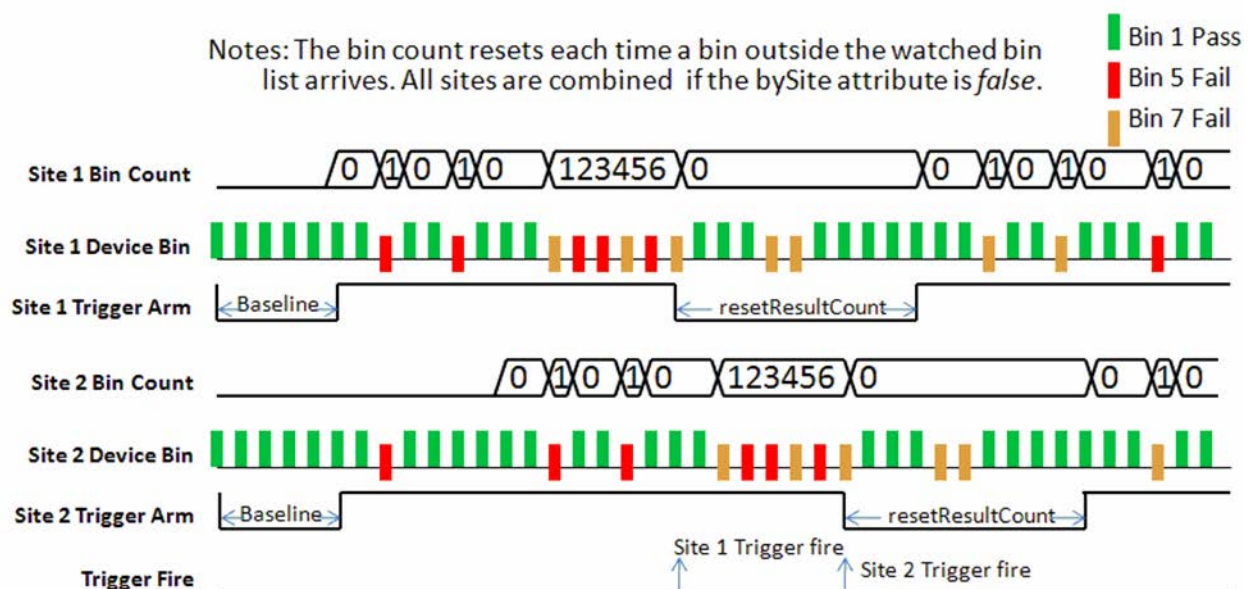
Element	Type	Default	Validation	Description
resetEnabled	bool	false		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
resetResultCount	int	10		Minimum number of devices tested before trigger is re-armed. SwiftTest-MAX will test the number of devices defined in the baseline/resetResultCount parameter before arming/rearming the trigger.
maxResetCount	int	-1		Maximum number of times this trigger can re-arm. A value of -1 means no maximum, the trigger can re-arm indefinitely.
Bins	tag			Begin bin definition section, multiple bins can be defined
Bin	tag			Define a single watched bin
type	text	"Hardware" / "Software"	case insensitive	Hardware or software bin
number	int			Number of each hardware or software bin to monitor
DisabledSiteList	tag			Begin list of sites to be disabled
DisabledSite	int			Site number to be disabled.
ExBins	tag			Define bins to be excluded
type	text	"Hardware" / "Software"	case insensitive	Hardware or software bin
bin number	int			Number of each hardware or software bin to be excluded; one line per bin
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
message	text			Trigger firing message to pass to Exensio database/SC
TriggerAction	tag			Begin a single trigger action configuration

ConsecutiveBinTrigger Profile Example

```
<TriggerMethod type="ConsescutiveBinTrigger" name="MyConsecutiveBinTrigger"
enabled="true" triggerLevel="testrun" baselineEnabled="true" baseline="6"
maxFrequency="6" resetEnabled="true" resetResultCount="20" maxResetCount="1"
bySite="True" deviceGroup="all">
  <Bins>
    <Bin type="Software" number="5"/>
    <Bin type="Software" number="7"/>
  </Bins>
  <ExBins><<
    <Bin number="2050" type="software"/>
    <Bin number="2080" type="software"/>
  </ExBins>
  <DisabledSiteList>
    <DisabledSite number="0" />
    <DisabledSite number="2" />
  </DisabledSiteList>
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog"
      message="### write message trigger action log test message ###">
    </TriggerAction>
  </TriggerActions>
</TriggerMethod>
```



The system will not report bins if the count is zero.



TIMING DIAGRAM FOR CONSECUTIVEBINTRIGGER PROFILE EXAMPLE

ParametricTestSpcTrigger

The **ParametricTestSpcTrigger** fires when the parametric test results for the configured test violate the configured WECO (Western Electric Company Rules) rule.

ParametricTestSpcTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
type	text	ParametricTestSpcTrigger		The type of trigger being configured
name	text			User-defined trigger name
enabled	bool	false	true/false	Trigger is enabled
baselineEnabled	bool	true	true/false	Require baseline devices tested before arming trigger
baseline	int	1		Number of devices tested before arming trigger
windowSize	int	0		Length of the rolling window to compute bin counts. A value of 0 means there is no window, include all devices. Setting the windowSize attribute will define the size of a rolling window of data used to evaluate this trigger.
bySite	bool	false	true/false	(Optional.) Setting this triggers bySite attribute to true will treat each site population independently.
testNumber	int			Number of the monitored parametric test
subTestNumber	int			Subtest number of the monitored parametric test
description	text			The name of the parametric/functional test
resetEnabled	bool	false		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
resetResultCount	int	20		Minimum number of devices tested before trigger is re-armed. SwiftTest-MAX will test the number of devices defined in the baseline/resetResultCount parameter before arming/rearming the trigger.

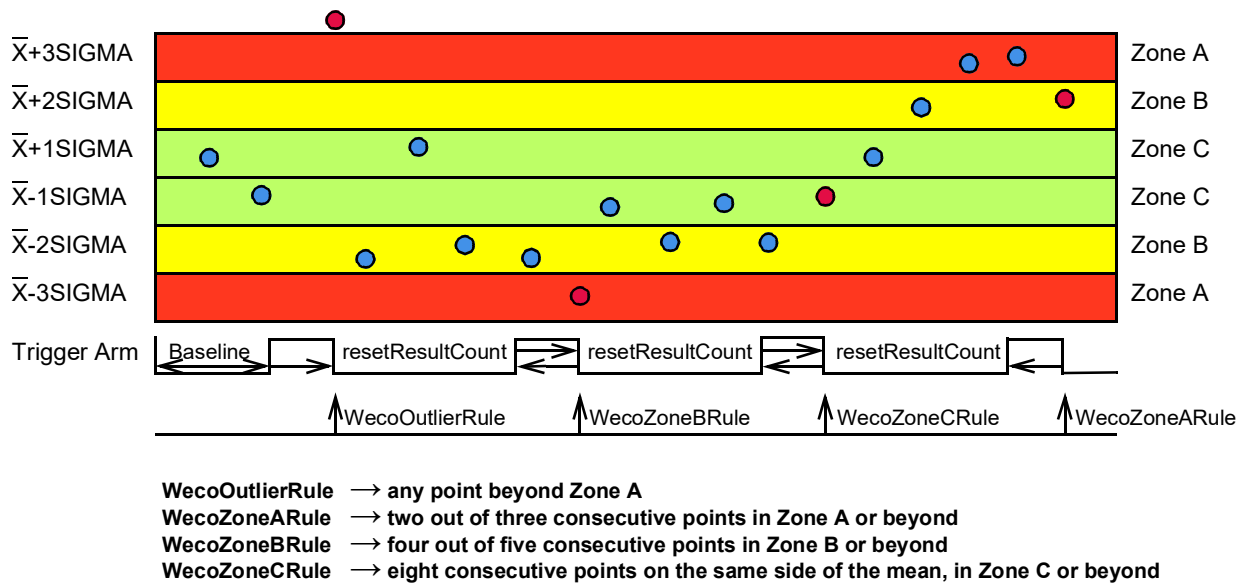
Element	Type	Default	Validation	Description
maxResetCount	int	-1		Maximum number of times this trigger can re-arm. A value of -1 means no maximum, the trigger can re-arm indefinitely.
RuleSet	tag			WECO rule definition set
Rule	tag			Define a single WECO rule
type	text		wecozonearule wecozonebrule wecozonecrule wecooutlierrule wecooscillatingrule wecotrendrule	The specific WECO rule applied
message	text			Trigger firing message to pass to Exensio database/SC
DisabledSiteList	tag			Begin list of sites to be disabled
DisabledSite	int			Site number to be disabled.
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag			Begin a single trigger action configuration

ParametricTestSpcTrigger Profile Example

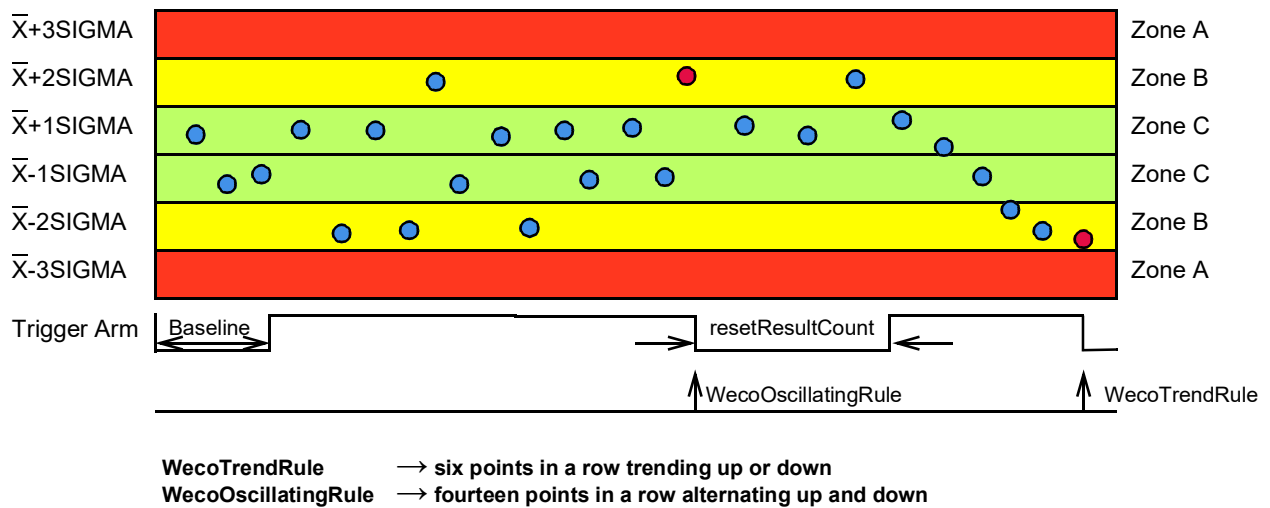
```

<TriggerMethod type="ParametricTestSpcTrigger"
name="MyParametricTestSpcTrigger" enabled="true" triggerLevel="testrun"
baselineEnabled="true" baseline="50" bySite='true' resetEnabled="true"
resetResultCount="3" maxResetCount="1" description='Test 100 MyTest100Name'
deviceGroup="all">
  <RuleSet>
    <Rule type="WECOOutlier"/>
    <Rule type="WECOZoneARule"/>
  </RuleSet>
  <DisabledSiteList>
    <DisabledSite number="0" />
    <DisabledSite number="2" />
  </DisabledSiteList>
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog"
      message="### write message trigger action log test message ###">
    <TriggerAction>
  </TriggerActions>
</TriggerMethod>

```



TIMING DIAGRAM FOR PARAMETRICTESTSPCTRIGGER ZONE RULES PROFILE EXAMPLE

TIMING DIAGRAM FOR
PARAMETRICTESTSPCTRIGGER TREND/OSCILLATING RULES PROFILE EXAMPLE

ParametricTestStatisticTrigger

The **ParametricTestStatisticTrigger** fires when the parametric test statistic for the configured parameter exceeds the allowed range defined by the **minimum** and **maximum** parameters.

ParametricTestStatisticTrigger Element Specifications:

Element	Type	Default	validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
type	text	ParametricTestStatisticTrigger		The type of trigger being configured
name	text			User-defined trigger name
enabled	bool	false	true/false	Trigger is enabled
baselineEnabled	bool	true	true/false	Require baseline devices tested before arming trigger
baseline	int	1		Number of devices tested before arming trigger
windowSize	int	0		Length of the rolling window to compute bin counts. A value of 0 means there is no window, include all devices. Setting the windowSize attribute will define the size of a rolling window of data used to evaluate this trigger.
bySite	bool	false	true/false	(Optional.) Setting this triggers bySite attribute to true will treat each site population independently.
testNumber	int			Number of the monitored parametric test
subTestNumber	int			Subtest number of the monitored parametric test
description	text			The name of the parametric/functional test
minimum	float	0		Minimum percent firing threshold
maximum	float	100		Maximum percent firing threshold

Element	Type	Default	validation	Description
statistic	text		mean median min max standard_deviation cp cpk robust_standard_deviation robust_cp robust_cpk	Parametric test statistic to be used. “statistic” or “statisticFormula” attribute must be provided; if missing, SwiftTest will disable the rule and log an ERROR message. - Priority is given to “statistic” attribute.
statisticFormula	text		mean median min max standard_deviation robust_standard_deviation cp cpk robust_cp robust_cpk	statisticFormula supports macros, all math operations and parentheses — examples: $\$(max) - \$(mean)$ $\$(mean) - \$(min)) /$ $\$(max) * 1.35)$ “statistic” or “statisticFormula” attribute must be provided; if missing, SwiftTest will disable the rule and log an ERROR message. - Priority is given to “statistic” attribute. - SwiftTest will validate “statisticFormula” attribute, and if it contains an invalid macro, the rule will be disabled and an ERROR message will be logged. For invalid syntax, like missing parentheses, SwiftTest will disable the rule and log an ERROR message.
resetEnabled	bool	true		Enable multiple firings of the trigger. Setting this trigger’s resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.

Element	Type	Default	validation	Description
resetResultCount	int	20		Minimum number of devices tested before trigger is re-armed. SwiftTest-MAX will test the number of devices defined in the baseline/resetResultCount parameter before arming/rearming the trigger.
maxResetCount	int	-1		Maximum number of times this trigger can re-arm. A value of -1 means no maximum, the trigger can re-arm indefinitely.
message	text			Trigger firing message to pass to Exensio database/SC
DisabledSiteList	tag			Begin list of sites to be disabled
DisabledSite	int			Site number to be disabled.
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag			Begin a single trigger action configuration.

ParametricTestStatisticTrigger Profile Example (with statistic element)

```

<TriggerMethod type="ParametricTestStatisticTrigger"
name="myParametricTestStatisticTrigger" enabled="true" triggerLevel="testrun"
baselineEnabled="true" baseline="6" testNumber="100" subTestNumber="1"
description="Test 100 MyTest100Name" statistic="mean" minimum="0.4"
maximum="1.4" resetEnabled="true" resetResultCount="10" maxResetCount="1"
windowSize="10" bySite="true" deviceGroup="all">
  <DisabledSiteList>
    <DisabledSite number="0"/>
    <DisabledSite number="2"/>
  </DisabledSiteList>
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog" message="### write message trigger
action log test message ###">
    </TriggerAction>
  </TriggerActions>
</TriggerMethod>

```

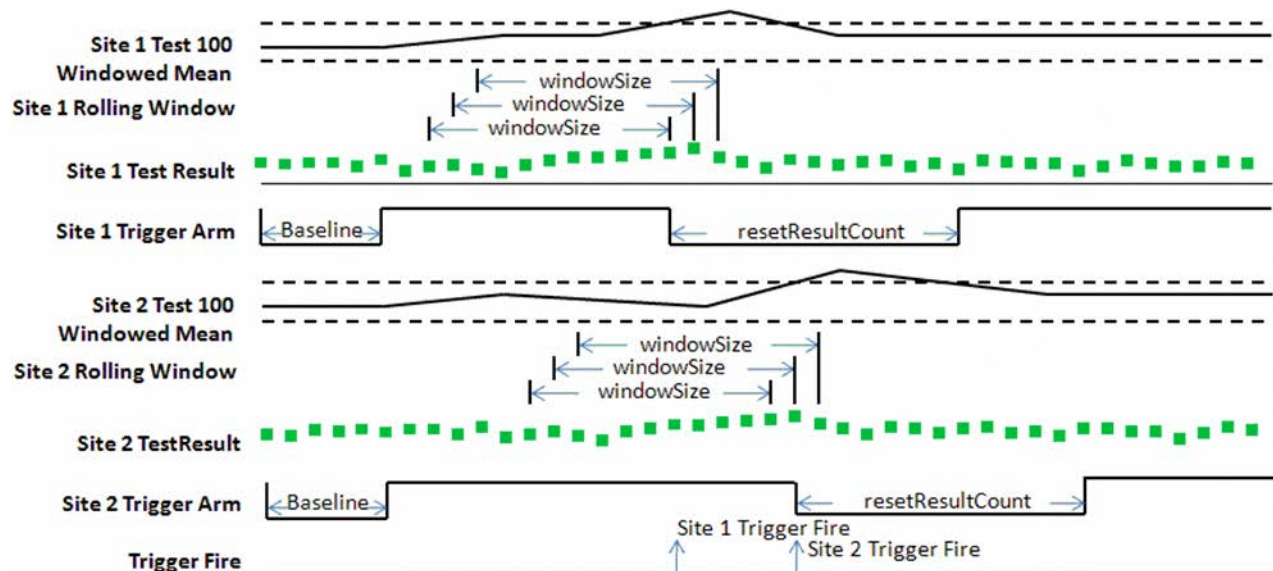
ParametricTestStatisticTrigger Profile Example (with statisticFormula element)

```

<TriggerMethod type="ParametricTestStatisticTrigger"
name="myParametricTestStatisticTrigger" enabled="true" triggerLevel="testrun"
baselineEnabled="true" baseline="6" testNumber="100" subTestNumber="1"
description="Test 100 MyTest100Name" statisticFormula="$ (max) - $ (mean) "
minimum="0.2" maximum="0.4" resetEnabled="true" resetResultCount="10"
maxResetCount="1" windowSize="10" bySite="true" deviceGroup="all">
  <DisabledSiteList>
    <DisabledSite number="0"/>
    <DisabledSite number="2"/>
  </DisabledSiteList>
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog" message="### write message trigger
    action log test message ###">
  </TriggerAction>
</TriggerActions>
</TriggerMethod>

```

Notes: The parametric statistic is calculated at each arm/rearm point and includes all devices inside the rolling window. After every device is tested, the statistic is recalculated and includes all measured values from all devices inside the rolling window. All sites are combined if the bySite attribute is set to *false*.



TIMING DIAGRAM FOR PARAMETRICTESTSTATISTICSTRIGGER PROFILE EXAMPLE

SiteToSiteParametricTestStatisticDeltaTrigger

The **SiteToSiteParametricTestStatisticDeltaTrigger** fires when the parametric test statistic differential between two or more test sites exceeds the minimum as defined in the **maximumValueDelta** parameter.

SiteToSiteParametricTestStatisticDeltaTrigger Element Specifications:



SwiftTest-MAX independently computes the percent delta for each siteA/siteB pair. If any site pair causes the trigger to fire, all site pairs will reset.

Element	Type	Default	Validation	Description
TriggerMethod	Tag		N/A	Denotes the beginning of a new trigger method configuration
type	Text	SiteToSiteParametricTestStatisticDeltaTrigger		The type of trigger being configured
name	Text			User-defined trigger name
enabled	Bool	True	True/False	Trigger is enabled
baselineEnabled	Bool	True	True/False	Require baseline devices tested before arming trigger
baseline	Int	20		Number of devices tested before arming trigger
maximumValueDelta	Float	0		The maximum allowed value difference between any two test sites
resetEnabled	Bool	True		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.

Element	Type	Default	Validation	Description
resetResultCount	Int	20		Minimum number of devices tested before trigger is re-armed. SwiftTest-MAX will test the number of devices defined in the baseline/resetResultCount parameter before arming/rearming the trigger.
maxResetCount	Int	-1		Maximum number of times this trigger can re-arm. A value of -1 means re-arm indefinitely
windowSize	Int	0		Length of the rolling window to compute bin counts. A value of 0 means there is no window, include all devices. Setting the windowSize attribute will define the size of a rolling window of data used to evaluate this trigger.
testNumber	Int			Test number of the monitored parametric test
subTestNumber	Int			Sub-test number of the monitored parametric test
description	Text			The name of the parametric/functional test
statistic	text		mean median min max standard_deviation cp cpk robust_standard_deviation robust_cp robust_cpk	Parametric test statistic to be used. • “statistic” or “statisticFormula” attribute must be provided; if missing, SwiftTest will disable the rule and log an ERROR message. • Priority is given to “statistic” attribute.

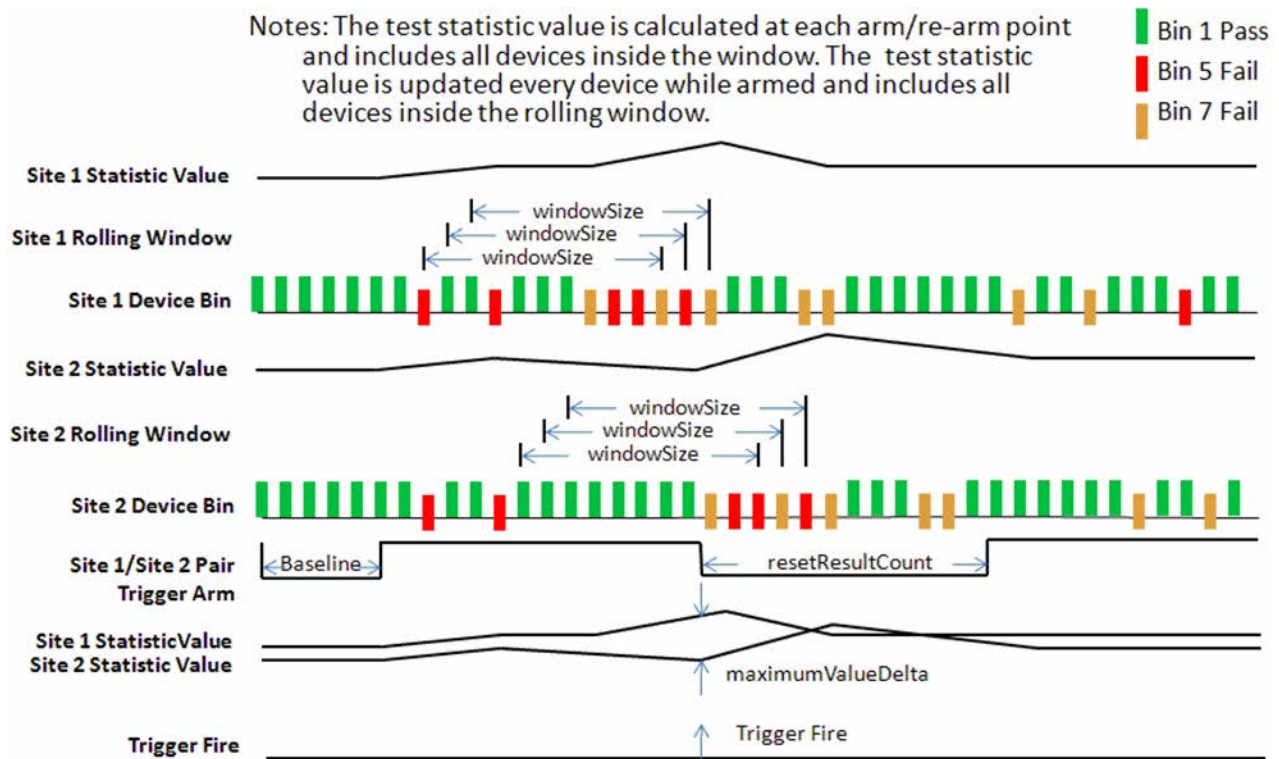
Element	Type	Default	Validation	Description
statisticFormula	text		mean median min max standard_deviation robust_standard_deviation cp cpk robust_cp robust_cpk	statisticFormula supports macros, all math operations and parentheses — examples: $\$(max) - \$(mean)$ $(\$(mean) - \$(min)) / (\$(max) * 1.35)$ • “statistic” or “statisticFormula” attribute must be provided; if missing, SwiftTest will disable the rule and log an ERROR message. • Priority is given to “statistic” attribute. • SwiftTest will validate “statisticFormula” attribute, and if it contains an invalid macro, the rule will be disabled and an ERROR message will be logged. For invalid syntax, like missing parentheses, SwiftTest will disable the rule and log an ERROR message.
message	Text			Trigger firing message to pass to Exensio database/SC
DisabledSiteList	tag			Begin list of sites to be disabled
DisabledSite	int			Site number to be disabled.
TriggerActions	Tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	Tag			Begin a single trigger action configuration.

SiteToSiteParametricTestStatisticDeltaTrigger Profile example (with statistic element)

```
<TriggerMethod type="SiteToSiteParametricTestStatisticDeltaTrigger"
name="mySiteToSiteParametricTestStatisticDeltaTrigger" enabled="true"
triggerLevel="testrun" baselineEnabled="true" baseline="6" testNumber="100"
subTestNumber="1" description="Test 100 MyTest100Name" statistic="mean"
maximumValueDelta="0.5" resetEnabled="true" resetResultCount="10"
maxResetCount="1" windowSize="10" bySite="true" deviceGroup="all">
  <DisabledSiteList>
    <DisabledSite number="0"/>
    <DisabledSite number="2"/>
  </DisabledSiteList>
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog" message="### write message trigger
action log test message ###">
    </TriggerAction>
  </TriggerActions>
</TriggerMethod>
```

SiteToSiteParametricTestStatisticDeltaTrigger Profile example (with statisticFormula element)

```
<TriggerMethod type="SiteToSiteParametricTestStatisticDeltaTrigger"
name="mySiteToSiteParametricTestStatisticDeltaTrigger" enabled="true"
triggerLevel="testrun" baselineEnabled="true" baseline="6" testNumber="100"
subTestNumber="1" description="Test 100 MyTest100Name"
statisticFormula="$(max) - $(mean)" maximumValueDelta="0.5"
resetEnabled="true" resetResultCount="10" maxResetCount="1" windowSize="10"
bySite="true" deviceGroup="all">
  <DisabledSiteList>
    <DisabledSite number="0"/>
    <DisabledSite number="2"/>
  </DisabledSiteList>
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog" message="### write message trigger
action log test message ###">
    </TriggerAction>
  </TriggerActions>
</TriggerMethod>
```



TIMING DIAGRAM FOR SITEToSITEPARAMETRICTESTSTATISTICDELTA TRIGGER IN PROFILE EXAMPLE

ParametricTestYieldTrigger

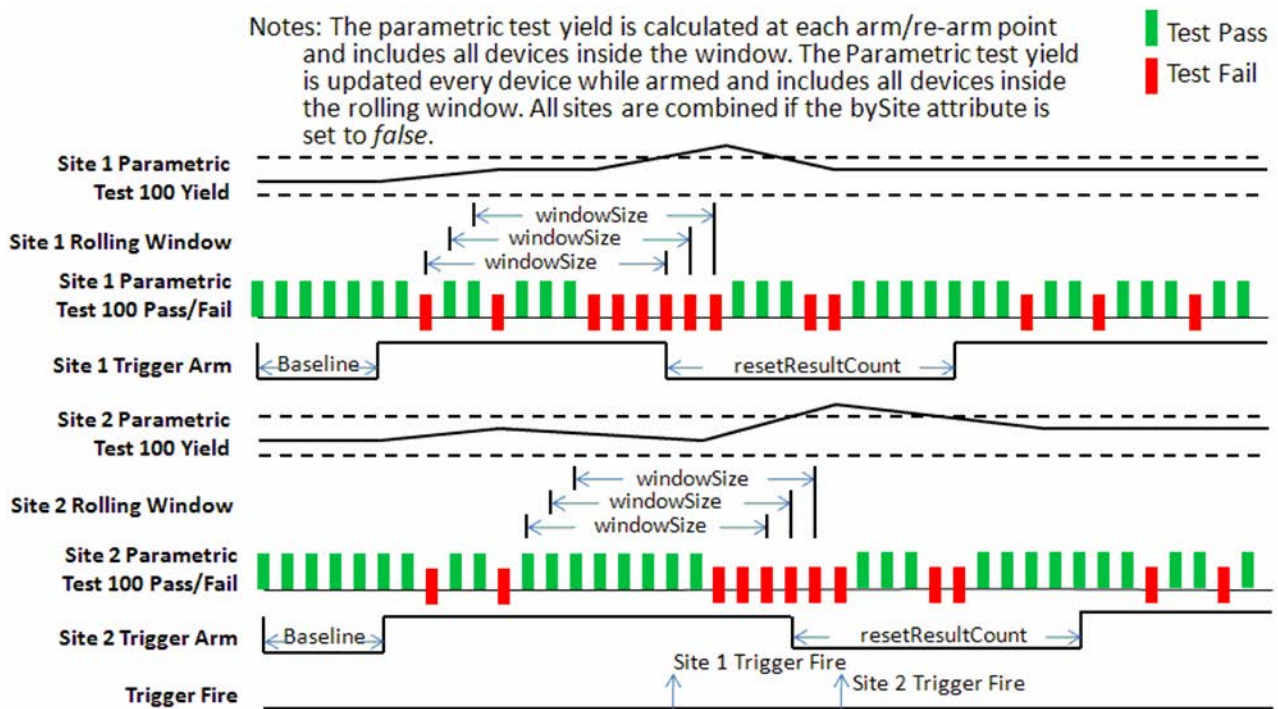
ParametricTestYieldTrigger fires when the percentage of passing devices for the defined parametric test exceeds the limits as defined by the **minimumPercent** and **maximumPercent** parameters.

ParametricTestYieldTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
type	text	ParametricTestYieldTrigger		The type of trigger being configured
name	text			User-defined trigger name
enabled	bool	false	true/false	Trigger is enabled
baselineEnabled	bool	true	true/false	Require baseline devices tested before arming trigger
baseline	int	1		Number of devices tested before arming trigger
windowSize	int	0		Length of the rolling window to compute bin counts. A value of 0 means there is no window, include all devices. Setting the windowSize attribute will define the size of a rolling window of data used to evaluate this trigger.
bySite	bool	false	true/false	(Optional.) Setting this triggers bySite attribute to true will treat each site population independently.
minimumPercent	int	0		Minimum percent firing threshold
maximumPercent	int	100		Maximum percent firing threshold
testNumber	int			Number of the monitored parametric test
subTestNumber	int			Subtest number of the monitored parametric test
description	text			The name of the parametric/functional test
resetEnabled	bool	true		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
resetResultCount	int	20		Minimum number of devices tested before trigger is re-armed. SwiftTest-MAX will test the number of devices defined in the baseline/resetResultCount parameter before arming/rearming the trigger.
maxResetCount	int	-1		Maximum number of times this trigger can re-arm. A value of -1 means no maximum, the trigger can re-arm indefinitely.
message	text			Trigger firing message to pass to Exensio database/SC
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag			Begin a single trigger action configuration.

ParametricTestYieldTrigger Profile Example

```
<TriggerMethod type="ParametricTestYieldTrigger"
name="MyParametricTestYieldTrigger" enabled="true" triggerLevel="testrun"
baselineEnabled="true" baseline="6" minimumPercent="60" maximumPercent="90"
resetEnabled="true" resetResultCount="10" maxResetCount="1" testNumber="100"
subtestNumber="1" window="10" bySite="true" description="Test 100
MyTest100Name" deviceGroup="all">
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog"
      message="### write message trigger action log test message ###">
  </TriggerActions>
</TriggerMethod>
```



TIMING DIAGRAM FOR PARAMETRICTESTYIELDTRIGGER PROFILE EXAMPLE

Multiple Tests/ Regular Expressions

The ParametricTestYieldTrigger also supports multiple tests and regular expressions for descriptions (in the **<TriggerTests>** section, one per line).

Example:

```
<TriggerMethod type="ParametricTestYieldTrigger" ...>
  <TriggerActions>
    ...
  </TriggerActions>
  <TriggerTests>
    <Test testNumber="1" subTestNumber="0" description="..." minimumPercent="0" maximumPercent="89.99" />
    <Test descriptionRegex=".*" minimumPercent="0" maximumPercent="89.99" />
    <Test description="..." minimumPercent="0" maximumPercent="89.99" />
  </TriggerTests>
</TriggerMethod>
```

EcidTrigger

The **EcidTrigger** fires on every test run and will put together an ECID from parameters in one or multiple STDF.PTR records. Literal text can also be injected to make the ECID readable and to add separators. Additionally, the system can also get the ECID from the PART_TEXT and/or PART_ID fields in the data file. This trigger writes the ECID, verifies that it is a valid ECID and it also checks for recurring ECIDs.

EcidTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	Tag		N/A	Denotes the beginning of a new trigger method configuration
type	Text	ParametricTestSetEcidTrigger		The type of trigger being configured
name	Text	PTSETTrigger		User-defined trigger name
enabled	Bool	True	True/False	Trigger is enabled
ecidSource	Text	N/A		Either "PARAMETRIC_TEST" or "PRR_RECORD"
bySite	bool	false	true/false	(Optional.) Setting this triggers bySite attribute to true will treat each site population independently.
validECIDRegex	regex	no default		Required. Regex pattern definition of ECID format. If it is missing, the trigger will be disabled.
CheckRecurring				Checks for recurring ECIDs.
enabled	Bool	True	True/False	CheckRecurring is enabled
minRecurrence	Int	1	1-10	Minimum number of recurring ECIDs before the trigger fires.
passingBinOnly	Bool	False	True/False	Don't check/write operate in ECID if it's not a passing device.
checkValidECIDsOnly	Bool	False	True/False	The ECID will be checked to see if it's valid; if not then will not check for recurring ECID.
PrrRecordSource	Tag		N/A	Source for PRR record.
ECIDFieldList	Tag			Optional. Holds the ECIDFieldList elements. If it is not there, SwiftTest will use the PRR.PART_TXT as the ECID field. If one or more ECIDFields are listed in the ECIDFieldList, SwiftTest will concatenate them together as an ECID field.
ECIDField name	Tag	text		Element that holds the PART_TEXT or PART_ID field that defines ECID.
ParametricTestSource	Tag			

Element	Type	Default	Validation	Description
numericToTextMap	Int			numericToTextMap “keys” can be any int value; they do NOT have to be contiguous, and do NOT have to start with “0”. See “numericToTextMap,” pg. 43.
ParametricTestList	Tag		N/A	The list of parametric tests used to compile the ECID.
ParametricTest	Tag		N/A	
testNumber	Int			Number of the monitored parametric test
subTestNumber	Int			Subtest number of the monitored parametric test
description	Text			The name of the parametric/functional test
TriggerActions	Tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	Tag			Begin a single trigger action configuration.

EcIdTrigger Profile Example

```

<TriggerMethod type="EcIdTrigger" name="EcIdTriggerName" enabled="true"
ecidSource="PARAMETRIC_TEST|PRR_RECORD" bySite="false" validECIDRegex="valid-
regex">
  <CheckRecurring enabled="true" minRecurrence="1" passingBinOnly="false"
  checkValidECIDsOnly="false|true"/>
  <PrrRecordSource>
    <ECIDFieldList>
      <ECIDField name="PART_TXT"/>
      <ECIDField name="PART_ID"/>
    </ECIDFieldList>
  </PrrRecordSource>
  <ParametricTestSource numericToTextMap="49:0,50:1,51:2,52:....."
  delimiter="">
    <ParametricTestList>
      <ParametricTest testNumber="1" subTestNumber="0"
      description="Test1 <> Group1" param="result">
        <Actions>
          <Action actionType="literal" value="LotId="/>
          <Action actionType="convert"/>
        </Actions>
      </ParametricTest>
      ....
    </ParametricTestList>
  </ParametricTestSource>
  <TriggerActions>
    <TriggerAction type='CreateEventReport' enabled='true'
    version="2">
      <OutputStrategies>

```

- Possible **ecidSource** values are PARAMETRIC_TEST or PRR_RECORD.
- Check valid ECIDs based on validECIDRegex.
When there is no validECIDRegex, the behavior is:
 - Parametric Results source: ECID is marked as invalid if one parametric result is unmapped.
 - PRR source: ECID is marked as invalid if **part_txt** or **part_id** is missing.
- Multi-sites supported. When **bySite=true**, SwifTest will look for recurring ECIDs on a site-by-site basis — e.g., the system will split the data by site, and monitor for recurring ECIDs on each site, independently.
- ECIDFields are PART_TXT or PART_ID, from PRR_Record.

numericToTextMap

The **numericToTextMap** section contains a comma-delimited string of colon-delimited key/value pairs. The first integer (key) in the pair represents the Parametric Test Result value sent by the tester in the STDF, the second alphanumeric value (value) in the pair represents the value that will be substituted for the key.

SiteToSiteBinPercentDeltaTrigger

The **SiteToSiteBinPercentDeltaTrigger** fires when the bin differential between two or more test sites exceeds the minimum as defined in the **maximumPercentDelta** parameter.



If a list of bins is defined, they are logically ANDed together when calculating the %. For example, if Bin1 and Bin2 are in the Bin list, then Site1Bin%= Site1Bin1%+Site1Bin2%.

SiteToSiteBinPercentDeltaTrigger Element Specifications:



SwiftTest-MAX independently computes the percent delta for each siteA/siteB pair. If any site pair causes the trigger to fire, all site pairs will reset.

Element	Type	Default	Validation	Description
TriggerMethod	Tag		N/A	Denotes the beginning of a new trigger method configuration
type	Text	SiteToSiteBinPercentDeltaTrigger		The type of trigger being configured
name	Text			User-defined trigger name
enabled	Bool	True	True/False	Trigger is enabled
baselineEnabled	Bool	True	True/False	Require baseline devices tested before arming trigger
baseline	Int	20		Number of devices tested before arming trigger
windowSize	Int	0		Length of the rolling window to compute bin counts. A value of 0 means there is no window, include all devices. Setting the windowSize attribute will define the size of a rolling window of data used to evaluate this trigger.
maximumPercentDelta	Float	5		The maximum allowed bin percentage difference between any two test sites

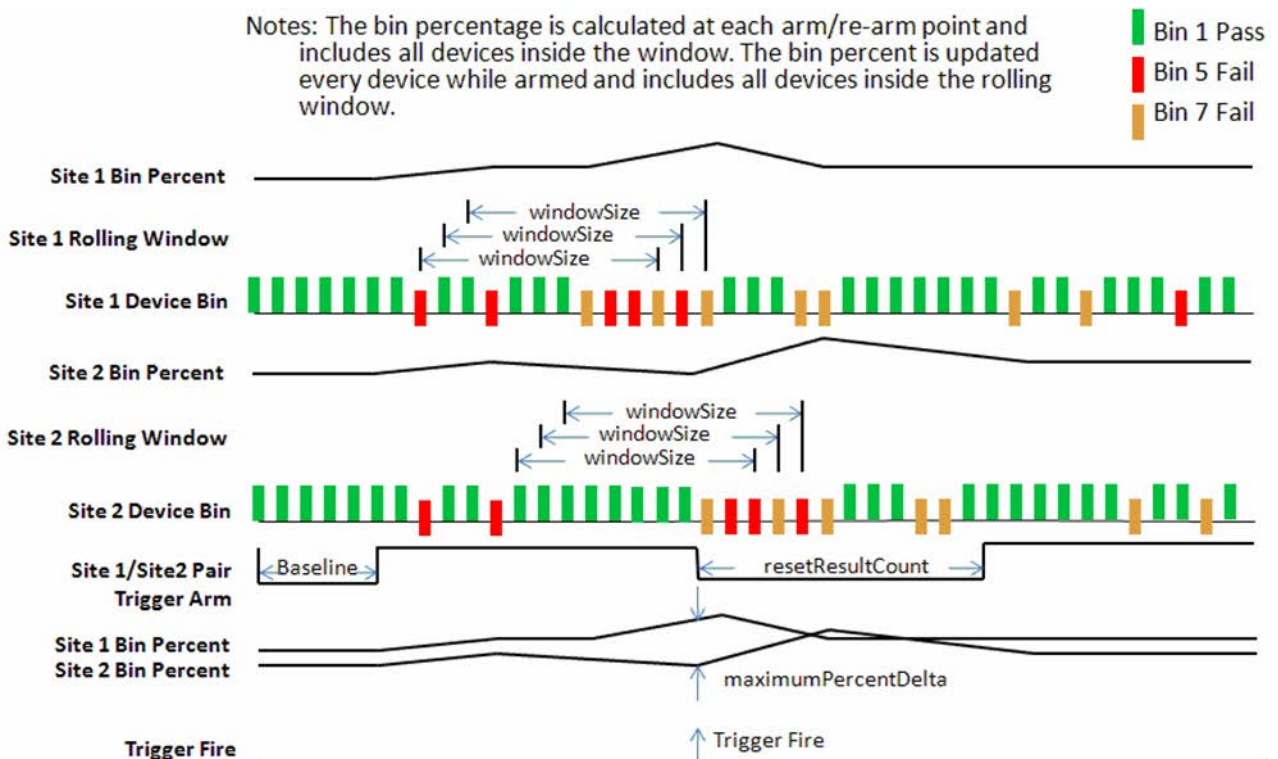
Element	Type	Default	Validation	Description
resetEnabled	Bool	True		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
resetResultCount	Int	20		Minimum number of devices tested before trigger is re-armed. SwiftTest-MAX will test the number of devices defined in the baseline/resetResultCount parameter before arming/rearming the trigger.
maxResetCount	Int	-1		Maximum number of times this trigger can re-arm. A value of -1 means re-arm indefinitely
Bins	Tag			Begin bin definition section, multiple bins can be defined. The list of Bins in this section are logically ANDed together when calculating the %. (See note at the top of this section.)
Bin	Tag			Define a single watched bin.
type	Text	"Hardware" / "Software"	Case insensitive	Hardware or software bin
number	Int			Number of each hardware or software bin to monitor
message	Text			Trigger firing message to pass to Exensio database/SC
DisabledSiteList	tag			Begin list of sites to be disabled
DisabledSite	int			Site number to be disabled.
TriggerActions	Tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	Tag			Begin a single trigger action configuration.

SiteToSiteBinPercentDeltaTrigger Profile Example

```
<TriggerMethod type="SiteToSiteBinPercentDeltaTrigger"
name="MySiteToSiteBinPercentDeltaTrigger" enabled="true"
triggerLevel="testrun" baselineEnabled="true" baseline="6"
maximumPercentDelta="5" resetEnabled="true" resetResultCount="10"
deviceGroup="all">
  <DisabledSiteList>
    <DisabledSite number="0" />
    <DisabledSite number="2" />
  </DisabledSiteList>
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog"
      message="### write message trigger action log test message ###">
    </TriggerAction>
  </TriggerActions>
</TriggerMethod>
```



The system will not report bins if the count is zero.



TIMING DIAGRAM FOR SITEToSITEBINPERCENTDELTA_TRIGGER IN PROFILE EXAMPLE

YieldTrigger

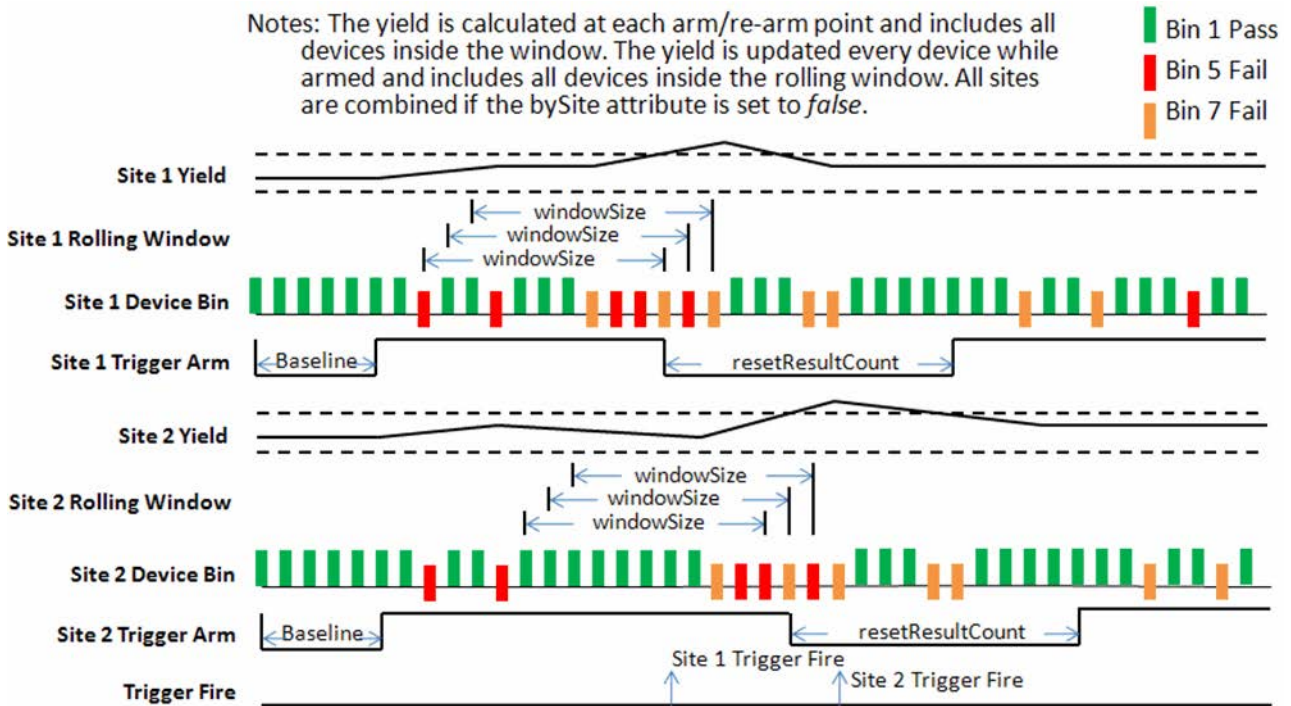
The **YieldTrigger** fires when the percentage of passing parts exceeds the configured range as defined by the **minimumPercent** and **maximumPercent** parameter.

YieldTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	Tag		N/A	Denotes the beginning of a new trigger method configuration
type	Text	YieldTrigger		The type of trigger being configured
name	Text			User-defined trigger name
enabled	Bool	False	True/False	Trigger is enabled
baselineEnabled	Bool	True	True/False	Require baseline devices tested before arming trigger
baseline	Int	1		Number of devices tested before arming trigger
windowSize	Int	0		Length of the rolling window to compute bin counts. A value of 0 means there is no window, include all devices. Setting the windowSize attribute will define the size of a rolling window of data used to evaluate this trigger.
bySite	Bool	False	True/False	(Optional.) Setting this triggers bySite attribute to true will treat each site population independently.
minimumPercent	Int	0		Minimum percent firing threshold
maximumPercent	Int	100		Maximumpercent firing threshold
resetEnabled	Bool	True		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
resetResultCount	Int	20		Minimum number of devices tested before trigger is re-armed. SwiftTest-MAX will test the number of devices defined in the baseline/resetResultCount parameter before arming/rearming the trigger.
maxResetCount	Int	-1		Maximum number of times this trigger can re-arm. A value of -1 means no maximum, the trigger can re-arm indefinitely.
message	Text			Trigger firing message to pass to Exensio database/SC
DisabledSiteList	tag			Begin list of sites to be disabled
DisabledSite	int			Site number to be disabled.
TriggerActions	Tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	Tag			Begin a single trigger action configuration.

YieldTrigger Profile Example

```
<TriggerMethod type="YieldTrigger" name="MyYieldTrigger" enabled="true"
triggerLevel="testrun" baselineEnabled="true" baseline="6" minimumPercent="60"
maximumPercent="80" resetEnabled="true" resetResultCount="10" maxResetCount="1"
bySite="true" deviceGroup="all">
  <DisabledSiteList>
    <DisabledSite number="0" />
    <DisabledSite number="2" />
  </DisabledSiteList>
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog"
      message="### write message trigger action log test message ###">
    </TriggerAction>
  </TriggerActions>
</TriggerMethod>
```



TIMING DIAGRAM FOR YIELDTRIGGER PROFILE EXAMPLE

RecoveryYieldTrigger

The **RecoveryYieldTrigger** is used during in-line retest of bad devices. If the delta between the original yield and the overall lot retest yield exceeds the defined delta, then the trigger fires.

Example message:

"Recovery yield trigger RecoveryYieldTrigger-name fired at lot end when yield difference between first pass 75.00% and final pass 89.00% is 14.00% exceeded allowable 1.00%."

RecoveryYieldTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		N/A	Denotes the beginning of a new trigger method configuration
name	Text			User-defined trigger name
type	text	RecoveryYieldTrigger		The type of trigger being configured
maximumPercentDelta	int	10		Enter the maximum allowable percentage yield delta between first pass yield and the final yield (final yield minus first pass yield). The trigger will fire if the delta is above this value.
resetEnabled	bool	True		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
maxResetCount	int	5		Maximum number of times this trigger can re-arm. A value of -1 means no maximum, the trigger can re-arm indefinitely.
enabled	bool	False	True/False	Trigger is enabled
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag			Begin a single trigger action configuration.
OutputStrategies	tag			Tag denoting the beginning of output methods for this trigger action.
OutputStrategy	tag			The beginning of the configuration information for a single output method.
maxRetryAttempts	int	2		The maximum number of retries to be attempted.
destination	Path			Path where output will be sent.
type	text		File, Socket, Email, or FTP	The name of the output strategy.

Element	Type	Default	Validation	Description
retryIntervalInMinutes	int	2		The frequency in minutes with which retransmits will be attempted.
isRetransmittable	bool	True	True/False	If selected this will allow attempts at retransmitting if the first attempt fails.

RecoveryYieldTrigger Profile Example

```
<TriggerMethod type="RecoveryYieldTrigger" maximumPercentDelta="10" name="a-name"
enabled="true">
  <TriggerActions>
    <TriggerAction type="CreateEventReport" enabled="true">
      <OutputStrategies>
        <OutputStrategy maxRetryAttempts="100" destination="reports" type="file"
          retryIntervalInMinutes="2" isRetransmittable="true" />
      </OutputStrategies>
    </TriggerAction>
  </TriggerActions>
</TriggerMethod>
```

Event Report Example

```
<?xml version='1.0' encoding='UTF-8'?>
<event type="Exception" method="RecoveryYieldTrigger" ... >
  <LotInfo>
    ..
  </LotInfo>
<TriggerDefinition>
<TriggerMethod ...>
  ...
</TriggerMethod>
</TriggerDefinition>
<param name="f_value" value="14.00%"/>
<param name="ucl" value="0.01"/>
<param name="f_ucl" value="1.00%"/>
<param name="device_id" value="10:0"/>
<param name="type" value="RecoveryYieldTrigger"/>
<param name="value" value="0.14"/>
<param name="description" value="Recovery yield trigger RecoveryYieldTrigger-name
fired at lot end when yield difference between first pass 75.00% and final pass 89.00%
is 14.00% exceeded allowable 1.00%"/>
  ...
<EventDeviceInfo exceptionType="LotException">
  <Device deviceId="10:0" deviceSequenceNumber="124" site="3" insertion="31"
    passing="false" partText="W-1:X-10:Y-0" partId="10:0" HWBin="2" SWBin="2">
    <EventDetails>
      <EventDetail firstPassYield="75.00%" finalYield="89.00%"/>
    </EventDetails>
  </Device>
</EventDeviceInfo>
</event>
```

Event Report Example 2

```
<?xml version='1.0' encoding='UTF-8'?>
<event>
  <param name="type" value="LotException"/>
  <param name="ruleID" value="466"/>
  <param name="ruleGUID" value="B259BBA68B734EF1A70C36F9070A4925"/>
  <param name="ruleRevision" value="4"/>
  <param name="actionID" value="956"/>
  <param name="timestamp" value="2019-10-23T14:00:33+02:00"/>
  <param name="guid" value="fd3d6070171fdb3b_76eeee7d_16df87ba376_-7ffe"/>
  <param name="partType" value="simulator"/>
  <param name="packageType" value=""/>
  <param name="userText" value=""/>
  <EventDeviceInfo>
    <ReferenceDevice deviceSequenceNumber="124" failValue="14.00%"/>
  </EventDeviceInfo>
</event>
```

Event Report Example 3 – Verbose Mode

```
<?xml version='1.0' encoding='UTF-8'?>
<event eventRecurrence="1">
  <LotInfo>
...
  </LotInfo>
  <TriggerDefinition>
    <TriggerMethod name="RecoveryYieldTrigger-name" type="RecoveryYieldTrigger"
      enabled="true" ruleID="466" revision="4"
      ruleGUID="B259BBA68B734EF1A70C36F9070A4925" maximumPercentDelta="1">
...
    </TriggerMethod>
  </TriggerDefinition>
  <param name="type" value="LotException"/>
  <param name="ruleID" value="466"/>
  <param name="ruleGUID" value="B259BBA68B734EF1A70C36F9070A4925"/>
  <param name="ruleRevision" value="4"/>
  <param name="timestamp" value="2019-10-23T14:00:33+02:00"/>
  <param name="guid" value="fd3d6070171fdb3b_76eeee7d_16df87ba376_-7ffe"/>
  <EventDeviceInfo>
    <ReferenceDevice deviceSequenceNumber="124" failValue="14.00%"/>
  </EventDeviceInfo>
</event>
```

YieldTriggerByTouchdown

The **YieldTriggerByTouchdown** fires when the yield for each touchdown is not above the defined minimum % value.

YieldTriggerByTouchdown Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		N/A	Denotes the beginning of a new trigger method configuration
name	Text			User-defined trigger name
type	text	YieldTriggerByTouchdown		The type of trigger being configured
minimum	int	85		Enter the minimum allowable percentage yield per touchdown (between 0 and 100), below which the trigger will fire.
resetEnabled	bool	True		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
maxResetCount	int	5		Maximum number of times this trigger can re-arm. A value of -1 means no maximum, the trigger can re-arm indefinitely.
enabled	bool	False	True/False	Trigger is enabled
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag			Begin a single trigger action configuration.
OutputStrategies	tag			Tag denoting the beginning of output methods for this trigger action.
OutputStrategy	tag			The beginning of the configuration information for a single output method.
maxRetryAttempts	int	2		The maximum number of retries to be attempted.
destination	Path			Path where output will be sent.
type	text		File, Socket, Email, or FTP	The name of the output strategy.
retryIntervalInMinutes	int	2		The frequency in minutes with which retransmits will be attempted.
isRetransmittable	bool	True	True/False	If selected this will allow attempts at retransmitting if the first attempt fails.

YieldTriggerByTouchdown Profile Example

```
<TriggerMethod name="TouchDown85Yield" type="YieldByTouchDownLimitTrigger" minimum="85"
resetEnabled="true" maxResetCount="5" enabled="true">
  <TriggerActions>
    <TriggerAction type="CreateEventReport" enabled="true">
      <OutputStrategies>
        <OutputStrategy maxRetryAttempts="100" destination="reports" type="file"
          retryIntervalInMinutes="2" isRetransmittable="true" />
      </OutputStrategies>
    </TriggerAction>
  </TriggerActions>
</TriggerMethod>
```

SiteToSiteYieldTrigger

The **SiteToSiteYieldTrigger** fires when the yield differential between two or more test sites exceeds the minimum as defined in the **maximumPercent** parameter.

SiteToSiteYieldTrigger Profile Example



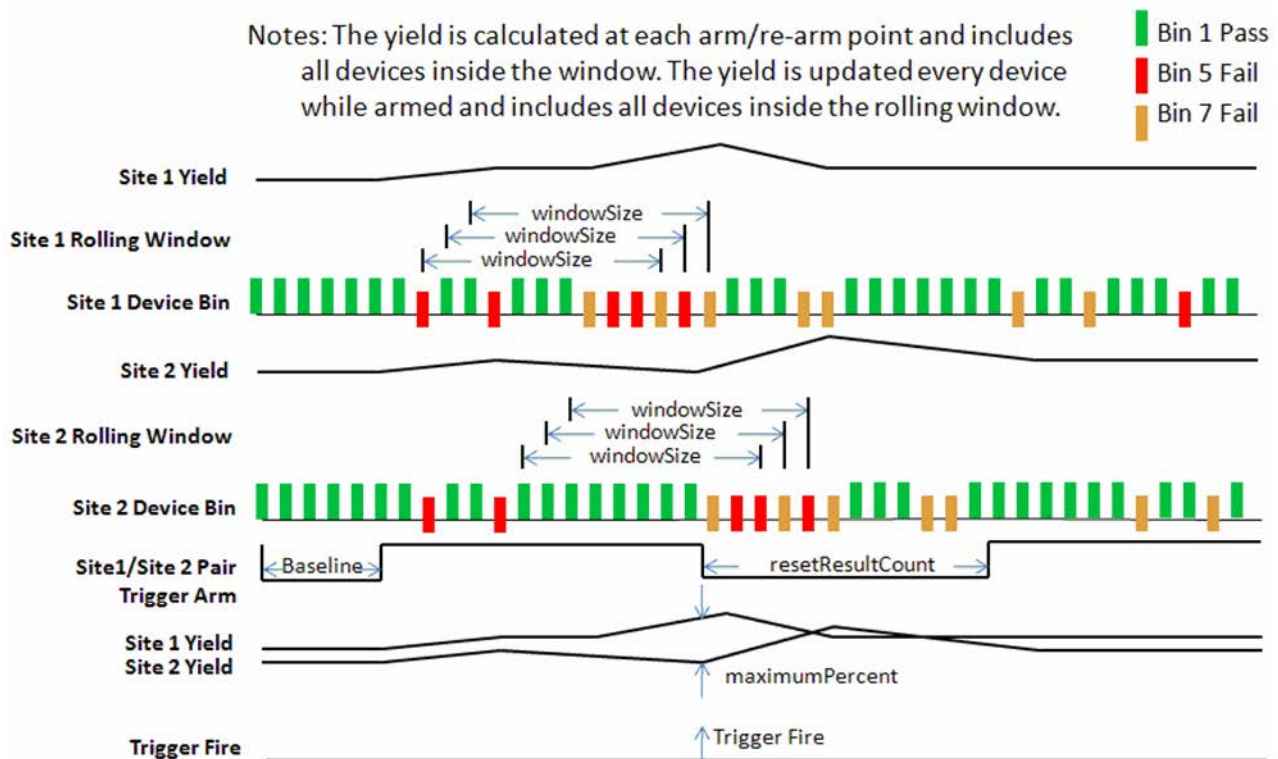
SwiftTest-MAX independently computes the percent delta for each siteA/siteB pair. If any site pair causes the trigger to fire, all site pairs will reset.

```
<TriggerMethod type = "SiteToSiteYieldTrigger" name="SiteToSiteYieldTrigger"
enabled="true" triggerLevel="testrun" baselineEnabled="true" baseline="6"
maximumPercent="5" resetEnabled="true" resetResultCount="10" deviceGroup="all">
  <DisabledSiteList>
    <DisabledSite number="0" />
    <DisabledSite number="2" />
  </DisabledSiteList>
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog"
      message="### write message trigger action log test message ###">
    <TriggerAction>
  </TriggerActions>
</TriggerMethod>
```

SiteToSiteYieldTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	Tag		N/A	Denotes the beginning of a new trigger method configuration
type	Text	SiteToSiteYieldTrigger		The type of trigger being configured
name	Text			User-defined trigger name
enabled	Bool	True	True/False	Trigger is enabled
baselineEnabled	Bool	True	True/False	Require baseline devices tested before arming trigger
baseline	Int	50		Number of devices tested before arming trigger
maximumPercent	Int	5		The maximum allowed yield difference between any two test sites
windowSize	Int	0		Length of the rolling window to compute bin counts. A value of 0 means there is no window, include all devices. Setting the windowSize attribute will define the size of a rolling window of data used to evaluate this trigger.

Element	Type	Default	Validation	Description
resetEnabled	Bool	True		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
resetResultCount	Int	10		Minimum number of devices tested before trigger is re-armed. SwiftTest-MAX will test the number of devices defined in the baseline/resetResultCount parameter before arming/rearming the trigger.
maxResetCount	Int	-1		Maximum number of times this trigger can re-arm. A value of -1 means re-arm indefinitely
message	Text			Trigger firing message to pass to Exensio database/SC
TriggerActions	Tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	Tag			Begin a single trigger action configuration.



TIMING DIAGRAM FOR SITEToSITEYIELDTRIGGER IN PROFILE EXAMPLE

DeviceFrequencyTrigger

DeviceFrequencyTrigger fires every n th number of devices, as configured in the **deviceFrequency** parameter.

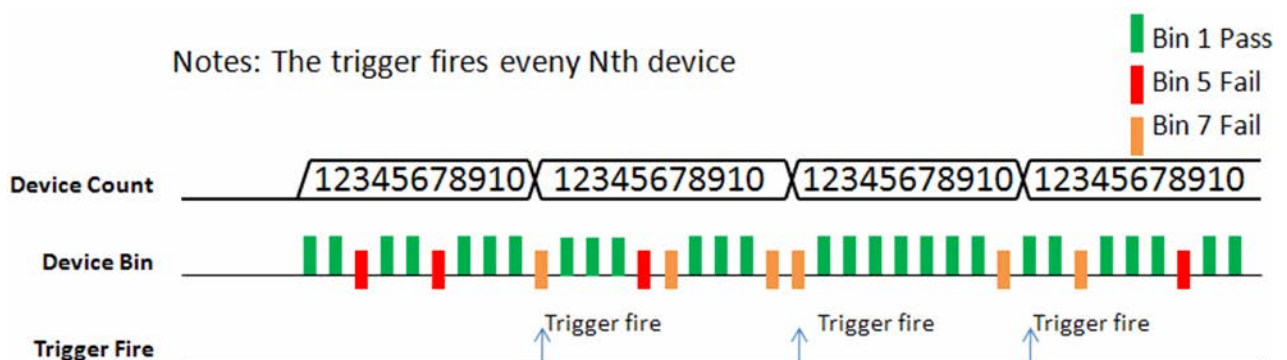
DeviceFrequencyTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
type	text	DeviceFrequencyTrigger		The type of trigger being configured
name	text			User-defined trigger name
enabled	bool	False	true/false	Trigger is enabled
deviceFrequency	int	250		Number of consecutive devices to cause trigger to fire
message	text			Trigger firing message to pass to Exensio database/SC
TriggerActions	tag			Begin the list of trigger actions that will be executed when this trigger fires.
TriggerAction	tag			Begin a single trigger action configuration.

Profile Example

```
<TriggerMethod type="DeviceFrequencyTrigger" name="MyDeviceFrequencyTrigger"
enabled="true" deviceFrequency="10">
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog"
      message="### write message trigger action log test message ###">
  </TriggerAction>
  </TriggerActions>
</TriggerMethod>
```

Notes: The trigger fires every Nth device



TIMING DIAGRAM FOR DEVICEFREQUENCYTRIGGER PROFILE EXAMPLE

LotBeginTrigger

The **LotBeginTrigger** fires when the STDF MIR record is received at the beginning of each lot. Some testers generate the MIR record immediately when the lot is opened and other testers generate the MIR record after the first device is tested.

LotBeginTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		N/A	Denotes the beginning of a new trigger method configuration.
type	text	LotBeginTrigger		The type of trigger being configured.
name	text			User-defined trigger name.
enabled	bool	True	true/false	Trigger is enabled.
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires.
TriggerAction	tag			Begin a single trigger action configuration.

Profile Example

```

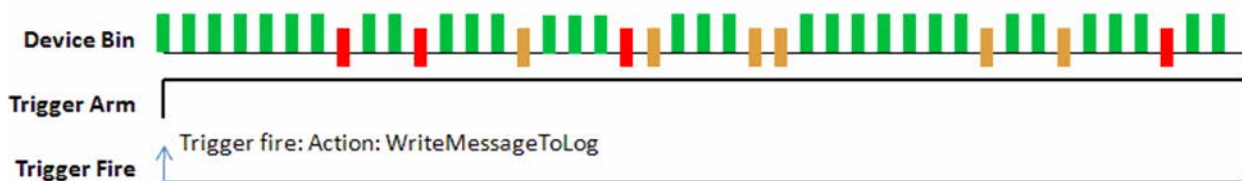
<TriggerMethod type="LotBeginTrigger" name="MyLotBeginTrigger" enabled="true"
  triggerLevel="testrun" deviceGroup="all">
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog"
      message="### write message trigger action log test message ###">

<TriggerAction>
  </TriggerActions>
</TriggerMethod>

```

Notes: The beginning of the lot is determined by the arrival of the STDF MIR record. This point can vary from tester to tester. Some testers write the MIR when the lot is opened. Other testers write the MIR after the first device is tested.

■ Bin 1 Pass
■ Bin 5 Fail
■ Bin 7 Pass



TIMING DIAGRAM FOR LOTBEGINTRIGGER PROFILE EXAMPLE

LotEndTrigger

The **LotEndTrigger** fires when the end of lot is detected. Commonly used for report generation, test result forwarding and operator notification, the **LotEndTrigger** fires when the STDF.MRR is received or the lot monitor session exceeds the timeout limit.

LotEndTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
type	text	LotEndTrigger		The type of trigger being configured
name	text			User-defined trigger name
baselineEnabled	bool	True	true/false	Trigger is enabled
baseline	int	50		Require baseline devices tested before arming trigger
enabled	bool	True	true/false	Number of devices tested before arming trigger
message	text			Trigger firing message to pass to Exensio database/SC
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag			Begin a single trigger action configuration.

Profile Example

```
<TriggerMethod type="LotEndTrigger" name="MyLotEndTrigger"
triggerLevel="testrun" baselineEnabled="true" baseline="50" enabled="true"
deviceGroup="all">
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog"
      message="### write message trigger action log test message ###">

<TriggerAction>
  </TriggerActions>
</TriggerMethod>
```

Notes: The end of the lot is determined by:

- the arrival of the STDF MRR record
- the time from the last device exceeds the lot monitor timeout
- the lot end button on the Status Console is pressed

Bin 1 Pass
Bin 5 Fail
Bin 7 Fail



TIMING DIAGRAM FOR LOTENDTRIGGER

LotValidationTrigger

The **LotValidationTrigger** sends all known STDF.MIR and SDR fields to Exensio database and fires if the return value from Exensio database is anything other than OK. This trigger can be used to validate any current lot against any data stored in Exensio database.

LotValidationTriggerElement Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
type	text	LotValidationTrigger		The type of trigger being configured
name	text			User-defined trigger name
enabled	bool	true	true/false	Number of devices tested before arming trigger
url	text			URL to the Exensio database lot validation checker
timeoutMillis	int			TCP/IP timeout in milliseconds
message	text			Trigger firing message to pass to Exensio database/SC
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag			Begin a single trigger action configuration.

STDF Data: Reads STDF.MIR and SDR.

Profile Example This trigger is in the SwiftTest code. It sends the MIR (LotInfo) to the specified URL and calls `_Lot_Validation.aspx`. It checks the database, verifies the lot settings and sends back an “OK” response.

```
<TriggerMethod type='LotValidationTrigger' name="MyLotValidationTrigger"
enabled="true" triggerLevel="testrun" url="webservice.pdfsc.com/_
_Lot_Validation.aspx" timeoutMillis="3000" deviceGroup="all">
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog"
      message="### write message trigger action log test message ###">
    </TriggerAction>
  </TriggerActions>
</TriggerMethod>
```

SegmentBeginTrigger

The **SegmentBeginTrigger** fires at the beginning of data each test program segment. The arrival of a WIR record qualifies as a segment, and this trigger will probably be most useful in tracking the beginning of new wafers within the same lot.

SegmentBeginTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
type	text	SegmentBeginTrigger		The type of trigger being configured
name	text			User-defined trigger name
enabled	bool	true	true/false	Trigger is enabled
message	text			Trigger firing message to pass to Exensio database/SC
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag			Begin a single trigger action configuration.

SegmentBeginTrigger Profile Example

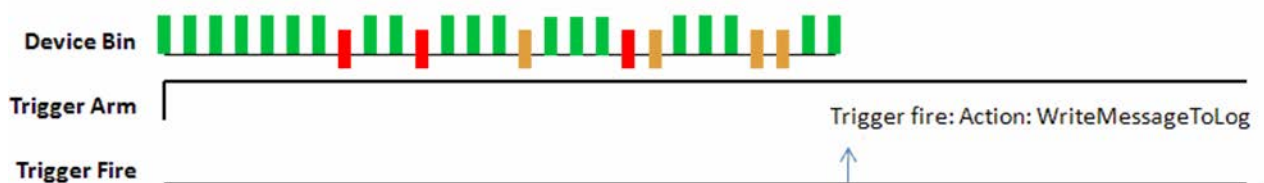
```
<TriggerMethod type="SegmentBeginTrigger" name="MySegmentBeginTrigger"
enabled="true" triggerLevel="testrun" deviceGroup="all">
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog"
      message="### write message trigger action log test message ###">

  <TriggerAction>
  </TriggerActions>
</TriggerMethod>
```

Notes: The end of the segment is determined by:

- the arrival of the STDF WIR or MRR record
- the time from the last device exceeds the lot monitor timeout
- the lot end button on the Status Console is pressed

 Bin 1 Pass
 Bin 5 Fail
 Bin 7 Fail



TIMING DIAGRAM FOR SEGMENTBEGINTRIGGER PROFILE EXAMPLE

SegmentEndTrigger

The **SegmentEndTrigger** fires at the completion of each segment. See previous entry for more information on segments.

SegmentEndTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	Tag		N/A	Denotes the beginning of a new trigger method configuration.
type	Text	SegmentEndTrigger		The type of trigger being configured
name	Text			User-defined trigger name
enabled	Bool	True	True/False	Trigger is enabled
message	Text			Trigger firing message to pass to Exensio database/SC
TriggerActions	Tag			Begin the list of trigger actions that will be executed when this trigger fires.
TriggerAction	Tag			Begin a single trigger action configuration.

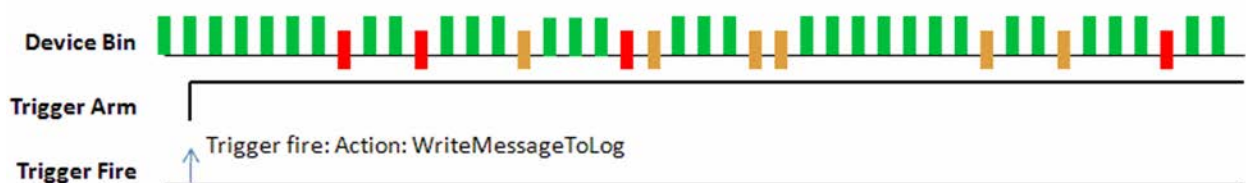
SegmentEndTrigger Profile Example

```
<TriggerMethod type="SegmentEndTrigger" name="MySegmentEndTrigger"
triggerLevel="testrun" baselineEnabled="true" baseline="50" enabled="true"
deviceGroup="all">
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog"
      message="### write message trigger action log test message ###">

  <TriggerAction>
  </TriggerActions>
</TriggerMethod>
```

Notes: The beginning of the segment is determined by the arrival of the STDF WIR or the first PIR record after the MIR. This point will always occur after the first device is tested.

■ Bin 1 Pass
■ Bin 5 Fail
■ Bin 7 Fail



TIMING DIAGRAM FOR SEGMENTENDTRIGGER PROFILE EXAMPLE

TimeFrequencyTrigger

Fires every n th number of minutes as defined in FrequencyInMinutes. This trigger will fire repeatedly during the course of the lot, sub lot, or wafer when **resetEnabled="true"**. At least one trigger action is required for this trigger.

TimeFrequencyTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	Tag		N/A	Denotes the beginning of a new trigger method configuration
type	Text	TimeFrequencyTrigger		The type of trigger being configured
name	Text			User-defined trigger name
enabled	Bool	True	True/False	Trigger is enabled
baselineEnabled	Bool	True	True/False	Require baseline devices tested before arming trigger
baseline	Int	50		Number of devices tested before arming trigger
timeFrequencyInMinutes	Int	120		Time interval between firings
message	Text			Trigger firing message to pass to Exensio database/SC
TriggerActions	Tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	Tag			Begin a single trigger action configuration

TimeFrequencyTrigger Profile Example

```
<TriggerMethod type="TimeFrequencyTrigger" name="MyTimeFrequencyTrigger"
enabled="true" triggerLevel="testrun" baselineEnabled="true" baseline="50"
timeFrequencyInMinutes="120" deviceGroup="all">
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog"
      message="### write message trigger action log test message ###">

  <TriggerAction>
  </TriggerActions>
</TriggerMethod>
```

Notes: The baseline number of devices must be tested before the trigger will arm. Once armed, the trigger implicitly resets and fires until the end of the lot.

Bin 1 Pass
Bin 5 Fail
Bin 7 Fail



TIMING DIAGRAM FOR TIMEFREQUENCYTRIGGER PROFILE EXAMPLE

EXCEPTION RULE TRIGGERS AND EXAMPLES



Exception rules only support hardware bins.

BadDeviceGoodBinTrigger

BadDeviceGoodBinTrigger fires when SwiftTest detects a failing device that has been put in a passing bin.

BadDeviceGoodBinTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
type	text	BadDeviceGoodBinTrigger		The type of trigger being configured
name	text			User-defined trigger name
enabled	bool	false	true/false	Trigger is enabled
baselineEnabled	bool	true	true/false	Require baseline devices tested before arming trigger
baseline	int	1		Number of devices tested before arming
bySite	bool	false	true/false	(Optional.) Setting this triggers bySite attribute to true will treat each site population independently.
resetEnabled	bool	false		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
resetResultCount	int	10		Minimum number of devices tested before trigger is re-armed. SwiftTest-MAX will test the number of devices defined in the baseline/resetResultCount parameter before arming/rearming the trigger.
maxResetCount	int	-1		Maximum number of times this trigger can re-arm. A value of -1 means no maximum, the trigger can re-arm indefinitely.
Bins	tag			Begin bin definition section, multiple bins can be defined
Bin	tag			Define a single watched bin
type	text	"Hardware" / "Software"	case insensitive	Hardware or software bin
number	int			Number of each hardware or software bin to monitor
message	text			Trigger firing message to pass to Exensio database/SC

Element	Type	Default	Validation	Description
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag			Begin a single trigger action configuration
triggerLevel	text	"testrun" (also accepts "lotend")	case insensitive	(Optional.) Scope for which trigger is evaluated — except EcidTrigger , DeviceTestTimeTrigger and TestCountTrigger .

STDF Data: Reads PRR.PART_FLG (bits 2, 3 or 4 not=0) and PRR.HARD_BIN.

User Input: Good Hardware Bin List.

BadDeviceGoodBinTrigger Profile Example

```
<TriggerMethod type='BadDeviceGoodBinTrigger' enabled='true'
name='MyBadDeviceGoodBinTrigger' baselineEnabled='true' baseline='50'
resetEnabled='true' resetResultCount='1' maxResetCount='100'
triggerLevel='lotend' deviceGroup="all">
<Bins>
  <Bin number='1' />
</Bins>
  <TriggerActions>
    <TriggerAction type='CreateEventReport' enabled='true'>
      <OutputStrategies>
        <OutputStrategy type="file" destination="c:/data/
reports" isRetransmittable="true"
retryIntervalInMinutes="2" maxRetryAttempts="100"
fileNameingSpec="BadDeviceGoodBinEvent_BadDeviceGoodBi
nTrigger_$(ReportDate)$(ReportTime)">
      </OutputStrategy>
    </OutputStrategies>
  </TriggerAction>
</TriggerActions>
</TriggerMethod>
```

GoodDeviceBadBinTrigger

GoodDeviceBadBinTrigger fires when SwiftTest detects a passing device that has been put in a failing bin.

GoodDeviceBadBinTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
type	text	GoodDeviceBadBinTrigger		The type of trigger being configured
name	text			User-defined trigger name
enabled	bool	false	true/false	Trigger is enabled
baselineEnabled	bool	true	true/false	Require baseline devices tested before arming trigger
baseline	int	1		Number of devices tested before arming trigger
bySite	bool	false	true/false	(Optional.) Setting this triggers bySite attribute to true will treat each site population independently.
resetEnabled	bool	false		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
resetResultCount	int	10		Minimum number of devices tested before trigger is re-armed. SwiftTest-MAX will test the number of devices defined in the baseline/resetResultCount parameter before arming/rearming the trigger.
maxResetCount	int	-1		Maximum number of times this trigger can re-arm. A value of -1 means no maximum, the trigger can re-arm indefinitely.
Bins	tag			Begin bin definition section, multiple bins can be defined
Bin	tag			Define a single watched bin
type	text	"Hardware" / "Software"	case insensitive	Hardware or software bin
number	int			Number of each hardware or software bin to monitor
message	text			Trigger firing message to pass to Exensio database/SC
DisabledSiteList	tag			Begin list of sites to be disabled
DisabledSite	int			Site number to be disabled.
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag			Begin a single trigger action configuration

Element	Type	Default	Validation	Description
triggerLevel	text	"testrun" (also accepts "lotend")	case insensitive	(Optional.) Scope for which trigger is evaluated — except EcidTrigger , DeviceTestTimeTrigger and TestCountTrigger .

STDF Data: Reads PRR.PART_FLG (bits 2, 3 and 4 = 0) and PRR.HARD_BIN.

User Input: Bad Hardware Bin List.

GoodDeviceBadBinTrigger Profile Example

```
<TriggerMethod type='GoodDeviceBadBinTrigger' enabled='true'
name='MyGoodDeviceBadBinTrigger' triggerLevel='testrun'
baselineEnabled='true' baseline='50' resetEnabled='true'
resetResultCount='1' maxResetCount='100' triggerLevel='lotend'
deviceGroup='all'>
  <Bins>
    <Bin number='2' />
  </Bins>
  <DisabledSiteList>
    <DisabledSite number="0" />
    <DisabledSite number="2" />
  </DisabledSiteList>
  <TriggerActions>
    <TriggerAction type='CreateEventReport' enabled='true'>
      <OutputStrategies>
        <OutputStrategy type="file" destination="c:/data/reports"
          isRetransmittable="true" retryIntervalInMinutes="2"
          maxRetryAttempts="100"
          fileNameingSpec="GoodDeviceBadBinEvent_GoodDeviceBadBi
nTrigger_$(ReportDate)$(ReportTime)">
        </OutputStrategy>
      </OutputStrategies>
    </TriggerAction>
  </TriggerActions>
</TriggerMethod>
```

DeviceTestTimeTrigger

DeviceTestTimeTrigger fires when a device takes more or less time to test than it should.

Calculations:

Fixed Device Test Time

(when **testTimeMillis** attribute is defined in profile):

$$\begin{aligned}\text{minDeviceTestTime} &= \text{testTimeMillis} - (\text{testTimeMillis} * (\text{lowerMarginPercent} / 100.0)) \\ \text{maxDeviceTestTime} &= \text{testTimeMillis} + (\text{testTimeMillis} * (\text{upperMarginPercent} / 100.0))\end{aligned}$$

Calculated Device Test Time

(when **testTimeMillis** attribute is not used)

$$\begin{aligned}\text{minDeviceTestTime} &= \text{median} - (\text{median} * (\text{lowerMarginPercent} / 100.0)) \\ \text{maxDeviceTestTime} &= \text{median} + (\text{median} * (\text{upperMarginPercent} / 100.0))\end{aligned}$$

DeviceTestTimeTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
type	text	DeviceTestTimeTrigger		The type of trigger being configured
name	text			User-defined trigger name
enabled	bool	false	true/false	Trigger is enabled
baselineEnabled	bool	true	true/false	Require baseline devices tested before arming trigger
baseline	int	1		Number of devices tested before arming trigger
bySite	bool	false	true/false	(Optional.) Setting this triggers bySite attribute to true will treat each site population independently.
resetEnabled	bool	false		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
resetResultCount	int	10		Minimum number of devices tested before trigger is re-armed. SwiftTest-MAX will test the number of devices defined in the baseline/resetResultCount parameter before arming/rearming the trigger.
maxResetCount	int	-1		Maximum number of times this trigger can re-arm. A value of -1 means no maximum, the trigger can re-arm indefinitely.
Bins	tag			Begin bin definition section, multiple bins can be defined
Bin	tag			Define a single watched bin

Element	Type	Default	Validation	Description
type	text	"Hardware" / "Software"	case insensitive	Hardware or software bin
number	int			Number of each hardware or software bin to monitor
message	text			Trigger firing message to pass to Exensio database/SC
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag			Begin a single trigger action configuration
triggerLevel	text	"testrun" (also accepts "lotend")	case insensitive	(Optional.) Scope for which trigger is evaluated — except EcidTrigger , DeviceTestTimeTrigger and TestCountTrigger .
upperMarginPercent	float			Used in calculation for fixed and calculated device test time (Fixed Device Test Time , Calculated Device Test Time above).
lowerMarginPercent	float			Used in calculation for fixed and calculated device test time (Fixed Device Test Time , Calculated Device Test Time above).
testTimeMillis	int			Fixed test time, in milliseconds. Used in calculation for fixed device test time (Fixed Device Test Time , above).

STDF Data: Reads PRR.HARD_BIN or PRR.PART_FLG (bits 2, 3 and 4 = 0) and PRR.TEST_T.

User Input: Good Hardware Bin List (else use passing devices), **upperMarginPercent**, **lowerMarginPercent** and **testTimeMillis** (for fixed device test time).

DeviceTestTimeTrigger Profile Example — Calculated

```

<?xml version='1.0' encoding='UTF-8'?>
<DeviceProfile xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://
www.w3.org/2001/XMLSchema" version="6.0">
  <DeviceInfo jobName="" partType="" company="" organization="" />
  <TestInfo company="" organization="" location="" facility="" />
  <Author name="DynamicProfile" email="support@pdf.com"
generatedBy="RecipeDeliveryService 4.1.2.43" profileRev="" generatedDate="2019-11-
05T08:52:37-08:00" />
  <MonitorSettings />
  <Features>
    <Feature type="Trigger" enabled="true">
      <TriggerMethodSet>
        <TriggerMethod type="DeviceTestTimeTrigger" name="CalculatedTestTimeRule"
enabled="true" resetResultCount="1" resetEnabled="true" baseline="20"
upperMarginPercent="15" lowerMarginPercent="10"
ruleID="1055" revision="1" ruleGUID="494706EF90C44DCF88AF79EA1468F941">
          <TriggerActions>
            <TriggerAction type="CreateEventReport" enabled="true" version="1"
actionID="2086">
              <OutputStrategies>
                <OutputStrategy maxRetryAttempts="100" destination="reports"
type="file" retryIntervalInMinutes="2" isRetransmittable="true"
/>
              </OutputStrategies>
            </TriggerAction>
          </TriggerActions>
        </TriggerMethod>
      </TriggerMethodSet>
    </Feature>
  </Features>
</DeviceProfile>

```

DeviceTestTimeTrigger Profile Example — Fixed

```

<?xml version='1.0' encoding='UTF-8'?>
<DeviceProfile xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://
www.w3.org/2001/XMLSchema" version="6.0">
  <DeviceInfo jobName="" partType="" company="" organization="" />
  <TestInfo company="" organization="" location="" facility="" />
  <Author name="DynamicProfile" email="support@pdf.com"
generatedBy="RecipeDeliveryService 4.1.2.43" profileRev="" generatedDate="2019-11-
05T08:52:37-08:00" />
  <MonitorSettings />
  <Features>
    <Feature type="Trigger" enabled="true">
      <TriggerMethodSet>
        <TriggerMethod type="DeviceTestTimeTrigger" name="CalculatedTestTimeRule"
enabled="true" resetResultCount="1" resetEnabled="true" baseline="20"
upperMarginPercent="15" lowerMarginPercent="10" testTimeMillis="650"
ruleID="1055" revision="1" ruleGUID="494706EF90C44DCF88AF79EA1468F941">
          <TriggerActions>
            <TriggerAction type="CreateEventReport" enabled="true" version="1"
actionID="2086">
              <OutputStrategies>
                <OutputStrategy maxRetryAttempts="100" destination="reports"
type="file" retryIntervalInMinutes="2"
isRetransmittable="true" />
              </OutputStrategies>
            </TriggerAction>
          </TriggerActions>
        </TriggerMethod>
      </TriggerMethodSet>
    </Feature>
  </Features>
</DeviceProfile>

```

TestCountTrigger

TestCountTrigger fires when the minimum required number of tests is not executed on a device.

TestCountTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration.
type	text	TestCountTrigger		The type of trigger being configured.
name	text			User-defined trigger name.
enabled	bool	false	true/false	Trigger is enabled.
baselineEnabled	bool	true	true/false	Require baseline devices tested before arming trigger.
baseline	int	1		Number of devices tested before arming trigger.
bySite	bool	false	true/false	(Optional.) Setting this triggers bySite attribute to true will treat each site population independently.
resetEnabled	bool	false		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
resetResultCount	int	10		Minimum number of devices tested before trigger is re-armed. SwiftTest-MAX will test the number of devices defined in the baseline/resetResultCount parameter before arming/rearming the trigger.
maxResetCount	int	-1		Maximum number of times this trigger can re-arm. A value of -1 means no maximum, the trigger can re-arm indefinitely.
Bins	tag			Begin bin definition section, multiple bins can be defined
Bin	tag			Define a single watched bin
type	text	"Hardware" / "Software"	case insensitive	Hardware or software bin
number	int			Number of each hardware or software bin to monitor
message	text			Trigger firing message to pass to Exensio database/SC
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag			Begin a single trigger action configuration
minTestCount	int			(Optional.) If missing calculation uses mode of baseline test counts.
useTestExecutionCount	bool	true		(Optional) True=MPR (Multiple Parametric Result) sub-tests included, False=MPR counted as 1 test

Reads PRR.HARD_BIN or PRR.PART_FLG (bits 2, 3 and 4 = 0) and PRR.NUM_TEST.

User Input: Good Hardware Bin List (else use passing devices) and **MinTestCount** (else use statistical test count mode).

Notes on minTestCount and Baseline

In general, if you provide a **minTestCount** value, SwiftTest will use that value from the beginning of the lot.

If you don't, SwiftTest will use the **baseline** value to monitor the tests executed on either passing devices OR the defined **Bins**. At the **baseline**, SwiftTest will calculate the mode, then set that as the **minTestCount** value.

- If you set a **baseline** value, you don't define any **Bins**, and you don't set a **minTestCount**, then SwiftTest will count the number of tests executed on each passing device and will set the **minTestCount** to the mode of those counts. Any passing device after the **baseline** that has reached the test count <**minTestCount**> will fire the trigger.
- If you set a **baseline**, you define some **Bin(s)**, and you don't set a **minTestCount**, then SwiftTest will count the number of tests executed on each device in the defined **Bin(s)** and will set the **minTestCount** to the mode of those counts. Any device after the **baseline** in the defined **Bins** that has reached the test count <**minTestCount**> will fire the trigger.*
For example, if **baseline=10**, and **binlist=bin2**. **baseline** is met when 10 devices are tested to **bin2**.
SwiftTest is looking at the number of tests executed for devices that fall into a certain bin.
- If you set a **baseline**, you don't define any bins, and you set a **minTestCount**, then SwiftTest will apply **minTestCount** from the beginning of the lot. Any passing device from the beginning of the lot that has reached the test count <**minTestCount**> will fire the trigger. The **baseline** will be ignored.
- If there is no **baseline**, you don't define any bins, and you set a **minTestCount**, then SwiftTest will apply **minTestCount** from the beginning of the lot. Any passing device from the beginning of the lot that has reached the test count <**minTestCount**> will fire the trigger.
- If there is no **baseline**, you define some **Bin(s)**, and you set a **minTestCount**, then SwiftTest will apply **minTestCount** from the beginning of the lot. Any passing device in the defined **Bin(s)** from the beginning of the lot that has reached the test count <**minTestCount**> will fire the trigger.

TestCountTrigger Profile Example

```
<TriggerMethod type='TestCountTrigger' enabled='true'
name='MyTestCountTrigger' triggerLevel='testrun' baselineEnabled='true'
baseline='50' resetEnabled='true' resetResultCount='10' maxResetCount='10'
windowSize='10' bySite='true' useTestExecutionCount='true'
minTestCount='200' deviceGroup='all'>
<Bins>
  <Bin number='5' />
</Bins>
  <TriggerActions>
    <TriggerAction type='CreateEventReport' enabled='true'>
      <OutputStrategies>
        <OutputStrategy type="file" destination="c:/data/
reports" isRetransmittable="true"
retryIntervalInMinutes="2" maxRetryAttempts="100"
fileNameingSpec="TestCountEvent_TestCountTrigger_$(Rep
ortDate)$(ReportTime)">
      </OutputStrategy>
    </OutputStrategies>
  </TriggerAction>
</TriggerActions>
</TriggerMethod>
```

LostSiteTrigger

LostSiteTrigger uses the **siteCountBaseline** period to determine the maximum number of test sites currently used for a specified lot during a given time. While monitoring the lot, it tracks all consecutive test runs where one or more sites have shut down. The trigger fires if the number of **maxConsecutiveLostSites** meets or exceeds the value assigned in the device profile. Because no statistical calculations occur, new or normal baselines are not necessary with each trigger reset. The consecutive lost site value is set to zero after the trigger fires and the trigger immediately begins counting lost sites again.

LostSiteTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
type	text	LostSiteTrigger		The type of trigger being configured
name	text			Name user gives to trigger.
siteCountbaseline	int	20		The number of test runs used to determine the maximum site count
maxConsecutiveLostSites	int	10		The maximum number of consecutive test runs with reduced site count before the trigger fires
enabled	bool	true	true/false	Is the trigger itself enabled. By default this is set to true.
message	text			Trigger firing message to pass to Exensio database/SC
TriggerActions	tag			Begin the list of trigger actions that will be executed when this trigger fires.
TriggerAction	tag			Begin a single trigger action configuration.

Profile Example

```
<TriggerMethod type="LostSiteTrigger" name="MyLostSiteTrigger"
enabled="true" triggerLevel='testrun' siteCountbaseline="20"
maxConsecutiveLostSites="10" resetEnabled="true">
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog"
      message="### write message trigger action log test message ###">
    </TriggerAction>
  </TriggerActions>
</TriggerMethod>
```

LotSetupTrigger

The **LotSetupTrigger** evaluates after the WIR or the first device record is received and fires if any of the named field values do not match the value specified in the regex definition for that field. The Lot Setup trigger is commonly used to validate the accuracy of lot metadata.

A complete listing of available field mappings is available in the SwiftTest-AMX Installation and Configuration Guide, Appendix C: *Field Mappings*.

LotSetupTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration.
type	text	LotSetupTrigger		The type of trigger being configured.
name	text			User-defined trigger name.
enabled	bool	true	true/false	Number of devices tested before arming trigger.
Fields	tag			Field definition set.
Field	tag			Field definition.
name	text			Name of the field to validate against Regex value.
regex	text			Regular expression to match against Name value.
caseSensitive	bool	true	true/false	Case sensitive.
message	text			Trigger firing message to pass to Exensio database/SC.
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires.
TriggerAction	tag			Begin a single trigger action configuration.

LotSetupTrigger Profile Example

```
<TriggerMethod type="LotSetupTrigger" name="MyLotSetupTrigger" enabled="true"
  triggerLevel="testrun">
  <Fields>
    <Field name="LotId" regex="48"/>
    <Field name="NodeName" regex="bEAst" caseSensitive="true"/>
    <Field name="JobName" regex="TestProgram1"/>
    <Field name="PartType" regex="simulator"/>
    <Field name="PartType" regex="Simulator" caseSensitive="false"/>
  </Fields>
  <TriggerActions>
    <TriggerAction type="WriteMessageToLog"
      message="### write message trigger action log test message ###">

<TriggerAction>
  </TriggerActions>
</TriggerMethod>
```

MixedLotLimitTrigger

The **MixedLotLimitTrigger** exception trigger fires if the device's lot ID does not match previous device's lot ID, based on **lotIDExtractRegex** or **LotIDSource** values entered in the rule.

Profile Example

```
<TriggerMethod type="MixedLotLimitTrigger" name="MixedLotLimitTrigger" enabled="true"
  resetEnabled="true" maxResetCount="10"
  lotIDSource="ECID|PART_TXT|PART_ID"
  lotIDExtractRegex="LotID:(.*)W:(.*)\" lotIDExtractIndex="1"
  maximum="1" usePassingDevicesOnly="false">
  <TriggerActions>
    ...
  </TriggerActions>
</TriggerMethod>
```

Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
type	text	MixedLotLimitTrigger		The type of trigger action being configured
name	text			User-defined trigger name
enabled	bool	true	true/false	Trigger method is enabled
resetEnabled	bool	false		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
maxResetCount	int	-1		Maximum number of times this trigger can re-arm. A value of -1 means no maximum, the trigger can re-arm indefinitely.
lotIDSource	text			Lot identification format: Part ID, Part Text or ECID. Empty LotIDSource value will be ignored.
lotIDExtractRegex	text			(Optional) regular expression used to extract LotIDSource value.
lotIDExtractIndex	int			Delimited index position of the lot ID in the lot extract string. (Mandatory if lotIDExtractRegex is populated.) This is the index of parenthetical group in lotIDExtractRegex .

Element	Type	Default	Validation	Description
maximum	int	1		Maximum number of mixed lots allowed, beyond which the rule will be triggered.
usePassingDevicesOnly	bool	false	true/false	When “true,” the rule is applied only to passing devices.
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag		N/A	Denotes the beginning of a new trigger action method configuration.

LimitExceptionTrigger

LimitExceptionTrigger fires when an STDF contains an invalid PTR:OPT_FLAG value.

LimitExceptionTrigger Element Specifications:

Element	type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
type	text	LimitExceptionTrigger		The type of trigger being configured
name	text			User-defined trigger name
enabled	bool	false	true/false	Trigger is enabled
baselineEnabled	bool	true	true/false	Require baseline devices tested before arming trigger
baseline	int	1		Number of devices tested before arming trigger
bySite	bool	false	true/false	(Optional.) Setting this triggers bySite attribute to true will treat each site population independently.
resetEnabled	bool	false		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
resetResultCount	int	10		Minimum number of devices tested before trigger is re-armed. SwiftTest-MAX will test the number of devices defined in the baseline/resetResultCount parameter before arming/rearming the trigger.
maxResetCount	int	-1		Maximum number of times this trigger can re-arm. A value of -1 means no maximum, the trigger can re-arm indefinitely.
Bins	tag			Begin bin definition section, multiple bins can be defined
Bin	tag			Define a single watched bin
type	text	"Hardware" / "Software"	case insensitive	Hardware or software bin
number	int			Number of each hardware or software bin to monitor
message	text			Trigger firing message to pass to Exensio database/SC
DisabledSiteList	tag			Begin list of sites to be disabled
DisabledSite	int			Site number to be disabled.
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag			Begin a single trigger action configuration

Element	type	Default	Validation	Description
triggerLevel	text	"testrun" (also accepts "lotend")	case insensitive	(Optional.) Scope for which trigger is evaluated.
bitmaskBinary	binary			(Optional.) Binary bitmask used to validate PTR:OPT_FLAG value. If no bitmask is provided, the default value of 11111101 (253) will be used.
bitmaskInt	int			(Optional.) Integer bitmask used to validate PTR:OPT_FLAG value. If no bitmask is provided the default value of 11111101 (253) will be used.
IncludeOnlyTestList	tag			(Optional.) Begin the exclusive list of tests to include in evaluation.
IncludeOnlyTest	tag			(Optional.) Single test to include in evaluation.
ExcludeTestList	tag			(Optional.) Begin the list of tests to exclude from evaluation.
ExcludeTest	tag			(Optional.) Single test to exclude from evaluation.
testNumber	int			(IncludeOnly/ExcludeTest attribute.) Test number for single test.
description	text			(IncludeOnly/ExcludeTest attribute.) Description (name) of single test.
startTestNumber	int			(IncludeOnly/ExcludeTest attribute.) First test number in a range of test numbers.
endTestNumber	int			(IncludeOnly/ExcludeTest attribute.) Last test number in a range of test numbers.
testNameRegex	regex			(IncludeOnly/ExcludeTest attribute.) Regex for full or partial test description (name).

STDF Data: Reads PRR.HARD_BIN or PRR.PART_FLG (bits 2, 3 and 4 = 0) and PTR.OPT_FLG&&Mask>0.

User Input: Good Hardware Bin List (else use passing devices) and Binary Mask.

LimitExceptionTrigger Profile Example

```

<TriggerMethod type='LimitExceptionTrigger' enabled='true'
name='MyLimitExceptionTrigger' triggerLevel='testrun' baselineEnabled='true'
baseline='50' resetEnabled='true' resetResultCount='0' maxResetCount='10'
bitmaskBinary='11110001'>
  <IncludeOnlyTestList>
    <IncludeOnlyTest testNumber="20"/>
    <IncludeOnlyTest description="Test20 <> Group2"/>
    <IncludeOnlyTestSet startTestNumber="10" endTestNumber="30"/>
    <IncludeOnlyTest
      testNameRegex="[t|T][e|E][s|S][t|T][n|N][a|A][m|M][e|E]" />
  </IncludeOnlyTestList>
  <ExcludeTestList>
    <ExcludeTest testNumber="20"/>
    <ExcludeTest description="Test20 <> Group2"/>
    <ExcludeTestSet startTestNumber="10" endTestNumber="30"/>
    <ExcludeTest testNameRegex="[t|T][e|E][s|S][t|T][n|N][a|A][m|M][e|E]" />
  </ExcludeTestList>
  <DisabledSiteList>
    <DisabledSite number="0" />
    <DisabledSite number="2" />
  </DisabledSiteList>
  <TriggerActions>
    <TriggerAction type='CreateEventReport' enabled='true'>
      <OutputStrategies>
        <OutputStrategy type="file" destination="c:/data/reports"
          isRetransmittable="true" retryIntervalInMinutes="2"
          maxRetryAttempts="100"
          fileNameingSpec="LimitExceptionEvent_LimitExceptionTrig
            ger_$(ReportDate)$(ReportTime)">
        </OutputStrategy>
      </OutputStrategies>
    </TriggerAction>
  </TriggerActions>
</TriggerMethod>

```

LimitFailGoodBinTrigger

LimitFailGoodBinTrigger fires when a device that is in a passing bin has a test result that is outside its upper/lower limits.

LimitFailGoodBinTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
type	text	LimitFailGoodBinTrigger		The type of trigger being configured
name	text			User-defined trigger name
enabled	bool	false	true/false	Trigger is enabled
baselineEnabled	bool	true	true/false	Require baseline devices tested before arming trigger
baseline	int	1		Number of devices tested before arming trigger
bySite	bool	false	true/false	(Optional.) Setting this triggers bySite attribute to true will treat each site population independently.
resetEnabled	bool	false		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
resetResultCount	int	10		Minimum number of devices tested before trigger is re-armed. SwiftTest-MAX will test the number of devices defined in the baseline/resetResultCount parameter before arming/rearming the trigger.
maxResetCount	int	-1		Maximum number of times this trigger can re-arm. A value of -1 means no maximum, the trigger can re-arm indefinitely.
Bins	tag			Begin bin definition section, multiple bins can be defined
Bin	tag			Define a single watched bin
type	text	"Hardware" / "Software"	case insensitive	Hardware or software bin
number	int			Number of each hardware or software bin to monitor
message	text			Trigger firing message to pass to Exensio database/SC
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag			Begin a single trigger action configuration
triggerLevel	text	"testrun" (also accepts "lotend")	case insensitive	(Optional.) Scope for which trigger is evaluated.
IncludeOnlyTestList	tag			(Optional.) Begin the exclusive list of tests to include in evaluation.

Element	Type	Default	Validation	Description
IncludeOnlyTest	tag			(Optional.) Single test to include in evaluation.
ExcludeTestList	tag			(Optional.) Begin the list of tests to exclude from evaluation.
ExcludeTest	tag			(Optional.) Single test to exclude from evaluation.
testNumber	int			(IncludeOnly/ExcludeTest attribute.) Test number for single test description. (IncludeOnly/ExcludeTest attribute) Description (name) of single test.
description	text			(IncludeOnly/ExcludeTest attribute.) Description (name) of single test.
startTestNumber	int			(IncludeOnly/ExcludeTest attribute.) First test number in a range of test numbers.
endTestNumber	int			(IncludeOnly/ExcludeTest attribute.) Last test number in a range of test numbers.
testNameRegex	regex			(IncludeOnly/ExcludeTest attribute.) Regex for full or partial test description (name).

STDF Data: Reads PRR.HARD_BIN and PTR.PARM_FLG (bits 3 and 4 not=0).

User Input: Parametric Test Include and/or Exclude List.

LimitFailGoodBinTrigger Profile Example

```

<TriggerMethod type='LimitFailGoodBinTrigger' enabled='true'
name='MyLimitFailGoodBinTrigger' triggerLevel='testrun'
baselineEnabled='true' baseline='50' resetEnabled='true'
resetResultCount='10' maxResetCount='100' triggerLevel='lotend'
deviceGroup='all'>
  <IncludeOnlyTestList>
    <IncludeOnlyTest testNumber="20"/>
    <IncludeOnlyTest description="Test20 <> Group2"/>
    <IncludeOnlyTestSet startTestNumber="10" endTestNumber="30"/>
    <IncludeOnlyTest
      testNameRegex="[t|T][e|E][s|S][t|T][n|N][a|A][m|M][e|E]" />
  </IncludeOnlyTestList>
  <ExcludeTestList>
    <ExcludeTest testNumber="30"/>
    <ExcludeTest description="Test30 <> Group3"/>
    <ExcludeTestSet startTestNumber="30" endTestNumber="60"/>
    <ExcludeTest
      testNameRegex="[t|T][e|E][s|S][t|T][n|N][a|A][m|M][e|E]" />
  </ExcludeTestList>
  <TriggerActions>
    <TriggerAction type='CreateEventReport' enabled='true'>
      <OutputStrategies>
        <OutputStrategy type="file" destination="c:/data/
reports" isRetransmittable="true"
retryIntervalInMinutes="2" maxRetryAttempts="100"
fileNameingSpec="LimitFailGoodBinTriggerEvent_LimitFai
lGoodBinTrigger_$(ReportDate)$(ReportTime)">
      </OutputStrategy>
    </OutputStrategies>
  </TriggerAction>
  <TriggerAction type="VisualAlert">
    <AlertDetails message="Limit Fail Good Bin Trigger"/>
  </TriggerAction>
</TriggerActions>
</TriggerMethod>

```

RetestLimitTrigger

RetestLimitTrigger fires when the configured limit on retests is reached.

RetestLimitTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethodSet	tag			Begin trigger method entries
TriggerMethod maximum	int	1	n/a	Maximum number of retests, beyond which the rule will be triggered.
usePassingDevicesOnly	bool	false	true/false	When “true,” the rule is applied only to passing devices.
name	text			User-defined trigger name
enabled	bool	false	true/false	Trigger is enabled
ruleID	int			Rule ID in E –T Central database, delivered by RDWS.
type	text	RetestLimitTrigger		The type of trigger being configured
revision	int			Rule revision in E –T Central database, delivered by RDWS.
ruleGUID	text			Rule Guid in E –T Central database, delivered by RDWS.
deviceIDSource	text	“Not_Set”, mandatory	one of these: XY, PART_ID, PART_TXT, ECID	Source for device ID.
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag			Begin a single trigger action configuration
triggerLevel	text	Accepts “TestRun” only.	case insensitive	(Optional.) Scope for which trigger is evaluated.
actionID	int			Action ID in E –T Central database, delivered by RDWS.
type	text	CreateEventReport		The type of trigger action being configured
OutputStrategies	tag			Tag denoting the beginning of output methods for this trigger action.
OutputStrategy	tag			The beginning of the configuration information for a single output method.
maxRetryAttempts	int	2		The maximum number of retries to be attempted.
destination	Path			Path where output report will be sent.
type	text		File, Socket, Email, or FTP	The name of the output strategy.
retryIntervalInMinutes	int	2		The frequency in minutes with which retransmits will be attempted.
isRetransmittable	bool	True	True/False	If selected this will allow attempts at retransmitting if the first attempt fails.

RetestLimitTrigger Profile Example

```

<DeviceProfile xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://
www.w3.org/2001/XMLSchema" version="6.0">
  <DeviceInfo jobName="" partType="" company="" organization="" />
  <TestInfo company="" organization="" location="" facility="" />
  <Author name="DynamicProfile" email="support@pdf.com"
generatedBy="RecipeDeliveryService 4.1.0.82" profileRev="" generatedDate=
"2019-08-27T06:59:45-07:00" />
  <MonitorSettings />
  <Features>
    <Feature type="Trigger" enabled="true">
      <TriggerMethodSet>
        <TriggerMethod maximum="1" usePassingDevicesOnly="true"
name="monitor-parts-retests" enabled="true" ruleID="404"
type="RetestLimitTrigger" revision="1"
ruleGUID="64AA2B37196B4BCF98E2F1309444F3F0" deviceIDSource="PART_ID">
          <TriggerActions>
            <TriggerAction version="1" actionID="936"
type="CreateEventReport" enabled="true">
              <OutputStrategies>
                <OutputStrategy maxRetryAttempts="100" destination="reports"
type="file" retryIntervalInMinutes="2"
isRetransmittable="true" />
              </OutputStrategies>
            </TriggerAction>
          </TriggerActions>
        </TriggerMethod>
      </TriggerMethodSet>
    </Feature>
  </Features>
</DeviceProfile>

```

RetestLimitTrigger event reports will show all the violated devices lists at LotEnd.

MeasurementExceptionTrigger

MeasurementExceptionTrigger fires when an STDF contains an invalid TEST_FLG or PARM_FLG value.

MeasurementExceptionTrigger Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
type	text	MeasurementExceptionTrigger		The type of trigger being configured
name	text			User-defined trigger name
enabled	bool	False	true/false	Trigger is enabled
baselineEnabled	bool	True	true/false	Require baseline devices tested before arming trigger
baseline	int	1		Number of devices tested before arming trigger
bySite	bool	False	true/false	(Optional.) Setting this triggers bySite attribute to true will treat each site population independently.
resetEnabled	bool	False		Enable multiple firings of the trigger. Setting this trigger's resetEnabled attribute to true will allow the trigger to fire and re-arm multiple times up to and including the value specified in the maxResetCount.
resetResultCount	int	10		Minimum number of devices tested before trigger is re-armed. SwiftTest-MAX will test the number of devices defined in the baseline/ resetResultCount parameter before arming/rearming the trigger.
maxResetCount	int	-1		Maximum number of times this trigger can re-arm. A value of -1 means no maximum, the trigger can re-arm indefinitely.
Bins	tag			Begin bin definition section, multiple bins can be defined
Bin	tag			Define a single watched bin
type	text	"Hardware" / "Software"	case insensitive	Hardware or software bin
number	int			Number of each hardware or software bin to monitor
message	text			Trigger firing message to pass to Exensio database/SC
DisabledSiteList	tag			Begin list of sites to be disabled
DisabledSite	int			Site number to be disabled.
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires

Element	Type	Default	Validation	Description
TriggerAction	tag			Begin a single trigger action configuration
triggerLevel	text	"testrun" (also accepts "lotend")	case insensitive	(Optional.) Scope for which trigger is evaluated.
IncludeOnlyTestList	tag			(Optional.) Begin the exclusive list of tests to include in evaluation.
IncludeOnlyTest	tag			(Optional.) Single test to include in evaluation.
ExcludeTestList	tag			(Optional.) Begin the list of tests to exclude from evaluation.
ExcludeTest	tag			(Optional.) Single test to exclude from evaluation.
testNumber	int			(IncludeOnly/ExcludeTest attribute.) Test number for single test description. (IncludeOnly/ExcludeTest attribute) Description (name) of single test.
description	text			(IncludeOnly/ExcludeTest attribute.) Description (name) of single test.
startTestNumber	int			(IncludeOnly/ExcludeTest attribute.) First test number in a range of test numbers.
endTestNumber	int			(IncludeOnly/ExcludeTest attribute.) Last test number in a range of test numbers.
testNameRegex	regex			(IncludeOnly/ExcludeTest attribute.) Regex for full or partial test description (name).

STDF Data: Reads PTR.HARD_BIN or PTR.PART_FLG (bits 2, 3 and 4 = 0) and PTR.TEST_FLG>0 or PTR.PARM_FLG (bits 0thru4)>0.

User Input: Good Hardware Bin List (else use passing devices).

A MeasurementExceptionTrigger is looking for something that tells the system that the STDF.PTR.RESULT field is not reliable.

There are times when the system gets a test result value in the STDF.PTR.RESULT, but one or more of the TEST_FLG or PARM_FLG bits is set.

SwiftTest checks if any of the following bits are set to 1.

TEST_FLG TEST_FLG Contains the following fields:

- bit 0:
0 = No alarm
1 = Alarm detected during testing

- bit 1:
0 = The value in the RESULT field is valid (see note on RESULT)
1 = The value in the RESULT field is not valid. This setting indicates that the test was executed, but no datalogged value was taken. You should read bits 6 and 7 of TEST_FLG to determine if the test passed or failed.
- bit 2:
0 = Test result is reliable
1 = Test result is unreliable
- bit 3:
0 = No timeout
1 = Timeout occurred
- bit 4:
0 = Test was executed
1 = Test not executed
- bit 5:
0 = No abort
1 = Test aborted

PARM_FLG PARM_FLG Is the parametric flag field, and contains the following bits:

- bit 0:
0 = No scale error
1 = Scale error
- bit 1:
0 = No drift error
1 = Drift error (unstable measurement)
- bit 2:
0 = No oscillation
1 = Oscillation detected

MeasurementExceptionTrigger Profile Example

```

<TriggerMethod type='MeasurementExceptionTrigger' enabled='true'
name='MyMeasurementExceptionTrigger' triggerLevel='testrun'
baselineEnabled='true' baseline='50' resetEnabled='true'
resetResultCount='10' maxResetCount='10' triggerLevel='lotend'
deviceGroup='all'>
  <IncludeOnlyTestList>
    <IncludeOnlyTest testNumber="20"/>
    <IncludeOnlyTest description="Test20 <> Group2"/>
    <IncludeOnlyTestSet startTestNumber="10" endTestNumber="30"/>
    <IncludeOnlyTest
      testNameRegex="[t|T][e|E][s|S][t|T][n|N][a|A][m|M][e|E]" />
  </IncludeOnlyTestList>
  <ExcludeTestList>
    <ExcludeTest testNumber="30"/>
    <ExcludeTest description="Test30 <> Group3"/>
    <ExcludeTestSet startTestNumber="30" endTestNumber="60"/>
    <ExcludeTest testNameRegex="[t|T][e|E][s|S][t|T][n|N][a|A][m|M][e|E]" />
  </ExcludeTestList>
  <DisabledSiteList>
    <DisabledSite number="0" />
    <DisabledSite number="2" />
  </DisabledSiteList>
  <TriggerActions>
    <TriggerAction type='CreateEventReport' enabled='true'>
      <OutputStrategies>
        <OutputStrategy type="file" destination="c:/data/reports"
          isRetransmittable="true" retryIntervalInMinutes="2"
          maxRetryAttempts="100"
          fileNameingSpec="MeasurementExceptionEvent_Measurement
            ExceptionTrigger_$(ReportDate)$(ReportTime)">
        </OutputStrategy>
      </OutputStrategies>
    </TriggerAction>
  </TriggerActions>
</TriggerMethod>

```

UnitsPerHourLimitTrigger

UnitsPerHourLimitTrigger fires if the defined minimum units per hour (UPH) is not met.

Profile Example

```
<TriggerMethod name='UPH-440' enabled='true' subtractIdleEventTimes='false'
type='UnitsPerHourLimitTrigger' targetUPH='440'>
  <TriggerActions>
    <TriggerAction type='CreateEventReport' enabled='true'>
      <OutputStrategies>
        <OutputStrategy maxRetryAttempts='2' destination='reports' type='file'
        retryIntervalInMinutes='2' isRetransmittable='true' />
      </OutputStrategies>
    </TriggerAction>
  </TriggerActions>
</TriggerMethod>
```

Element Specifications:

Element	Type	Default	Validation	Description
TriggerMethod	tag		n/a	Denotes the beginning of a new trigger method configuration
name	text			User-defined trigger name
enabled	bool	true	true/false	Trigger method is enabled
subtractIdleEventTimes	bool	false	true/false	If true, idle event times will be subtracted from the UPH calculation.
type	text	UnitsPerHourLimitTrigger		The type of trigger action being configured
targetUPH	int			If the number of units processed in an hour (Units Per Hour) does not reach this target threshold, the rule is triggered.
TriggerActions	tag			Begin the list of trigger actions executed when this trigger fires
TriggerAction	tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	text	CreateEventReport		The type of trigger action being configured.
enabled	bool	true	true/false	Trigger action is enabled
OutputStrategies	tag			Tag denoting the beginning of output methods for this trigger action.
OutputStrategy	tag			The beginning of the configuration information for a single output method.
maxRetryAttempts	int	2		The maximum number of retries to be attempted.

Element	Type	Default	Validation	Description
destination	Path			Path where output will be sent.
type	text		File, Socket, Email, or FTP	The name of the output strategy.
retryIntervallInMinutes	int	2		The frequency in minutes with which retransmits will be attempted.
isRetransmittable	bool	True	True/False	If selected this will allow attempts at retransmitting if the first attempt fails.

TRIGGER ACTIONS

CreateBinningAlertReport

As the name implies, this is a brief summary that alerts users to binning issues including information about lot attributes, testing summary, binning summary, and triggers activated. This trigger action requires an output strategy ([“Reporting Output Strategies,”](#) pg. 107).

Profile Example

```
<TriggerAction type="CreateBinningAlertReport">
  <OutputStrategies>
    <OutputStrategy type="file" destination="reports"
      isRetransmittable="true"
      retryIntervalInMinutes="2" maxRetryAttempts="100">
    </OutputStrategy>
  </OutputStrategies>
</TriggerAction>
```

Element Specifications:

Element	Type	Default	Validation	Description
TriggerAction	Tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	Text	CreateBinningAlertReport		The type of trigger action being configured
OutputStrategies	Tag			Tag denoting the beginning of output methods for this trigger action.
OutputStrategy	Tag			The beginning of the configuration information for a single output method.
type	Text		File, Socket, Email, or FTP	The name of the output strategy.
destination	Path			Path where output will be sent.
isRetransmittable	Bool	True	True/False	If selected this will allow attempts at retransmitting if the first attempt fails.
retryIntervalInMinutes	Int	2		The frequency in minutes with which retransmits will be attempted.
maxRetryAttempts	Int	100		The maximum number of retries to be attempted.

CreateBinningReport

Reports lot attributes, yield summary, site summary, trigger activity, and a binning summary. This trigger action requires an output strategy ("[Reporting Output Strategies](#)," pg. 107).

Profile Example

```
<TriggerAction type="CreateBinningReport">
  <OutputStrategies>
    <OutputStrategy type="file" destination="reports" isRetransmittable="true"
      retryIntervalInMinutes="2" maxRetryAttempts="100">
    </OutputStrategy>
  </OutputStrategies>
</TriggerAction>
```

Element Specifications:

Element	Type	Default	Validation	Description
TriggerAction	Tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	Text	CreateBinningReport		The type of trigger action being configured
OutputStrategies	Tag			Tag denoting the beginning of output methods for this trigger action.
OutputStrategy	Tag			The beginning of the configuration information for a single output method.
type	Text		File, Socket, Email, or FTP	The name of the output strategy.
destination	Path			Path where output will be sent.
isRetransmittable	Bool	True	True/False	If selected this will allow attempts at retransmitting if the first attempt fails.
retryIntervalInMinutes	Int	2		The frequency in minutes with which retransmits will be attempted.
maxRetryAttempts	Int	100		The maximum number of retries attempted.

CreateDynamicTestReport

Includes: lot attributes, a list of dynamic test methods, and events recorded during lot.

Profile Example

```
<TriggerAction type="CreateDynamicTestReport">
  <OutputStrategies>
    <OutputStrategy type="file" destination="reports" isRetransmittable="true"
      retryIntervalInMinutes="2" maxRetryAttempts="100">
    </OutputStrategy>
  </OutputStrategies>
</TriggerAction>
```

Element Specifications:

Element	Type	Default	Validation	Description
TriggerAction	Tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	Text	CreateDynamicTestReport		The type of trigger action being configured
OutputStrategies	Tag			Tag denoting the beginning of output methods for this trigger action.
OutputStrategy	Tag			The beginning of the configuration information for a single output method.
type	Text		File, Socket, Email, or FTP	The name of the output strategy.
destination	Path			Path where output will be sent.
isRetransmittable	Bool	true	True/False	If selected this will allow attempts at retransmitting if the first attempt fails.
retryIntervalInMinutes	Int	2		The frequency in minutes with which retransmits will be attempted.
maxRetryAttempts	Int	100		The maximum number of retries attempted.

CreateEventReport

This XML report includes detailed information about a trigger that has fired (device(s), tests, results, and violated parameters). It is also sent to the YMS database to track alerts.

Profile Example NEED EXAMPLE

```
<TriggerAction type="CreateEventReport">
  <OutputStrategies>
    <OutputStrategy type="file" destination="reports"
      isRetransmittable="true"
      retryIntervalInMinutes="2" maxRetryAttempts="100">
    </OutputStrategy>
  </OutputStrategies>
</TriggerAction>
```

Element Specifications

Element	Type	Default	Validation	Description
TriggerAction	Tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	Text	CreateEventReport		The type of trigger action being configured
OutputStrategies	Tag			Tag denoting the beginning of output methods for this trigger action.
OutputStrategy	Tag			The beginning of the configuration information for a single output method.
type	Text		File, Socket, Email, or FTP	The name of the output strategy.
destination	Path			Path where output will be sent.
isRetransmittable	Bool	true	True/False	If selected this will allow attempts at retransmitting if the first attempt fails.
retryIntervalInMinutes	Int	2		The frequency in minutes with which retransmits will be attempted.
maxRetryAttempts	Int	100		The maximum number of retries attempted.

CreateGenericReport

Reports lot attributes, a test summary, device summary and feature summary.

Profile Example

```
<TriggerAction type="CreateGenericReport">
  <OutputStrategies>
    <OutputStrategy type="file" destination="reports" isRetransmittable="true"
      retryIntervalInMinutes="2" maxRetryAttempts="100">
    </OutputStrategy>
  </OutputStrategies>
</TriggerAction>
```

Element Specifications

Element	Type	Default	Validation	Description
TriggerAction	Tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	Text	CreateGenericReport		The type of trigger action being configured
OutputStrategies	Tag			Tag denoting the beginning of output methods for this trigger action.
OutputStrategy	Tag			The beginning of the configuration information for a single output method.
type	Text		File, Socket, Email, or FTP	The name of the output strategy.
destination	Path			Path where output will be sent.
isRetransmittable	Bool	true	True/False	If selected this will allow attempts at retransmitting if the first attempt fails.
retryIntervalInMinutes	Int	2		The frequency in minutes with which retransmits will be attempted.
maxRetryAttempts	Int	100		The maximum number of retries attempted.

CreatellogReport

Generates a report in the special iLog format that contains statistical process control settings and summary STDF information. See the iLog specification for more details.

Profile Example

```
<TriggerAction type="CreateIlogReport">
  <OutputStrategies>
    <OutputStrategy type="file" destination="reports" isRetransmittable="true"
      retryIntervalInMinutes="2" maxRetryAttempts="100">
    </OutputStrategy>
  </OutputStrategies>
</TriggerAction>
```

Element	Type	Default	Validation	Description
TriggerAction	Tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	Text	CreatellogReport		The type of trigger action being configured
OutputStrategies	Tag			Tag denoting the beginning of output methods for this trigger action.
OutputStrategy	Tag			The beginning of the configuration information for a single output method.
type	Text		File, Socket, Email, or FTP	The name of the output strategy.
destination	Path			Path where output will be sent.
isRetransmittable	Bool	true	True/False	If selected this will allow attempts at retransmitting if the first attempt fails.
retryIntervalInMinutes	Int	2		The frequency in minutes with which retransmits will be attempted.
maxRetryAttempts	Int	100		The maximum number of retries attempted.

CreateLotStatusReport

The mid-lot report is a human readable (HTML) summary of the state of testing at the time the report was generated. It contains a snapshot of the yield, binning counts, etc. for that instant in time.

Profile Example

```
<TriggerAction type="CreateLotStatusReport">
  <OutputStrategies>
    <OutputStrategy type="file" destination="reports" isRetransmittable="true"
      retryIntervalInMinutes="2" maxRetryAttempts="100">
    </OutputStrategy>
  </OutputStrategies>
</TriggerAction>
```

Element Specifications:

Element	Type	Default	Validation	Description
TriggerAction	Tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	Text	CreateLotStatusReport		The type of trigger action being configured
OutputStrategies	Tag			Tag denoting the beginning of output methods for this trigger action.
OutputStrategy	Tag			The beginning of the configuration information for a single output method.
type	Text		File, Socket, Email, or FTP	The name of the output strategy.
destination	Path			Path where output will be sent.
isRetransmittable	Bool	true	True/False	If selected this will allow attempts at retransmitting if the first attempt fails.
retryIntervalInMinutes	Int	2		The frequency in minutes with which retransmits will be attempted.
maxRetryAttempts	Int	100		The maximum number of retries attempted.

CreateLotSummaryReport

Contains a record of lot attributes, testing summary, binning summary, triggers activated, and test statistics (Cp, Cpk, standard deviation, etc.).

Profile Example

```
<TriggerAction type="CreateLotSummaryReport">
  <OutputStrategies>
    <OutputStrategy type="file" destination="reports" isRetransmittable="true"
      retryIntervalInMinutes="2" maxRetryAttempts="100">
    </OutputStrategy>
  </OutputStrategies>
</TriggerAction>
```

Element Specifications:

Element	Type	Default	Validation	Description
TriggerAction	Tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	Text	CreateLotSummaryReport		The type of trigger action being configured
OutputStrategies	Tag			Tag denoting the beginning of output methods for this trigger action.
OutputStrategy	Tag			The beginning of the configuration information for a single output method.
type	Text		File, Socket, Email, or FTP	The name of the output strategy.
destination	Path			Path where output will be sent.
isRetransmittable	Bool	true	True/False	If selected this will allow attempts at retransmitting if the first attempt fails.
retryIntervalInMinutes	Int	2		The frequency in minutes with which retransmits will be attempted.
maxRetryAttempts	Int	100		The maximum number of retries attempted.

CreateMidLotReport

The mid-lot report is a machine readable (XML) summary of the state of testing at the time the report was generated. It contains a snapshot of the yield, binning counts, etc. for that instant in time.

Profile Example

```
<TriggerAction type="CreateMidLotReport">
  <OutputStrategies>
    <OutputStrategy type="file" destination="reports" isRetransmittable="true"
      retryIntervalInMinutes="2" maxRetryAttempts="100">
    </OutputStrategy>
  </OutputStrategies>
</TriggerAction>
```

Element Specifications:

Element	Type	Default	Validation	Description
TriggerAction	Tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	Text	CreateMidLotReport		The type of trigger action being configured
OutputStrategies	Tag			Tag denoting the beginning of output methods for this trigger action.
OutputStrategy	Tag			The beginning of the configuration information for a single output method.
type	Text		File, Socket, Email, or FTP	The name of the output strategy.
destination	Path			Path where output will be sent.
isRetransmittable	Bool	true	True/False	If selected this will allow attempts at retransmitting if the first attempt fails.
retryIntervalInMinutes	Int	2		The frequency in minutes with which retransmits will be attempted.
maxRetryAttempts	Int	100		The maximum number of retries attempted.

CreateOutlierDetectionReport

Reports lot attributes, detailed information about each outlier method and outliers detected by that method, and a summary of all outlier devices and the methods that detected each of them.

Profile Example

```
<TriggerAction type="CreateOutlierDetectionReport">
  <OutputStrategies>
    <OutputStrategy type="file" destination="reports" isRetransmittable="true"
      retryIntervalInMinutes="2" maxRetryAttempts="100">
    </OutputStrategy>
  </OutputStrategies>
</TriggerAction>
```

Element Specifications:

Element	Type	Default	Validation	Description
TriggerAction	Tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	Text	CreateOutlierDetectionReport		The type of trigger action being configured
OutputStrategies	Tag			Tag denoting the beginning of output methods for this trigger action.
OutputStrategy	Tag			The beginning of the configuration information for a single output method.
type	Text		File, Socket, Email, or FTP	The name of the output strategy.
destination	Path			Path where output will be sent.
isRetransmittable	Bool	True	True/False	If selected this will allow attempts at retransmitting if the first attempt fails.
retryIntervalInMinutes	Int	2		The frequency in minutes with which retransmits will be attempted.
maxRetryAttempts	Int	100		The maximum number of retries attempted.

CreateOutlierEventReport

This XML report includes detailed information about outliers that were detected during the lot (devices, tests, results, and violated parameters). It is also sent to the YMS database to track alerts.

Profile Example

```
<TriggerAction type="CreateOutlierEventReport">
  <OutputStrategies>
    <OutputStrategy type="file" destination="reports"
      isRetransmittable="true"
      retryIntervalInMinutes="2" maxRetryAttempts="100">
    </OutputStrategy>
  </OutputStrategies>
</TriggerAction>
```

Element Specifications

Element	Type	Default	Validation	Description
TriggerAction	Tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	Text	CreateOutlierEventReport		The type of trigger action being configured
OutputStrategies	Tag			Tag denoting the beginning of output methods for this trigger action.
OutputStrategy	Tag			The beginning of the configuration information for a single output method.
type	Text		File, Socket, Email, or FTP	The name of the output strategy.
destination	Path			Path where output will be sent.
isRetransmittable	Bool	true	True/False	If selected this will allow attempts at retransmitting if the first attempt fails.
retryIntervalInMinutes	Int	2		The frequency in minutes with which retransmits will be attempted.
maxRetryAttempts	Int	100		The maximum number of retries attempted.

CreatePDLReport

The Parametric Datalog (PDL) report contains raw test data in a binary format. Test data are accessed using supplementary applications designed for that purpose.

Profile Example

```
<TriggerAction type="CreatePDLReport">
  <OutputStrategies>
    <OutputStrategy type="file" destination="reports" isRetransmittable="true"
      retryIntervalInMinutes="2" maxRetryAttempts="100">
    </OutputStrategy>
  </OutputStrategies>
</TriggerAction>
```

Element Specifications:

Element	Type	Default	Validation	Description
TriggerAction	Tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	Text	CreatePDLReport		The type of trigger action being configured
OutputStrategies	Tag			Tag denoting the beginning of output methods for this trigger action.
OutputStrategy	Tag			The beginning of the configuration information for a single output method.
type	Text		File, Socket, Email, or FTP	The name of the output strategy.
destination	Path			Path where output will be sent.
isRetransmittable	Bool	true	True/False	If selected this will allow attempts at retransmitting if the first attempt fails.
retryIntervalInMinutes	Int	2		The frequency in minutes with which retransmits will be attempted.
maxRetryAttempts	Int	100		The maximum number of retries attempted.

CreateSamplingReport

Contains lot attributes, sampled tests settings, triggers activated, per-device summary (x-coordinate, y-coordinate, hardware bin, software bin, and etc.), and skipped tests.

Profile Example

```
<TriggerAction type="CreateSamplingReport">
  <OutputStrategies>
    <OutputStrategy type="file" destination="reports" isRetransmittable="true"
      retryIntervalInMinutes="2" maxRetryAttempts="100">
    </OutputStrategy>
  </OutputStrategies>
</TriggerAction>
```

Element Specifications

Element	Type	Default	Validation	Description
TriggerAction	Tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	Text	CreateSamplingReport		The type of trigger action being configured
OutputStrategies	Tag			Tag denoting the beginning of output methods for this trigger action.
OutputStrategy	Tag			The beginning of the configuration information for a single output method.
type	Text		File, Socket, Email, or FTP	The name of the output strategy.
destination	Path			Path where output will be sent.
isRetransmittable	Bool	true	True/False	If selected this will allow attempts at retransmitting if the first attempt fails.
retryIntervalInMinutes	Int	2		The frequency in minutes with which retransmits will be attempted.
maxRetryAttempts	Int	100		The maximum number of retries attempted.

ExecuteProxyCommand

Sends a command to the SwiftTestProxy command interface allowing SwiftTestProxy to execute the command.

```
<TriggerAction type='ExecuteProxyCommand'>
  <Proxy host="localhost" port="60001"
    proxyCommand="ProxyExecuteSystemCommand">
    <SystemCommand commandPath="/bin/ls" parameters="-la "
      outputPath="/usr/" />
    </Proxy>
  </TriggerAction>
```

Element Specifications:

Element	Type	Default	Validation	Description
TriggerAction	Tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	Text	ExecuteProxyCommand		The type of trigger action being configured
Proxy	Tag			Begin definition of a SwiftTestProxy instance where commands will be sent.
host	Host			The hostname or IP address of the SwiftTestProxy.
port	Int			The TCP/IP port through which SwiftTest will communicate with the SwiftTestProxy.
proxyCommand	Tag	ProxyExecuteSystemCommand		The command that will be sent to the SwiftTestProxy (see the SwiftTestProxy Users Guide of a complete list of commands).

ExecuteSystemCommand

Reads a command and optional arguments, defined in the device profile, executed in a separate process within SwiftTest-MAX.

Profile Example

```
<TriggerAction type='ExecuteSystemCommand'>
  <Command commandPath="/bin/ls.exe" parameters="-la ${LotId}"
    outputPath="/tmp" />
</TriggerAction>
```

Element Specifications

Element	Type	Default	Validation	Description
TriggerAction	Tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	Text	ExecuteSystemCommand		The type of trigger action being configured
Command	Tag			The Unix/Linux/Windows system command that will be executed.
commandPath	Path	/bin/ls.exe		The full path to the Unix/Linux/Windows system command that will be executed.
parameters	Text	-la \$(LotID)		Command line parameters used by the system command.
outputPath	Path	/tmp		The output path for the results of the system command.

VisualAlert

Through a Visual Alert window in the system's GUI, the system notifies the equipment operator of events or alarms during current lot. This automatically includes basic lot information. Use this action to customize addition alert messages with optional details in the device profile.

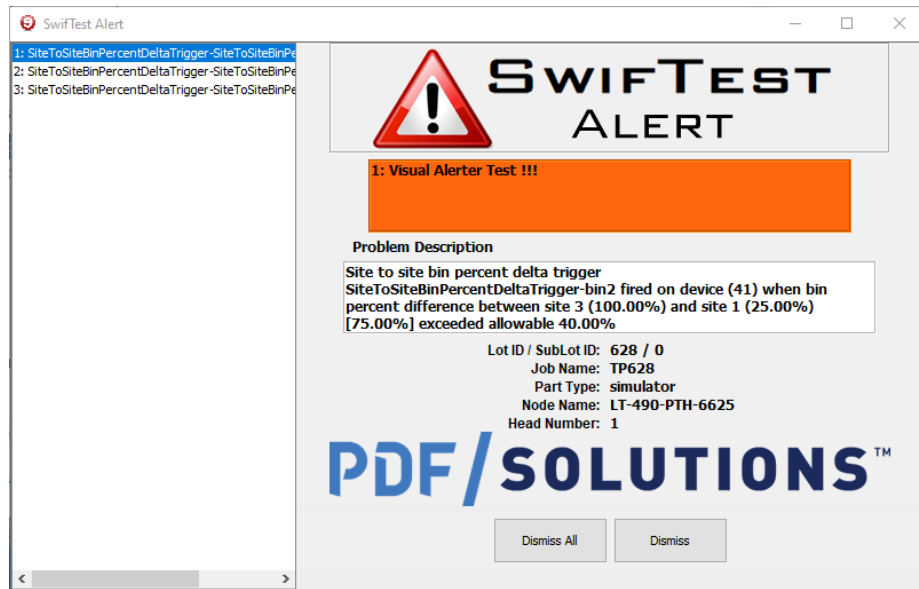
Profile Example

```
<TriggerAction type="VisualAlert">
  <AlertDetails message="This is the message."/>
</TriggerAction>
```

Element Specifications

Element	Type	Default	Validation	Description
TriggerAction	Tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	Text	VisualAlert		The type of trigger action being configured
AlertDetails	Tag			The tag denoting the beginning of the message section of this action.
message	Text	This is the message.		The message to be displayed in the Visual Alert.

When the action is triggered, one or multiple Alert warnings will appear in one window, with a scrolling list of alerts. Operators can select and dismiss the alerts in bulk.



WriteMessageToLog

A very simple trigger action for debugging, its only function is to write a message in the swifttest.log file when the action is executed. This is useful for testing a device profile and ensuring that the triggers are functioning correctly.

Profile Example

```
<TriggerAction type="WriteMessageToLog"/>
```

Element Specifications:

Element	Type	Default	Validation	Description
TriggerAction	Tag		N/A	Denotes the beginning of a new trigger action method configuration.
type	Text	WriteMessageToLog	File, Socket, Email, or FTP	The type of trigger action being configured

REPORTING OUTPUT STRATEGIES

Settings in the Output Strategy determine the transmit method of the report to the end user. Defined below are the four output methods available: File, Socket, Email and FTP.

File Output Strategy

The file output strategy will write the Exensio database data to the specified directory.

Example

```
<OutputStrategy type="File" destination="reports" isRetransmittable=[Bool]
  retryIntervalInMinutes=[Int] maxRetryAttempts=[Int] />
```

Element Specifications

Element	Type	Default	Validation	Description
OutputStrategy	Tag		N/A	Determines how SwiftTest communicates
type	Text		File, Socket, Email, or FTP	Specifies which type of output to use
Destination	Path	reports		The location where the reports should be written to
isRetransmittable	Bool	true	True/False	If SwiftTest should retry delivery of the report if there is a problem
retryIntervalInMinutes	Int	30	1 – 1440	The number of minutes between retries
maxRetryAttempts	Int	100	0 - 1000	The number of allowed retries

Socket Output Strategy

The socket output strategy sends data to a Exensio database socket listener and allows specifying multiple listeners for failover redundancy. SwiftTest does not send to all specified listeners at once but iterates through an optionally prioritized list until successfully completing a transmission.

Example

```
<OutputStrategy type="Socket" requireChecksum=[Bool] ackTimeoutInSeconds=[Int]
  isRetransmittable=[Bool] retryIntervalInMinutes=[Int]
  maxRetryAttempts=[Int]>
  <Listeners>
    <Listener host=[Host] port=[Int] enabled=[Bool]/>
  </Listeners>
</OutputStrategy>
```

Element Specifications:

Element	Type	Default	Validation	Description
OutputStrategy	Tag		N/A	Determines how SwiftTest communicates
type	Text		File, Socket, Email, or FTP	Specifies which type of output to use
requireChecksum	Bool	true	True/False	Indicates if SwiftTest should verify the checksum returned by the socket listener.
ackTimeoutInSeconds	Int	30	1 - 3600	The amount of time that the listener is allowed to take to generate the ack message
isRetransmittable	Bool	true	True/False	If SwiftTest should retry delivery of the report if there is a problem
retryIntervalInMinutes	Int	30	1 - 1440	The number of minutes between retries
maxRetryAttempts	Int	100	0 - 1000	The number of allowed retries
Listeners	Tag			Contains one or more Listener elements.
Listener	Tag			Information about a specific listener instance.
host	Host	localhost	A valid host name or IP address	The system where the listener is running
port	Int	7926	1024-65,535	The port that the listener is listening on
enabled	Bool	true	True/False	Enables or disables this entry

FTP Output Strategy

Writes the Exensio database data to a specified FTP server or multiple FTP servers to provide fail over redundancy. SwiftTest does not send to all specified FTP servers at once but iterates through an optionally prioritized list until successfully completing a transmission.

Example

```
<OutputStrategy type="FTP" isRetransmittable=[Bool]
  retryIntervalInMinutes=[Int]
    maxRetryAttempts=[100]>
  <FtpServers>
    <FtpServer host=[Host] user=[Text] password=[Text] directory=[Path] />
  </FtpServers>
</OutputStrategy>
```

Element Specifications:

Element	Type	Default	Validation	Description
OutputStrategy	Tag		N/A	Determines how SwiftTest communicates
type	Text		File, Socket, Email, or FTP	Specifies which type of output to use
isRetransmittable	Bool	true	True/False	If SwiftTest should retry delivery of the report if there is a problem
retryIntervalInMinutes	Int	30	1 - 1440	The number of minutes between retries
maxRetryAttempts	Int	100	0 - 1000	The number of allowed retries
FtpServers	Tag			Contains one or more FTP servers
FtpServer	Tag			
host	Host		A valid host name or IP address	The system where the FTP server is running
user	Text			The user name to use when logging into the server
password	Text			The password to use when logging into the server
directory	Path	/		The remote directory to upload the data to
enabled	Bool	true	True/False	Enables or disables this entry

Email Output Strategy

Sends the Exensio database data to an account on an SMTP server or Multiple SMTP servers to provide failover redundancy. SwiftTest does not send to all SMTP servers at once but iterates through a list until successfully completing a transmission. However, the data can be sent to all recipients of a list if instructed.

Example

```
<OutputStrategy type="Email" isRetransmittable=[Bool]
retryIntervalInMinutes=[Int]
  maxRetryAttempts=[Int]>
  <SmtpServers>
    <SmtpServer host=[Host] user=[Text] password=[Text]
      requiresAuthentication=[Bool]/>
  </SmtpServers>
  <EmailMessage sender=[Text] subject=[Text] body=[Text]/>
  <EmailRecipients>
    <EmailRecipient name=[Text] email=[Text] type=[Text]/>
  </EmailRecipients>
```

Element Specifications:

Element	Type	Default	Validation	Description
OutputStrategy	Tag		N/A	Determines how SwiftTest communicates
type	Text		File, Socket, Email, or FTP	Specifies which type of output to use
isRetransmittable	Bool	true	True/False	If SwiftTest should retry delivery of the report if there is a problem
retryIntervalInMinutes	Int	30	1 – 1440	The number of minutes between retries
maxRetryAttempts	Int	100	0 - 1000	The number of allowed retries
SmtpServers	Tag			Contains one or more SMTP servers to use for sending email
SmtpServer	Tag			The beginning tag of a single instance of an SMTP server.
host	Host		A valid host name or IP address	The system where the FTP server is running
user	Text			The user name to use when logging into the server
password	Text			The password to use when logging into the server
requiresAuthentication	Bool	false	True/False	Indicates if this server requires authentication in order to send email
EmailMessage	Tag			The beginning tag of an email message instance.
sender	Text		A valid email address	The email address of the sender of the message
subject	Text			The subject line of the message

Element	Type	Default	Validation	Description
body	Text			The body text of the message
EmailRecipients	Tag			Contains one or more EmailRecipient elements
EmailRecipient	Tag			The email address of the recipient(s).
name	Text			The name of the recipient
email	Text			The email address of the recipient
type	Text	to	to, cc, bcc	The type of recipient, to, cc, or bcc

Notification Output Strategy

SwiftTest rule trigger alarm message can use the **Notification** output strategy. When the trigger fires it will send a Notification Report.

Example

```
<?xml version='1.0' encoding='UTF-8'?>
<report timestamp="2020-03-29T13:49:36+03:00">
  <LotInfo>
    <StartMillis>1585478972000</StartMillis>
    <FinishMillis>0</FinishMillis>
    <TimeZoneOffsetMillis>7200000</TimeZoneOffsetMillis>
    <LotStartTime>March 29, 2020 1:49:32 PM IDT</LotStartTime>
    <ISOLotStartTime>2020-03-29T13:49:32+03:00</ISOLotStartTime>
    <LotStopTime>N/A</LotStopTime>
    <LotProcessingTime>N/A</LotProcessingTime>
    <IdleTimeMillis>0</IdleTimeMillis>
    <IdleTime/>
    <JobRev>2</JobRev>
    <TesterType>simulator</TesterType>
    <TestTemp>25c</TestTemp>
    <WaferID>1</WaferID>
    <LotID>539</LotID>
    ...
  </LotInfo>
  <ReportType>Rule Event</ReportType>
  <ContentType>zip</ContentType>
  <OriginalFileName>SwiftTest_event_LT-2....zip</OriginalFileName>
  <ContentBase64Encoded>true</ContentBase64Encoded>
  <Content>UESDBBQACAAIA.....</Content>
</report></OutputStrategy>
```

EXAMPLES

EXAMPLE 1

This example shows the configuration of a **LotEndTrigger** with a **CreateLotSummaryReport** trigger action. The output strategy is configured to send the report to a file in the ./reports directory.

```
<Feature type="Trigger" enabled="true">
  <TriggerMethodSet>
    <TriggerMethod type="LotEndTrigger" name="LotEndTrigger" enabled="true">
      <TriggerActions>
        <TriggerAction type="CreateLotSummaryReport">
          <OutputStrategies>
            <OutputStrategy type="file" destination="reports"
              isRetransmittable="true" retryIntervalInMinutes="2"
              maxRetryAttempts="100">
          </OutputStrategy>
        </OutputStrategies>
      </TriggerAction>
    </TriggerActions>
  </TriggerMethod>
</TriggerMethodSet>
</Feature>
```

Element Specifications:

Element	Type	Default	Validation	Description
OutputStrategy	Tag		N/A	Determines how SwiftTest communicates
type	Text		File, Socket, Email, or FTP	Specifies which type of output to use
isRetransmittable	Bool	true	True/False	If SwiftTest should retry delivery of the report if there is a problem
retryIntervalInMinutes	Int	30	1 - 1440	The number of minutes between retries
maxRetryAttempts	Int	100	0 - 1000	The number of allowed retries
SmtpServers	Tag			Contains one or more SMTP servers to use for sending email
SmtpServer	Tag			
host	Host		A valid host name or IP address	The system where the FTP server is running
user	Text			The user name to use when logging into the server
password	Text			The password to use when logging into the server

Element	Type	Default	Validation	Description
requiresAuthentication	Bool	false	True/False	Indicates if this server requires authentication in order to send email
EmailMessage	Tag			
sender	Text		A valid email address	The email address of the sender of the message
subject	Text			The subject line of the message
body	Text			The body text of the message
EmailRecipients	Tag			Contains one or more EmailRecipient elements
EmailRecipient	Tag			
name	Text			The name of the recipient
email	Text			The email address of the recipient
type	Text	to	to, cc, bcc	The type of recipient, to, cc, or bcc

EXAMPLE 2

LotEndTrigger with an **ExecuteSystemCommand** trigger action that executes a simple system command when called.

```
<Feature type="Trigger" enabled="true">
  <TriggerMethodSet>
    <TriggerMethod type="LotEndTrigger" name="LotEndTrigger" enabled="true">
      <TriggerActions>
        <TriggerAction type='ExecuteSystemCommand'>
          <Command commandPath="/bin/ls" parameters="-la" outputPath="/usr/" />
        </TriggerAction>
      </TriggerActions>
    </TriggerMethod>
  </TriggerMethodSet>
</Feature>
```

Element Specifications:

Element	Type	Default	Validation	Description
OutputStrategy	Tag		N/A	Determines how SwiftTest communicates.
type	Text		File, Socket, Email, or FTP	Specifies which type of output to use
isRetransmittable	Bool	true	True/False	If SwiftTest should retry delivery of the report if there is a problem
retryIntervalInMinutes	Int	30	1 - 1440	The number of minutes between retries
maxRetryAttempts	Int	100	0 - 1000	The number of allowed retries
SmtpServers	Tag			Contains one or more SMTP servers to use for sending email
SmtpServer	Tag			
host	Host		A valid host name or IP address	The system where the FTP server is running
user	Text			The user name to use when logging into the server
password	Text			The password to use when logging into the server
requiresAuthentication	Bool	false	True/False	Indicates if server requires authentication to send email
EmailMessage	Tag			
sender	Text		A valid email address	The email address of the sender of the message
subject	Text			The subject line of the message
body	Text			The body text of the message
EmailRecipients	Tag			Contains one or more EmailRecipient elements
EmailRecipient	Tag			
name	Text			The name of the recipient
email	Text			The email address of the recipient
type	Text	to	to, cc, bcc	The type of recipient, to, cc, or bcc

CUSTOM TRIGGER AND TRIGGER ACTIONS

Use SwiftTest-MAX to quickly and easily create custom triggers and trigger actions. Implemented in Java and packaged as a JAR files, simply drop these new custom modules into an existing instance of SwiftTest. Apply them immediately by adding the new triggers and trigger actions configuration information to a device profile.

REPORTS

HEARTBEAT.XML

The **heartbeat.xml** file is sent to Exensio –Test TDS as soon as SwiftTest starts and every XX minutes thereafter (configurable). The frequency that the heartbeat.xml file is sent is configured in the swifttest.xml. Any changes to the swifttest.xml file require a restart of SwiftTest to pick up the changes. Inside this file is:

1. version='7.1.0-0'
2. current_time='2018-03-06T08:48:16-06:00'
3. boot_time='2018-03-06T08:48:14-06:00'
4. node='LT-240-DFW-5455'
5. ip_address='192.168.1.12'
6. operating_system='Windows 7'
7. operating_system_version='6.1'
8. current_swifttest_directory_size='2,065,500,164 bytes'
9. max_heap_space='954,466,304 bytes'
10. current_memory_usage: ='9,109,504 bytes'
11. free_disk_space='4,499,099,648 bytes'
12. license_expiration='2018-09-06T24:00:00-06:00'

Heartbeat.xml is used to tell Exensio –Test TDS the current version of SwiftTest and via the “Current_time”, tells TDS that SwiftTest is still running.

Below is an example of a typical Heartbeat.xml.

```
<?xml version='1.0' encoding='UTF-8'?>
<Heartbeat version='7.1.0-0' current_time='2018-03-06T08:48:16-06:00'
boot_time='2018-03-06T08:48:14-06:00' node='LT-240-DFW-5455' ip_ad-
dress='192.168.1.12' operating_system='Windows 7' operating_system_ver-
sion='6.1' current_swifttest_directory_size='2,065,500,164 bytes'
max_heap_space='954,466,304 bytes' current_memory_usage: ='9,109,504 bytes'
free_disk_space='4,499,099,648 bytes' license_expiration='' />
```

DIERESULTS REPORTS

```

<DieResults Clear="False" IPAddress="10.10.9.3"
Guid="94077603985d7536_-7de018fb_1626369ff90_-7fff" Descrip-
tor="5401-001-20180326-104520"
SystemId="vmc-ts-vmss01">
  <DieResult>
    <X>2</X>
    <Y>5</Y>
    <DieType>TESTED</DieType>
    <Bin>2</Bin>
    <SBin>2</SBin>
    <Site>0</Site>
    <Pass>0</Pass>
  </DieResult>
  <DieResult>
    <X>2</X>
    <Y>6</Y>
    <DieType>TESTED</DieType>
    <Bin>1</Bin>
    <SBin>1</SBin>
    <Site>1</Site>
    <Pass>1</Pass>
  </DieResult>
  <DieResult>
    <X>2</X>
    <Y>7</Y>
    <DieType>TESTED</DieType>
    <Bin>1</Bin>
    <SBin>1</SBin>
    <Site>2</Site>
    <Pass>1</Pass>
  </DieResult>
  <DieResult>
    <X>2</X>
    <Y>8</Y>
    <DieType>TESTED</DieType>
    <Bin>1</Bin>
    <SBin>1</SBin>
    <Site>3</Site>
    <Pass>1</Pass>
  </DieResult>
</DieResults>

```

The Parametric Data feature DieType information distinguishes between first pass, inline retest and inline QA retest.

CHAPTER 3

SWIFTEST COMMANDS

REMOTE COMMAND INTERFACE

SwiftTest ships with a set of remote commands that can be used when integrating with external systems.

AbortLot The **AbortLot** command forces SwiftTest to abort processing of a currently running lot. SwiftTest will stop processing additional data from this lot and begin lot end processing.

Syntax `Command=AbortLot;GUID=<The GUID obtained from the OpenLots command>`

Response `<Response Command='AbortLot' CommandStatus='Success' ReportTime='2012-03-06T16:22:27-06:00' />`

AddRemoteSystemEventListener

The **AddRemoteSystemEventListener** command allows systems to register to be notified of events. This requires a server socket to be running on the listener that can accept “callbacks” from SwiftTest in response to events. The event messages are:

```
Event=LotMonitorCreated;GUID=<The GUID>
Event=LotMonitorClosed;GUID=<The GUID>
Event=DateProcessingStarted;GUID=<The GUID>
Event=DateProcessingEnded;GUID=<The GUID>
Event=LotIdentified;GUID=<The GUID>
Event=FinalStatisticsStarted;GUID=<The GUID>
Event=FinalStatisticsEnded;GUID=<The GUID>
Event=LotEndProcessingStarted;GUID=<The GUID>
Event=LotEndProcessingEnded;GUID=<The GUID>
Event=LotMonitorIdleEventStarted;GUID=<The GUID>
Event=LotMonitorIdleEventEnded;GUID=<The GUID>
```

Syntax `Command=AddRemoteSystemEventListener;host=<The listening host name or IP address>;port=<The listening port number>`

Response <Response Command='AddRemoteSystemEventListener'
 CommandStatus='Success' ReportTime='2012-08-07T13:34:28-05:00'><AddRemoteSystemEventListener
 instanceId='fb6c0ba46a341f44_31e0cd05_1390259087c_-8000' /></Response>

BeginLotMonitorSession

The **BeginLotMonitorSession** begins a lot monitor session.

Syntax Command= BeginLotMonitorSession;DataFilePath=<The path to the STDF file>;ProfileFilePath=<The path to the device profile>

Response <Response Command='BeginLotMonitorSession'
 CommandStatus='Success' ReportTime='2012-08-07T13:49:35-05:00' />
 <Response Command='BeginLotMonitorSession'
 CommandStatus='Error' ReportTime='2012-08-07T13:48:12-05:00' Message='Unable to locate data file
 C:\SwiftTest\data\archive\MAX3997_F_SV1ZAU003C_FT_ROOM_20120103201630.std' />

BeginParametricDataSession

Enables Parametric Data on demand functionality. When parametric data is turned off by default, this command turns it again. Then it will be reset to the *off* default at the end of each lot. This is a way to get real-time data from the tester, while looking at the Spotfire template for it.

This command backfills the results to beginning of lot.

Syntax Command=beginParametricDataSession;GUID=[put GUID here];start=true

BeginSamplingSession

The **BeginSamplingSession** command begins a sampling session and returns the port of the data pump.

Syntax Command=
 BeginSamplingSession;LotID=MyLotID;SubLotID=MySubLotID;NodeName=MyNodeNameJobName=MyJobName;PartType=MyPartType;HeadNumber=1

Response Status=Success;Port=60003

BeginSocketLotMonitorSession

The **BeginSocketLotMonitorSession** returns an XML document with the port for the requested socket connection.

This command will:

- Accept and use an externally set GUID.
- Set the default **DateStrategy**.
- Return a string acknowledgment with the port to be used.

Syntax Command= BeginSocketLotMonitorSession;ProfileFilePath=<The path to the device profile>

Response <Response Command='BeginSocketLotMonitorSession'
CommandStatus='Success' ReportTime='2012-08-07T14:49:27-05:00' port='1386' />



*The **BeginSocketLotMonitorSession** command replaces the **OpenNewStdSocketConnection** command (previously used by the Agent).*

BeginSwiftTestProxyConnection

The **BeginSwiftTestProxyConnection** returns an XML document with the connection status of the current lot.

Syntax Command=BeginSwiftTestProxyConnection;
GUID=550247a84a9220b7_-23c804d4_135ea16f27b_-7fff;Time=Hello

Response <Response Command='BeginSwiftTestProxyConnection'
CommandStatus='Error' ReportTime='2012-08-07T14:58:38-05:00' Message='Fatal error' />

BinSummary The **BinSummary** returns an XML document with the current bin counts for the specified lot.

Syntax Command=BinSummary;GUID=<The GUID obtained from the OpenLots command>

DisableTestRunTriggers

The **DisableTestRunTriggers** disables the SPC triggers for a given GUID.

Syntax `Command=DisableTestRunTriggers;GUID=550247a84a9220b7_-23c804d4_135ea16f27b_-7fff`

Response `<Response Command='DisableTestRunTriggers'
CommandStatus='Success' ReportTime='2012-03-06T16:18:44-06:00' />`

EndSamplingSession

The **EndSamplingSession** command will end a sampling session.

Syntax `Command=EndSamplingSession;SessionID=2352553`

Response `Status=Success`

EndSwifttestProxyConnection

The **EndSwiftTestProxyConnection** command will close a SwiftTest proxy connection.

Syntax `Command= EndSwiftTestProxyConnection;
GUID=550247a84a9220b7_-23c804d4_135ea16f27b_-7fff;Time=;Csum=927364`

Response

ForwardMessage

The **ForwardMessage** command will forward a message string to Exensio database (could be TDS or YMS) if the **ExensioTestIntegration** feature in SwiftTest is enabled.

Syntax `Command=ForwardMessage;Message=<A text message>`

Response `<Response Command='ForwardMessage' CommandStatus='Error'
ReportTime='2012-03-06T15:17:11-06:00' Message='Message forwarding is disabled.' />`

GetActiveTriggers

The **GetActiveTriggers** command returns a list of all active triggers defined for the current lot.

Syntax Command=GetActiveTriggers

Response <Response Command='GetActiveTriggers'
 CommandStatus='Success' ReportTime='2016-07-06T11:00:04-07:00'><GetActiveTriggers><TriggerInfo
 GUID='94077603985d7536_26a01238_155c15db631_-7fff'
 NodeName='vmc-ts-vmss01' IPAddress='10.10.9.3'
 Type='LotEndTrigger' Name='LotEndTrigger' FireCount='0'
 Scope='lotend' Enabled='true' /></GetActiveTriggers></Response>

GetDeviceProfile

The **GetDeviceProfile** command will return the device profile in use for the specified lot.

Syntax Command=GetDeviceProfile;GUID=<The GUID obtained from the
 OpenLots command>

GetHistory The **GetHistory** command will return an XML document listing all SwiftTest startups, shutdowns, lot starts, and lot ends for the specified time period.

Syntax Command=GetHistory;Days=<The number of days worth of
 history, or 'ALL'>

Response <Response Command='GetHistory' CommandStatus='Success'
 ReportTime='2012-08-07T17:21:24-05:00'><EventList><Event
 type='STARTUP' date='2012-08-07T13:28:51-05:00'
 version='6.3.5' /><Event type='STARTUP' date='2012-08-07T13:29:14-05:00' version='6.3.5' /><Event
 type='LOT_START' date='2012-08-07T13:49:37-05:00'
 guid='fb6c0ba46a341f44_31e0cd05_1390259087c_-7ffe'
 lotId='SV1ZAU003C' subLotId='' jobName='MAX3997_F'
 partType='myPartType' nodeName='myNodeName' headNumber='1'
 lotClass='Unknown' profileName='myProfile.profile.xml' /><Event type='LOT_END' date='2012-08-07T13:49:54-05:00'
 guid='fb6c0ba46a341f44_31e0cd05_1390259087c_-7ffe' /></EventList></Response>

GetLibDirectory The **GetLibDirectory** command asks SwiftTest what lib directory it is running from.

Syntax Command=getLibDirectory

GetLogFile The **GetLogFile** command will return the requested log file (zipped).

Syntax Command=GetLogFile;Filename=<The log file name obtained from the GetLogFileList command>

GetLogFileList The **GetLogFileList** command will return an XML document listing all available log files. Each entry contains the log file name, current size, and a flag to indicate if it is a live log file or an archive log file.

Syntax Command=GetLogFileList

Response <Response Command='GetLogFileList' CommandStatus='Success' ReportTime='2012-08-07T17:23:40-05:00'><LogFileList><LogFile filename='swifttest-kevinxp.log' date='2012-08-07T17:23:40-05:00' size='83985' live='true' /><LogFile filename='swifttest-kevinxp-2012-07-05.log.zip' date='2012-07-05T00:00:00-05:00' size='6637' live='false' /></LogFileList></Response>

GetLotDetails The **GetLotDetails** command will return an XML document containing the full set of known lot metadata. This includes all the information contained in the STDF MIR and SDR records.

Syntax Command=GetLotDetails;GUID=<The GUID obtained from the OpenLots command>

GetOpenDataSourceList

The **GetOpenDataSourceList** command will return an XML document containing a list of all open data sources.

Syntax Command= GetOpenDataSourceList

Response <Response Command='GetOpenDataSourceList' CommandStatus='Success' ReportTime='2012-08-10T11:42:07-05:00'><GetOpenDataSourceList><DataSource

```

Type='StdRecordFactory' URI='file:/C:/swiftest/data/
archive/MAX3997_F_SV1ZAU003C_FT_ROOM_20120103201630.std'
GUID='ee2cc9635ce5f055_78b8e802_13911698a1a_-7ffc'/
><DataSource Type='StdRecordFactory' URI='file:/C:/
swiftest/data/archive/
ftxwea013ac_QA_ROOM_01142012_042446_wafer19.std'
GUID='ee2cc9635ce5f055_78b8e802_13911698a1a_-7ffe'/
><DataSource Type='StdRecordFactory' URI='file:/C:/
swiftest/data/archive/N8C057-04_20120503_231512.STDF'
GUID='ee2cc9635ce5f055_78b8e802_13911698a1a_-7ffa'/></
GetOpenDataSourceList></Response>

```

GetSettings

The **GetSettings** command will return an XML document containing the current *swiftest.xml* configuration file embedded as CDATA.

Syntax

Command=GetSettings

Response

```

<Response Command='GetSettings' CommandStatus='Success'
ReportTime='2012-08-07T17:17:36-
05:00'><GetSettings><Settings
source='config\swiftest.xml'><![CDATA[<?xml version='1.0'
encoding='UTF-8'?>
<SwiftTestSettings>
  <ServerSettings commandPort='60000'
maxLotMonitorCount='5' captureEnabled='false'
captureDir='data/capture'>
.
.
.
</SwiftTestSettings>]]></Settings></GetSettings></Response>

```

LicenseInfo

The **LicenseInfo** command will return an XML document listing all checked out licenses.

Syntax

Command=LicenseInfo

Response

```

<Response Command='LicenseInfo' CommandStatus='Success'
ReportTime='2012-08-07T17:30:42-
05:00'><LicenseInfo><MonitorLicenses/><SamplingLicenses/
><PATLicenses/><OutlierLicenses/><DynamicTestLicenses/></
LicenseInfo></Response>

```

OpenLots

The **OpenLots** command will return an XML document listing all currently running lots.

Syntax Command=OpenLots

Response

```
<Response Command='OpenLots' CommandStatus='Success'
ReportTime='2012-08-10T11:14:56-05:00'><OpenLots><LotInfo
JobName='MyJobName' PartType='MyPartType'
NodeName='Tester052' HeadNumber='1' LotID='MyLotID'
SubLotID='' StartTime='2012-01-14T04:31:58-06:00'
WaferID='19' DeviceCount='10' LotClass='FIRSTPASS'
LastRecordTime='2012-08-10T11:14:56-05:00' State='Lot
Identified' GUID='ee2cc9635ce5f055_1b4f9f05_139114f2dd5_-
7ffe' URI='file:/C:/swiftest/data/archive/MySTDFFile.std'/
><LotInfo JobName='MyJobName' PartType='MyPartType'
NodeName='Tester9328' HeadNumber='1' LotID='MyLotID'
SubLotID='' StartTime='2012-05-03T23:15:12-05:00'
WaferID='04' DeviceCount='12' LotClass='FIRSTPASS'
LastRecordTime='2012-08-10T11:14:56-05:00' State='Lot
Identified' GUID='ee2cc9635ce5f055_1b4f9f05_139114f2dd5_-
7ffa' URI='file:/C:/swiftest/data/archive/
<MySTDFFile.STDF'><LotInfo JobName='MyJobName'
PartType='MyPartType NodeName='Tester040' HeadNumber='1'
LotID='MyLotID' SubLotID='' StartTime='2012-01-
03T20:16:30-06:00' WaferID='' DeviceCount='61'
LotClass='UNKNOWN' LastRecordTime='2012-08-10T11:14:56-
05:00' State='Lot Identified'
GUID='ee2cc9635ce5f055_1b4f9f05_139114f2dd5_-7ffc'
URI='file:/C:/swiftest/data/archive/MySTDFFile.std'></
OpenLots></Response>
```

OpenNewStdDataConduitConnection

The **OpenNewStdDataConduitConnection** command will return a port value where a client can initiate an STDF data stream through a socket connection.

Syntax Command= OpenNewStdDataConduitConnection

Response Status=Success;Port=60001

RemoveRemoteSystemEventListener

The **RemoveRemoteSystemEventListener** command will remove a listener that previously registered to receive events with the **AddRemoteSystemEventListener** command.

Syntax Command=RemoveRemoteSystemEventListener;host=<The listening host name or IP address>;port=<The listening port number>

Response	<pre><Response Command='RemoveRemoteSystemEventListener' CommandStatus='Success' ReportTime='2012-08-10T11:25:27- 05:00' /></pre>
SetCapture	The SetCapture command will turn on or off the capture feature of SwiftTest. Note that this has no effect on already processing lots. The capture setting will revert to what is configured in <i>swifttest.xml</i> the next time SwiftTest is restarted.
Syntax	<code>Command=SetCapture;Value=<true or false></code>
Response	<pre><Response Command='SetCapture' CommandStatus='Success' ReportTime='2012-08-10T11:26:48-05:00' /></pre>
SetDebug	The SetDebug command will turn on or off debug logging. The log level setting will revert to what is configured in <i>log4j.xml</i> the next time SwiftTest is restarted.
Syntax	<code>Command=SetDebug;Value=<true or false></code>
Syntax	<pre><Response Command='SetDebug' CommandStatus='Success' ReportTime='2012-08-10T11:28:01-05:00' /></pre>
SetLotClass	The SetLotClass command will turn set the lot class of the running lot. This will not affect the lot class as reported in the ilog. It is used to disable triggers and alerts if for example the failing parts are retested at the end of a lot.
Syntax	<code>Command= SetLotClass;NewLotClass=QA</code>
Response	<pre><Response Command='SetLotClass' CommandStatus='Success' ReportTime='2012-08-10T11:28:01-05:00' /></pre>
Shutdown	The Shutdown command will cause SwiftTest to begin shutting down. SwiftTest will stop accepting new lots but will complete processing of any running lots when issued the Shutdown command.
Syntax	<code>Command=Shutdown;When=0</code>
Response	<pre><Response Command='Shutdown' CommandStatus='Success' ReportTime='2012-08-10T11:29:50-05:00' /></pre>

ShutdownNow The **ShutdownNow** command will cause SwiftTest to shut down immediately. All running lots will be aborted and no lot end processing will occur.

Syntax Command=ShutdownNow

Response `<Response Command='ShutdownNow' CommandStatus='Success' ReportTime='2012-08-10T11:29:50-05:00' />`

SiteSummary The **SiteSummary** command will return an XML document describing the counts and yields for each site for the specified lot.

Syntax Command=SiteSummary;GUID=<The GUID obtained from the OpenLots command>

Stop The **Stop** command will cause SwiftTest to shut down immediately. All running lots will be finished and all lot end processing will occur.

Syntax Command=Stop

Syntax `<Response Command='Stop' CommandStatus='Success' ReportTime='2012-08-10T11:35:10-05:00' />`

SystemStatus The **SystemStatus** command will return an XML document containing information about the running SwiftTest instance, including the version, last startup date, running lots, and total lots processed since last started.

Syntax Command=SystemStatus

Response `<Response Command='SystemStatus' CommandStatus='Success' ReportTime='2012-08-10T11:33:46-05:00'><ServerStatus><SystemInfo SwiftTestVersion='6.3.5' SwiftTestBuild='2206' SwiftTestRevision='4842' SwiftTestBuildDate='2012-07-05T14:05:37-05:00' RunningSince='2012-08-10T11:31:38-05:00' /><ActivityStatus CurrentMemUsage='337551360' LotsProcessedSinceStartup='3' LotsCurrentlyInProcess='2' /></ServerStatus></Response>`

TriggerReset

The **TriggerReset** command resets all statistics/counters used to evaluate a trigger. Returns a series of questions from the command-line that user answers.

Syntax `Command=TriggerReset;GUID=<The GUID obtained from the
OpenLots command>;TriggerId=<The Trigger Id for the trigger
to reset>;Site=<Site number to reset or 'All'>`

Response `<Response Command='TriggerReset' CommandStatus='Success'
ReportTime='2016-07-06T11:25:05-07:00' />`

COMMAND-LINE INTERFACE

SwiftTest provides methods for displaying information about available commands, and about sending commands to a running instance through a command-line window. Command-line windows should be opened from (or navigated to) the [SWIFT TEST_HOME] directory. Command options are listed by entering the command **swiftest -help**. — for example:

```
D:\PDF\Projects\swiftest>swiftest -help
```

swiftest USAGE

-abort_lot	<option> “guid” to have SwiftTest abort processing of the specified lot.
-capture	<option> “on” to enable data capture, “off” to disable it. (Takes effect on the next lot.)
-config	Forces SwiftTest to reread configuration file.
-debug	<option> “on” to enable debug logging, “off” to disable it.
-engineering	Opens Engineering Console.
-help	Print this message.
-info	Display information about SwiftTest.
-libDir	Asks E –T SwiftTest what lib directory it is running from.
-lots	Display information about currently running lots.
-shutdown	Initiate a graceful shutdown of SwiftTest.
-start	Start SwiftTest. Optional lib and noPurge arguments can be used with the -start command. On the command line, you would type: swiftest -start lib noPurge OR swiftest -start lib2 noPurge
-stop	Stops SwiftTest (same as “shutdown now”).
-triggerReset	Resets trigger data.
-triggers	<option> Trigger Activity. Use “guid” argument for specified lot. (No argument for all lots.)

USAGE EXAMPLES

```
D:\PDF\Projects\swifttest>swifttest -abort_lot
94077603985d7536_40374aa6_1567ae9f9cd_-7ffe
```

Done.

```
D:\PDF\Projects\swifttest>swifttest -capture on

Capture mode enabled.
```

```
D:\PDF\Projects\swifttest>swifttest -capture off

Capture mode disabled.
```

```
D:\PDF\Projects\swifttest>swifttest -config

(Re-reads portions of the SwiftTest configuration file,)
```

```
D:\PDF\Projects\swifttest>swifttest -debug on

Debug mode enabled.
```

```
D:\PDF\Projects\swifttest>swifttest -debug off

Debug mode disabled.
```

```
D:\PDF\Projects\swifttest>swifttest -engineering

(Opens SwiftTest Console.)
```

```
D:\PDF\Projects\swifttest>swifttest -info

Pintail SwiftTest 7.0.0, build: @BUILD_NUMBER@, revision:
@REVISION_NUMBER@, build date: Thu Aug 11 11:43:09 PDT 2016

Lots currently processing: 1
```

```
Started: Thu Aug 11 11:42:49 PDT 2016
```

Lots processed since startup: 1

Current memory usage: 257294336

D:\PDF\Projects\swiftest>swiftest -lots

Node, LotId, SubLotId, JobName, PartType, LotClass,
DeviceCount, GUID

vmc-ts-vmsse01, 5401, 0, TestProgram1, simulator, GRR, 20,
94077603985d7536_40374aa6_1567ae9f9cd_-7ffe

D:\PDF\Projects\swiftest>swiftest -shutdown

(Shuts down SwiftTest.)

D:\PDF\Projects\swiftest>swiftest -start

SwiftTest 7.0.0 is starting up

Starting the scheduler service... OK

Starting logging service... OK

Starting dynamic class loader service... OK

Starting the History service... OK

Starting device profile service... OK

Starting lot monitor Service... OK

Starting persistence service... OK

Starting reporting service... OK

Starting the Parametric Data service... OK

Starting FeatureLicenseManager Service... OK

Starting data communication service... OK

Starting the SwiftTest Web service...2016-08-11
11:40:49.007:INFO::jetty-7.2.0.v20101020

2016-08-11 11:40:49.145:INFO::Started
SelectChannelConnector@0.0.0.0:6080

```
D:\PDF\Projects\swifttest>swifttest -stop
```

```
SwiftTest is shutting down.
```

```
D:\PDF\Projects\swifttest>swifttest -triggerReset
```

```
Select GUID:
```

```
1 - 94077603985d7536_1aef7526_155c1745891_-7fff
```

```
2 - quit
```

```
1
```

```
94077603985d7536_1aef7526_155c1745891_-7fff (y/n?)
```

```
y
```

```
Select from available triggers:
```

```
1 - BinFrequencyTrigger
```

```
2 - LotEndTrigger
```

```
3 - quit
```

```
1
```

```
Reset site data for: BinFrequencyTrigger (y/n?)
```

```
y
```

```
Select Sites to reset:
```

```
1 - site(0)
```

```
2 - site(1)
```

```
3 - site(2)
```

```
4 - site(3)
```

```
5 - All
```

```
6 - quit
```

```
5
```

```
Reset Trigger data for - guid:
```

```
94077603985d7536_1aef7526_155c1745891_-7fff, trig
```

```
ger: BinFrequencyTrigger, site: All (y/n?)
```

```
y
```

```
Successfully reset trigger site data for:
```

```
BinFrequencyTrigger, Site: All, GUID:
```

94077603985d7536_1aef7526_155c1745891_-7fff

D:\PDF\Projects\swifttest>swifttest -triggers

Guid: 94077603985d7536_-5b1bddf5_155c13f9366_-7fff

 NodeName: vmc-ts-vmss01

 IPAddress: 10.10.9.3

 TriggerType: LotEndTrigger

 TriggerName: LotEndTrigger

 FireCount: 0

 Scope: lotend

 Enabled: true

CHAPTER 4

SWIFTEST FOX

INTRODUCTION TO SWIFTEST-FOX

FOX Outlier detection in SwiftTest operates in one of three scopes: during test run, test-run end, and wafer end. Each scope is explained below:

During Test Run

In this mode of operation, the test program can send to SwiftTest multiple measurements taken from the current DUT and wait for a pass/fail response from SwiftTest before continuing with testing. This is useful for measuring variance and outliers that may occur across a single DUT. For example, the test program can stop-on-fail early in the test run, thus reducing wasted test time on devices with outliers.

Test-Run End

A test-run is defined as a touchdown or insertion. Algorithms operating in this scope will execute at the end of a DUT during index time and are useful for reducing the amount of processing spend at wafer end or for making decisions about future devices. For example, real-time dynamic PAT limits are calculated at test run end and establish the dynamic limits for the next DUT.

Wafer End

SwiftTest-Fox builds a complete data model of the test process, including all parametric and functional test results, and can perform any mode of analysis on that model at wafer end. Nearest neighbor, principal component analysis, and wafer level PAT are all examples of wafer end algorithms. Even though this mode of operation is a post-processing method, many calculations have already been made in real-time, so overall throughput is faster than traditional off-line, post processing.

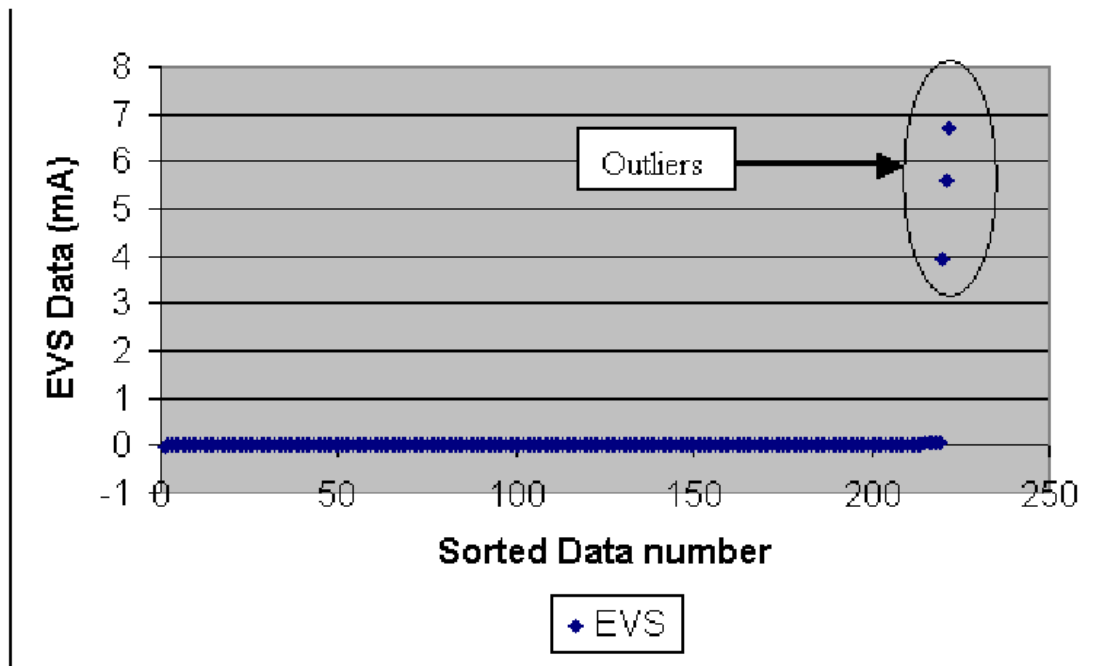
SwiftTest also includes a comprehensive collection of algorithms for each of these scopes, as well as the ability to plug-in custom algorithms. The current catalog of algorithms as well as instructions on how to create new algorithms is detailed in this document.

OUTLIER DETECTION ALGORITHM CATALOG

SwiftTest includes a comprehensive catalog of outlier detection algorithms that can be utilized in various scopes. Selecting an algorithm and performing characterization analysis to identify parameters is beyond the scope of this document.

ADJACENT RANK DELTA

The adjacent rank delta (ARD) algorithm identifies an outlier as being outside of a set of limits defined by a multiple of the average absolute difference between adjacent ranks of the main, defect-free population as defined by the lower and upper percentiles of the ranked measurements. ARD reduces sensitivity to parameter changes with measurement sequence, e.g. die self-heating effect on Iddq with vector activity.



EXAMPLE OF OUTLIERS FROM ADJACENT PATTERN DELTAS IN AFTER/BEFORE STRESS MINVDD DIFFERENCES

General ARD Process

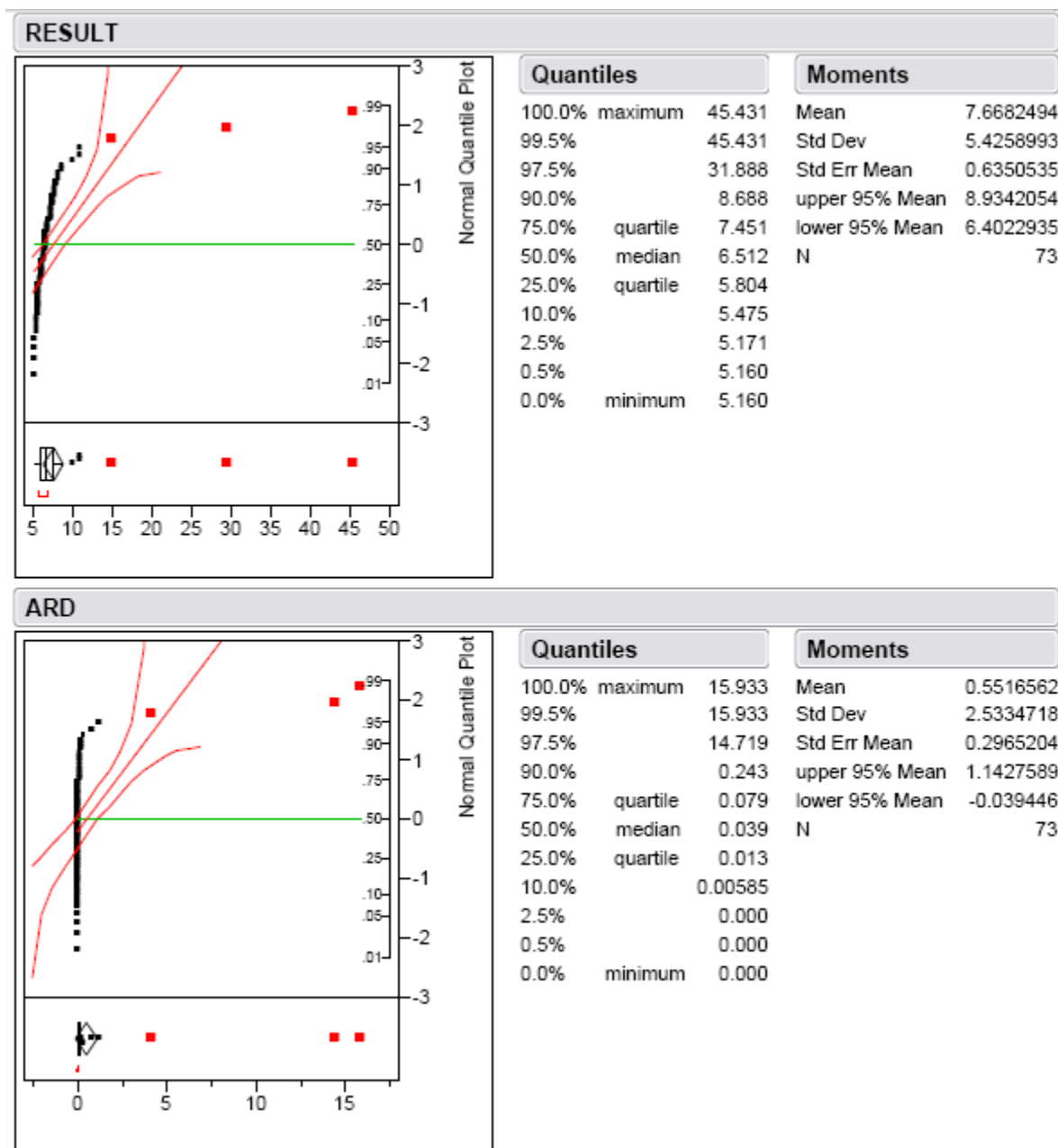
Summary of the ARD process in SwiftTest:

1. In preproduction characterization, establish the lower and upper percentiles of the main, defect-free population, **LP** and **UP**.
2. Calculate the absolute value of the delta between adjacent points (ranks) between the **LP** and **UP**.

3. Take the average of the absolute values of these deltas and call it the *mean adjacent rank delta*.
4. Calculate the *multiplier* of the *mean adjacent rank delta* for the desired acceptance probability for the assumed distribution.
5. Outside the range from **LP** to **UP**, compare the adjacent rank deltas to *mean adjacent rank delta*^X *multiplier*^X *N*. Where *N* is the number of measurements in the sample and is needed to compensate the average delta for the decrease in delta as more points are fit into the same interpercentile range. If the adjacent rank delta is greater, then the die is a failure. It would also make sense to use the interpercentile range of the adjacent rank deltas rather than the original measurements to define the main population from which the limit is calculated.

ARD over Interquartile Range Process (IQR)

1. Calculate the delta between adjacent points between the 25th and 75th percentiles of the original data (not the deltas).
2. Take the average absolute value of all of the deltas and call it the *mean adjacent rank delta* MARD.
3. Calculate the *multiplier*, it should be $k^X N/2.7$. Where *k* is the number of sigmas. *k*=4 will be used for this example.) *N* is the number of measurements in the sample. And 2.7 is the estimate of the standard deviation of the mean delta.
4. Starting at the 25th percentile and going down; if the delta between the value at *i+1* and *i* is greater than *mean adjacent rank delta*^X *multiplier*, then the die is a failure.
5. Starting from the 75th percentile and going up; if the delta between value *i* and *i-1* is greater than *mean adjacent rank delta*^X *multiplier*, then the die is a failure.



NORMAL PROBABILITY & BOX PLOTS OF MAX. IDDQ (RESULT) OVER VECTOR SET AND ARD OF IDDQ

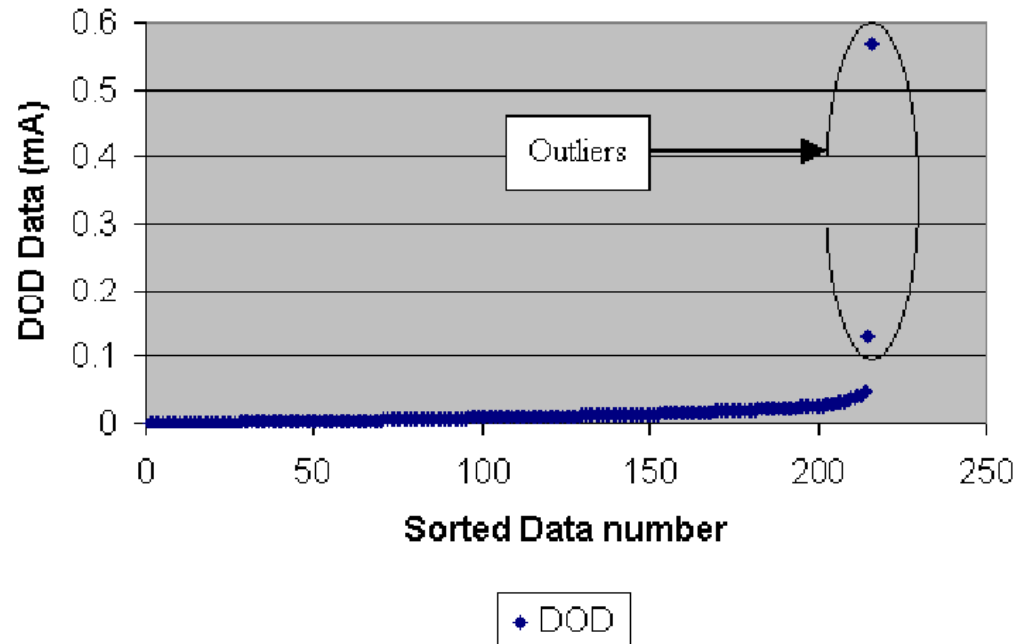
ARD has a sharper distribution than the current from which it is derived thus making outliers more distinct.

Example: Difference of Difference – DoD

Look at the difference with stress and the difference of some parameter measurements. You can use either signed difference or absolute value.

Advantage This method is useful in finding small Iddq defects, i.e. those which cause a small difference in current when activated by stress. These are presently applied only to Iddq:

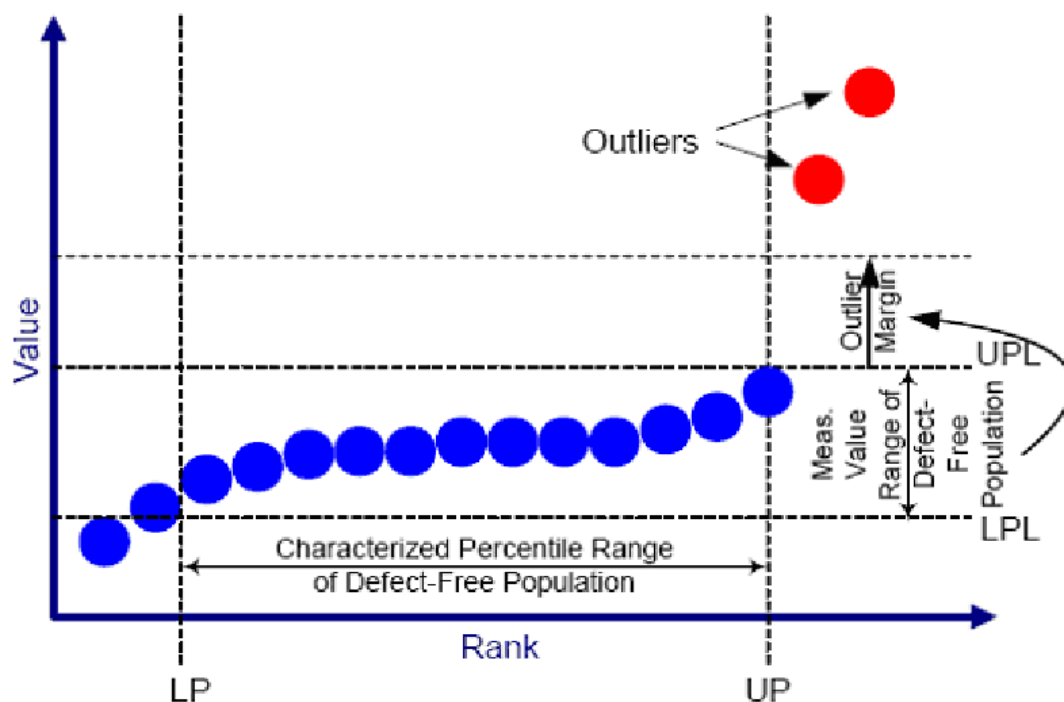
- $\text{Diff_A} = (\text{Max. over vector set}) - (\text{Min. over vector set})$ After voltage stress
- $\text{Diff_B} = (\text{Max. over vector set}) - (\text{Min. over vector set})$ Before voltage stress
- $\text{Diff_C} = \text{Diff_A} - \text{Diff_B}$
- $\text{Diff_D} = |\text{Diff_C}|$



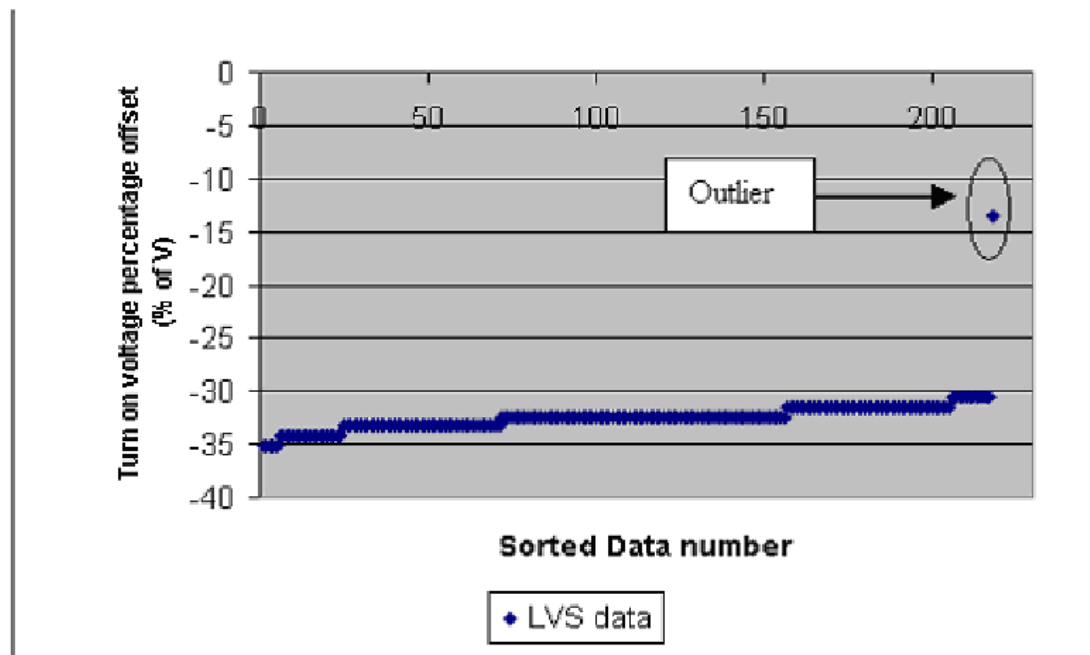
EXAMPLE OF OUTLIERS FROM DIFFERENCE WITH STRESS OF IDDQ VECTOR DIFFERENCE

INTERPERCENTILE RANGE - IPR

The interpercentile range algorithm identifies an outlier as being outside of a set of limits defined by a multiple of the range of the main, defect-free population as defined by the lower and upper percentiles of the ranked measurements. The advantage of this method is that it is easy to compute and insensitive to extreme outliers and distribution asymmetry.



INTERPERCENTILE RANK PROCESS



EXAMPLE OF RANKED MIN VDD DATA INDICATING OUTLIER

General Interpercentile Range (IPR) Procedure

The upper and lower percentiles (**UP** and **LP**, respectively) defining the main, defect-free population must be determined prior to production test.

1. Measure the parameter on every die on wafer and record value with die location.
2. Rank values from lowest to highest.
3. Identify values at the upper and lower limit percentiles (**UPL** and **LPL**, respectively) which were previously characterized to rarely include defects. Compute the interpercentile range **IPR** = **UPL** – **LPL**.
4. Identify die locations corresponding to values outside some multiple of **IPR** plus the median, as determined by the distribution assumed, and re-assign to failure bins in data file.

PRINCIPAL COMPONENT ANALYSIS

Project the measurements over the vector set on their principal components. The largest of the normalized (to its variance) principal component coefficients, is the signature of the variability of the measurement with vector set. This is called the X-statistic. Rank order the X-statistics of the parts in the sample and apply interpercentile range outlier test to find the bad parts. This produces a current signature that makes use of the measurements at all the vectors without having to assume a shape or distribution. Proven correlation to other tests.

Principal component analysis (PCA) seeks to represent the joint variation of a set of variables with the fewest number of uncorrelated new variables synthesized as linear combinations of the original ones. The first principal component (PC) accounts for the most variation a single component can while the second accounts for as much of the remaining variation as it can and so on. A set of p variables has p corresponding PCs where, depending on the correlation of the original variables, a sufficient portion of their variation can be represented by a subset of the PCs. Thus PCA is very useful in reducing the dimensionality of a problem such as statistical design and simulation of a circuit over process variation. It also can be used to identify the fewest and most important tests to assure the analog performances of a circuit.

But the purpose of PCA can be inverted to identify outliers. Since PCA efficiently describes the natural, expected variation of a set of variables, i.e. their signature, what it does not describe can be viewed as the extrinsic, unexpected behavior, i.e. the outliers.

Principal component analysis is based on eigen decomposition of a square matrix. PCA for making a test decision starts with choosing the matrix that best represents the variation of the data. Simple to compute is the covariance matrix, which measures how the variables vary together and has as its diagonal the variances (squares of the standard deviations) of the individual variables.

Given a data matrix A whose p columns are response variables, e.g. tests, with unit u and whose n rows are observations, e.g. parts, of those variables, the covariance C is:

$$C = \frac{A_{zero}^T \times A_{zero}}{n - 1}$$

(1)

...where A_{zero} is matrix A with the mean of each column subtracted from all its elements. This computation may be more clear in its element-wise description:

$$C_{ij} = \frac{\sum_{i=1}^n (a_{ji} - m_i)(a_{ij} - m_j)}{n - 1}$$

(2)

...where a_{ij} and c_{ij} are the elements of matrices A and C , respectively and the column means

$$m_j = \frac{\sum_{i=1}^n a_{ij}}{n}$$

The covariance matrix C is square of dimension $p \times p$ and has units of u^2 . Because it has units, the covariance matrix can only be used when the measurements (variables) are all of the same type such as current. However, even when they are of the same type, measurements with greatly different means or variances, such as input leakage and active supply currents, will be unfairly weighted by covariance.

Both of these issues are addressed by using the correlation matrix R , which normalizes the covariance by the product of the standard deviations ($s = \sqrt{\text{variance}}$) of each variable pair.

$$r_{ij} = \frac{c_{ij}}{s_i s_j}$$

(3)

The correlation matrix also is square of dimension $p \times p$ but is unitless. Therefore, the correlation matrix should be used for mixes of variables like oscillator frequencies, supply currents, and offset voltages measuring analog circuit performance while the covariance matrix should be used for same type measurements of similar values like IDDQ measured at multiple states.

The next step is the actual decomposition of the chosen square matrix into its principal components. This is done by performing an eigen decomposition resulting in a $1 \times p$ vector L of eigenvalues and an associated $p \times p$ matrix V of eigenvectors. Eigen decomposition is a complicated function that is available in many mathematical and statistical software toolboxes and function libraries so there is no need to describe how it is done here.

Each of the p principal components is composed of an eigenvalue with units u^2 and a corresponding unitless eigenvector, which are produced in decreasing order of significance. The eigenvalue is the variance of the PC, which is the measure of its significance, i.e. the fraction of the total variance it explains. To reduce the dimensions of the data, the PCs are chosen in order until their eigenvalues sum to the desired fraction of the total variance, usually at the knee of the eigenvalue vs. order “scree” plot.

The eigenvector is the coefficients of the original variables as they are linearly combined to form the PC. As such, these coefficients also give the importance of the original variables in measuring the total variation.

The last step in preparing to make a test decision with PCA is to project, or rotate, the original zero-mean data on the PCs producing an $n \times p$ matrix of what are known as Z -scores. The Z -scores have the same units u as the original data if the covariance was decomposed or are unitless if the correlation was decomposed. The Z matrix’s rows are observations while its columns correspond to PCs.

$$Z = A_{zero}xV \quad (4)$$

Principal Component Outliers

Now we are ready to use the projected data, the Z -scores, to identify outliers from the behavior the PCs describe. This involves reducing the projected data to a single number for each observation (part tested) and then applying a univariate outlier test. In his survey of the literature, Jolliffe [2] identifies four outlier criteria which he labels $d1$ through $d4$. These will also be given descriptive names but they will be easier to understand if another important multivariate metric is described first: the Mahalanobis distance, which will be labeled $d0$.

The Mahalanobis distance is the Euclidean distance of an observation i to the centroid of the projected data sample normalized to its variance.

$$d_{0i} = \sqrt{\sum_{k=1}^p \frac{z_{ik}^2}{l_k}} \quad (5)$$

It is possible to use the Mahalanobis as an outlier criterion but PCA provides more insight into outlier behavior that we can exploit.

Three of Jolliffe’s outlier criteria make use of the distinction between significant PCs, those that explain most of the expected variation, and insignificant PCs, those that explain little of the expected variation. The number of insignificant PCs is q , with $q < p$, so their indices run from $p-q+1$ to p . The assumption is that defects, being unexpected occurrences, will affect the Z -scores of the insignificant PCs more than the expected ones, which should mostly explain the expected process variation to which the defective behavior will contrast.

The simplest criterion based on this insight is the sum of the squares of the insignificant Z-scores, which is Jolliffe's first.

$$d_{1i}^2 = \sum_{k=p-q+1}^p z_{ik}^2$$

(6)

...where z_{ik} is the value of the projection on the k^{th} PC for the i^{th} observation. This can be called the *total defect excursion*. Theoretically, these values should follow a gamma distribution so that can be used to determine which are outliers.

The $d1$ criterion should best apply to data whose insignificant PCs have a narrow range of variances (eigenvalues) or, simpler yet, variances of essentially zero. For data whose insignificant PCs have variances that are appreciable compared to the excursions caused by defects, and that cover a wide range, the Z-score can be normalized to its PC's variance in Jolliffe's second criterion.

$$d_{2i}^2 = \sum_{k=p-q+1}^p \frac{z_{ik}^2}{l_k}$$

(7)

...where l_k is the eigenvalue (variance) of the k^{th} PC. $d1$ can be called the *insignificant Mahalanobis distance* because it is the distance from the mean of the sample of the projections on the insignificant PCs.

Jolliffe also considers a criterion where the Z-scores are weighted by the variances of their PCs so as to emphasize the effect of the first few PCs. This can be called the *emphasized significant PC excursion*.

$$d_{3i}^2 = \sum_{k=1}^p l_k z_{ik}^2$$

(8)

The problem with $d3$ is that the significant PCs it emphasizes are already strongly reflected in the original variables themselves thus obviating the need for the computational difficulty of an eigen decomposition.

Finally, since defects can have very localized effects, Jolliffe describes using the largest absolute normalize excursion of any single insignificant Z-score.

$$d_{4i} = \max_{p-q+1 \leq k \leq p} \left(\frac{z_{ik}}{\sqrt{l_k}} \right)$$

(9)

This avoids the dilution of the sharp and unexpected excursion of one PC caused by adding it to all the other insignificant PCs. This only adds defect contrast when the total of the normalized defect-free Z-scores is comparable to the excursion a defect causes in one of them. This criterion is the same as the X -statistic advocated by Sharma et al.

Because all of these criteria, when they include the effects of defects, can cover wide ranges, their common logarithms are displayed in the remainder of this document. This is further justified by the leakage current that comprises the defect-free IDDQ being lognormally distributed so that taking the logarithm allows the cumulative probabilities of these criteria to be visualized in readily available normal probability plots. Also note that equations 5 and 9 return a simple distance while equations 6, 7, and 8 return the squares of distances. For brevity, in the remainder of this document the criteria will be referred to by their distances, d with a number subscript (d_0 , d_1 , d_2 , d_3 , and d_4) when what is actually meant is the common logarithm of the equation, as written above.

Issues With PCA-based Production Test

In the order of execution of the PCA method, the first thing we need to know is which sets of tests are best represented by covariance and which by correlation. Then we need to classify test sets by their most effective outlier criterion. Finally we need to know measurement resolution this criterion requires to clearly distinguish good from bad.

Whereas it is straightforward to perform PCA outlier detection on an engineering workstation on a dataset after it has been completely acquired, it is another matter to do so on a tester as the data is being collected. The economics of production test demand detecting defects as early in the test flow as possible so that subsequent tests can be skipped. Testers usually don't have much computing power available for statistical processing yet they are such expensive machines that they must spend almost all their time measuring and handling material and can afford to spend little if any additional time processing data.

These considerations may preclude performing frequent eigen decompositions on the tester. In contrast, computing power to process data after the material is no longer on the tester is cheap but this precludes skipping tests. Finally, since the test and decision-making must be completely automatic, no human judgment and interaction may be incorporated into the production flow as is common in academic "proof of concept" studies.

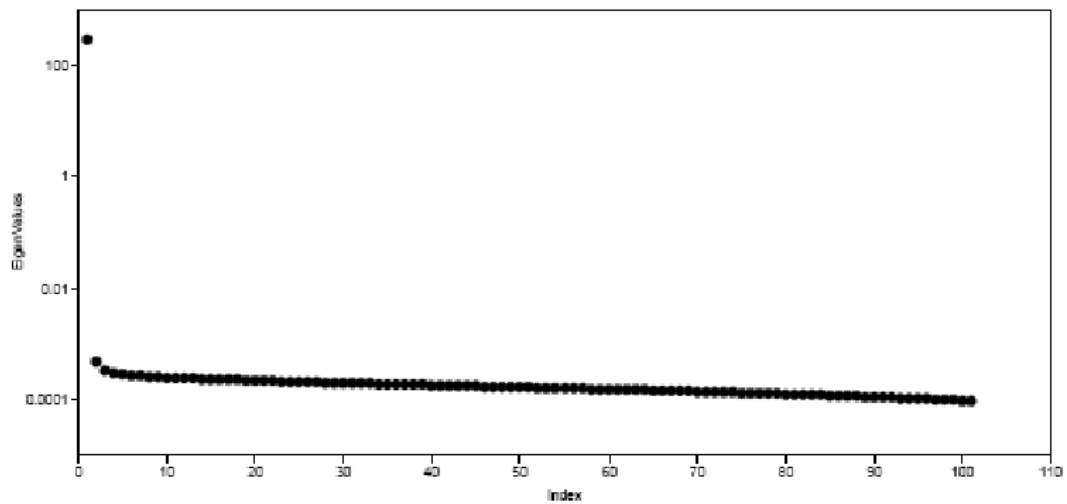
These considerations suggest that a given product design be characterized by an eigen decomposition and outlier criterion and threshold prior to production. As will be shown later, eigen decomposition is affected by outliers, so that creating a principal component signature of a set of tests on a given product is an iterative process requiring human interaction.

The hope is that the eigenvalue vector L and eigenvector matrix V thus decomposed from the covariance or correlation matrix, whichever is most applicable, of the characterization data matrix A then will describe the defect-free behavior of all units of the design that will ever be produced. Similarly it is hoped that the outlier criterion and its decision threshold, also determined in characterization, will apply to all units produced. If these assumptions are valid then production process could be:

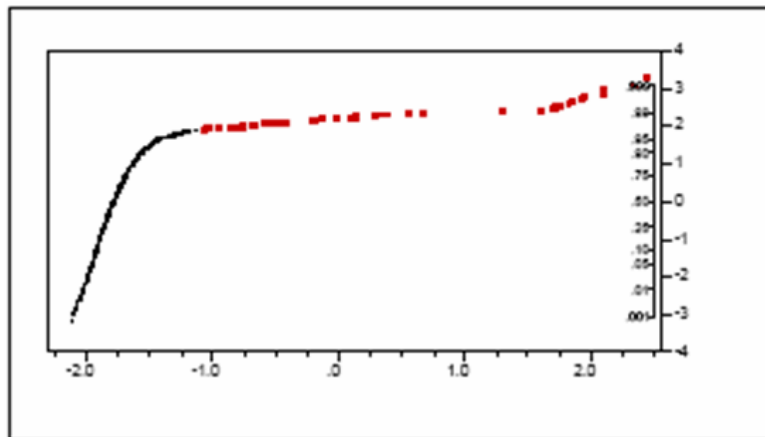
1. Acquire a sample matrix B of the p measurements with sufficient observations i to make a decision. The observations are dice on a wafer or within a package lot.
2. Project the sample matrix with its column means subtracted B_{zero} on the PCs from characterization to obtain its Z -scores. $Z = B_{zero} \times V$. The mathematics will work if B contains a single observation, i.e. is a row vector, such that each observation can be evaluated right after it is measured. However this requires that the means of the variables be determined before the sample is complete, e.g. assumed to be constant at their characterization values.
3. Compute the outlier criterion for each unit i of the sample (row of Z) and compare it to the previously determined constant threshold.

Lot	GR3935		GR3936		GR3937	
Rank	1 st	2 nd	1 st	2 nd	1 st	2 nd
1	293.65	289.31	312.20	302.76	474.99	472.99
2	0.04	0.00	0.17	0.00	0.10	0.00
3	0.03	0.00	0.15	0.00	0.05	0.00
4	0.03	0.00	0.09	0.00	0.05	0.00
5	0.01	0.00	0.08	0.00	0.04	0.00
6	0.01	0.00	0.06	0.00	0.02	0.00
7	0.01	0.00	0.05	0.00	0.01	0.00
8	0.00	0.00	0.04	0.00	0.01	0.00
9	0.00	0.00	0.04	0.00	0.01	0.00
10	0.00	0.00	0.04	0.00	0.01	0.00

EIGENVALUES OF COVARIANCE OF I_{DDQ} BY LOT, DECOMPOSITION BEFORE (1ST) AND AFTER (2ND) d_1 OUTLIER REMOVAL



FINAL EIGENVALUES OF COVARIANCE OF I_{DDQ} FOR LOT GR3936 (LOG SCALE)



NORMAL QUANTILE PLOT OF d_1 OF I_{DDQ} OF LOT GR3936. OUTLIERS IN RED ARE FOR $d_1 > -1.1$

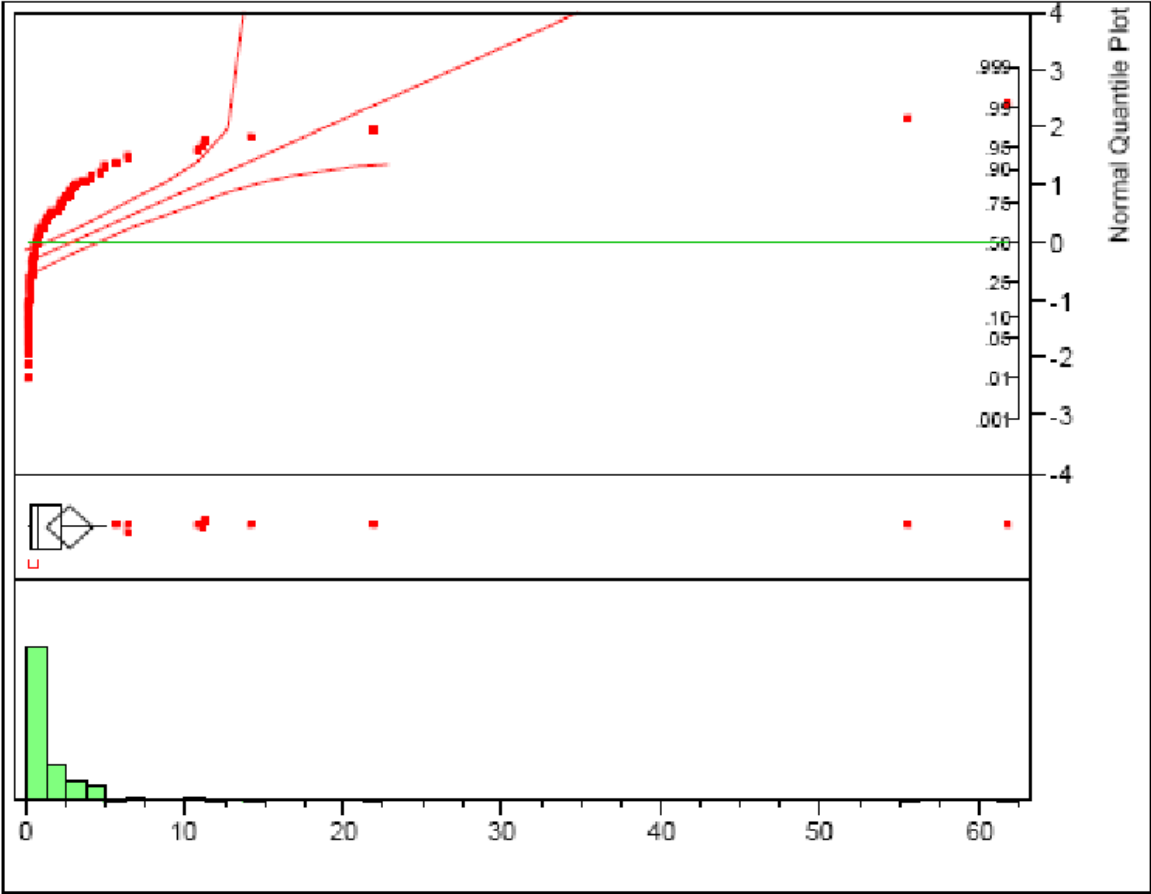
Defect Resolution

The decision metric X is the maximum deviation of the observation (part) along a principal component normalized to the variance of that PC of the defect-free population. In actuality this normalization is to an estimate of the population (the characterization sample) that contains a small number of defects. Ideally, characterization should be iterated until no outliers are identified so that all the defective devices can be assumed to have been eliminated.

The decision thresholds are the limits of the distribution of this metric over the defect-free population. This is similar to a multidimensional coefficient of variation, i.e. threshold value divided by standard deviation. The X -statistic threshold can be unwound into measurement units the same way. The lower bound on the decision resolution is the interval along the least insignificant principal component PC_p .

$$Z_{limit} = \sqrt{l_p} \cdot 10^{x_{limit}}$$

More directly, the resolution can be found by recording the Z score associated with each part's X -statistic and then looking at the Z score distribution for the parts that fail the X -statistic threshold. The lowest Z score of this distribution is the required measurement resolution since Z has the dimension of the measurement.

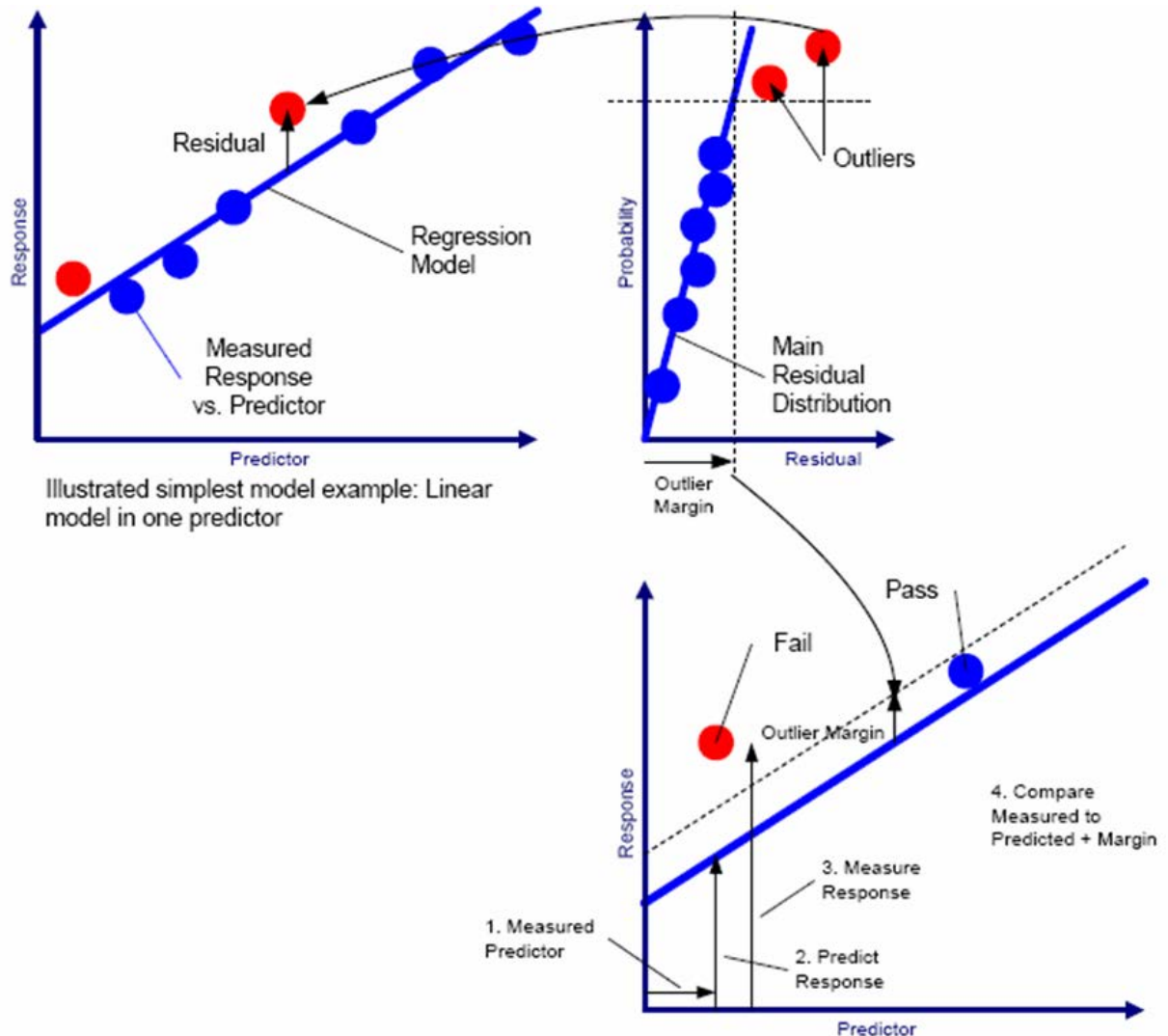


Quantiles	Std Err Mean	
100.0%	upper 95%	minimum
99.5%	Mean	
97.5%	lower 95%	Moments
90.0%	mean	
75.0%	N	
50.0%		
25.0%		
10.0%	maximum	
2.5%		
0.5%		
0.0%		
Moments	quartile	
	median	61.803
	quartile	61.803
Mean		22.717
Std Dev		4.933
		2.254

DISTRIBUTION OF Z AT XMAX OF FAILURES – GIVING RESOLUTION OF 150µA

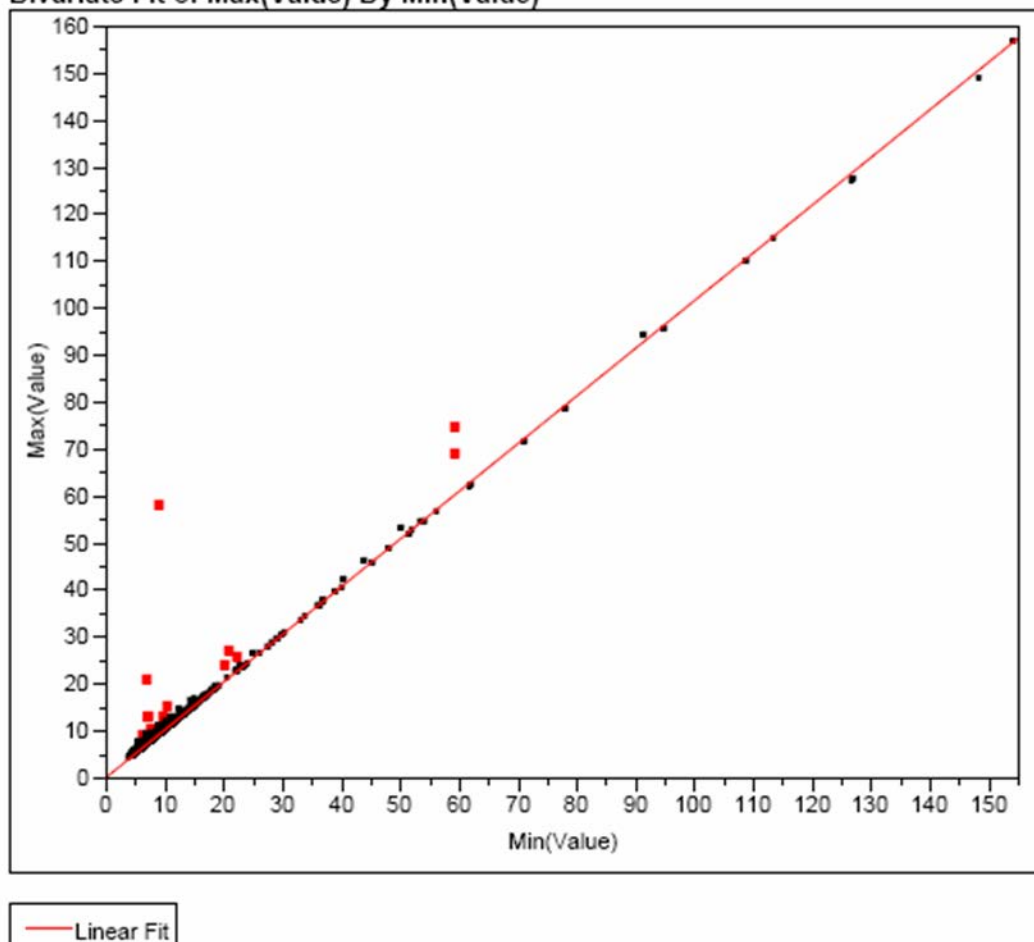
SINGLE REGRESSION RESIDUAL

Single Regression Residual uses a linear regression model to predict the value of one variable from the value of another variable. Compare the measured value of the first variable to its predicted value. The advantage of this method is that it compensates for manufacturing shifts in the defect-free parameter level. Current Ratio: Compensates for change in background (sub-threshold) leakage when current values can't be measured at each pattern but high/low comparison is available. We presently apply this to Iddq at different vectors. Could also apply to min. Vdd at different vectors if we logged the failing vectors instead of just whether any in the set of vectors failed.



REGRESSION OUTLIER PRINCIPAL

Bivariate Fit of Max(Value) By Min(Value)



Linear Fit

$\text{Max(Value)} = 0.0048993 + 1.013919 \text{ Min(Value)}$

Summary of Fit

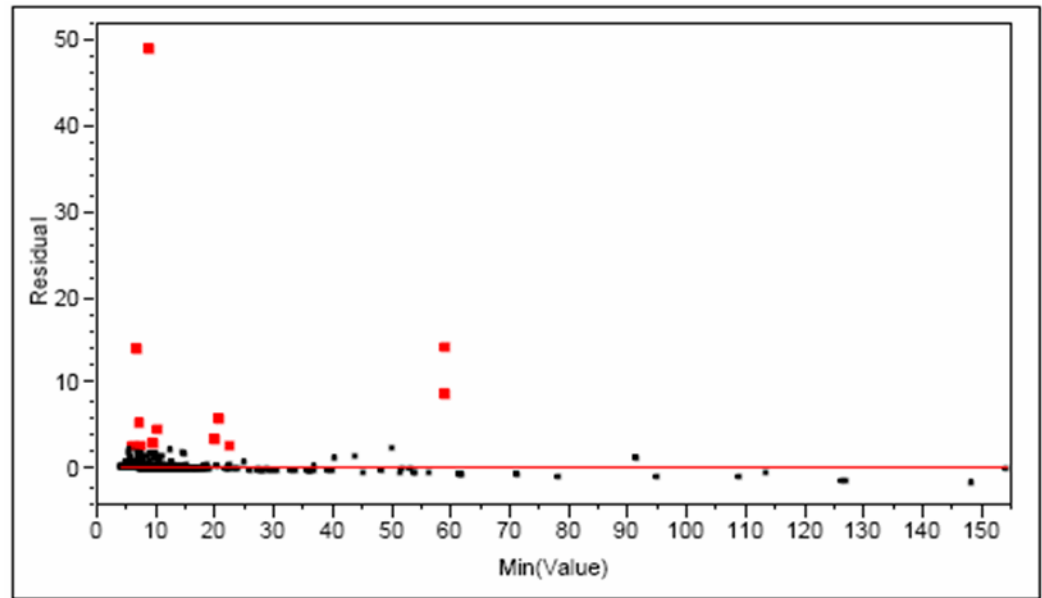
RSquare	0.983707
RSquare Adj	0.983703
Root Mean Square Error	0.773375
Mean of Response	8.464297
Observations (or Sum Wgts)	5076

Analysis of Variance

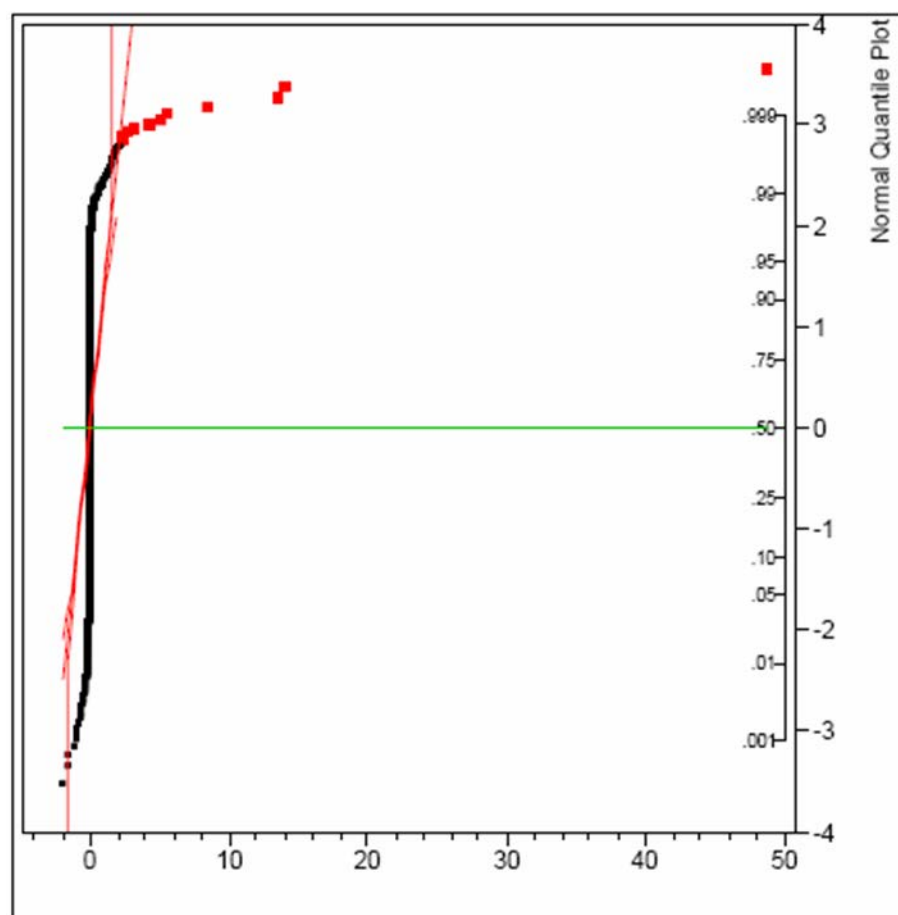
Source	DF	Sum of Squares	Mean Square	F Ratio
Model	1	183225.25	183225	306341.2
Error	5074	3034.80	0.598108	Prob > F
C. Total	5075	186260.06		0.0000

Parameter Estimates

Term	Estimate	Std Error	t Ratio	Prob> t
Intercept	0.0048993	0.018746	0.26	0.7938
Min(Value)	1.013919	0.001832	553.48	0.0000



REGRESSION OF I_{DDQ} MAX. ON MIN.



Quantiles

100.0%	maximum	48.71
99.5%		1.46
97.5%		0.07723
90.0%		-0.0059
75.0%	quartile	-0.018
50.0%	median	-0.0296
25.0%	quartile	-0.042
10.0%		-0.0564
2.5%		-0.0889
0.5%		-0.35
0.0%	minimum	-1.91

Moments

Mean	-2.78e-16
Std Dev	0.7732985
Std Err Mean	0.0108539
upper 95% Mean	0.0212784
lower 95% Mean	-0.021278
N	5076

PROBABILITY PLOT OF REGRESSION RESIDUALS

PART AVERAGE TESTING (PAT) WITH FOX

PAT is also implemented in SwiftTest as a stand-alone tool. See “[SwiftTest Part Average Testing \(PAT\)](#),” pg. 189.

Overview – PAT Within FOX

The Automotive Engineering Council (AEC) outlier detection guideline defines PAT, in part, as the following: “... a statistically based method ...for removing parts with abnormal characteristics from the semiconductors supplied”

SwiftTest’s PAT methods implement the essence of this guideline, taking into account process variation from one wafer to the next or one test setup to the next. SwiftTest takes a baseline number of measurements, then calculates and applies new PAT limits based on the current wafer/lot being tested, then assigns “PAT-bins” to the outliers.

For outlier detection within FOX, PAT is typically used with the wafer-end method: at wafer-end, the system goes back and recalculates outliers and limits based on test results, then re-bins outliers to PAT-bins.

AEC PAT

Standard AEC PAT defines limits as being n standard deviations from the mean. The number of standard deviations on each side of the mean is independently configurable.

Box Plot

Similar to the interpercentile range method, box-plot calculates limits based on percentiles from the median and a margin multiplier.

Robust Sigma

The robust PAT method uses a robust (quartile) based calculation for standard deviation that is more agreeable to non-normal distributions.

PAT By Site

When implemented (configured in the profile), calculates different limits per site.

STDF Update Example

The following profile example includes the STDF update that takes the originally read STDF file and inserts new parametric result records based on the PAT results. It also updates the binning records. The function will generate a new STDF file in a specified destination directory.

```
<Feature type='OutlierDetection' enabled='true'>
  <Options>
    <UpdateStdfile enabled='true' destination='reports' />
  </Options>
  <WaferMethodSet>
    <WaferMethod type='PAT' name='PATAcceptance'
      enabled='true'>
      <Parameters>
        <Parm name='lowerMinOffset' value='0.0' />
        <Parm name='type' value='Standard' />
        <Parm name='upperSigma' value='6.0' />
        <Parm name='lowerSigma' value='6.0' />
        <Parm name='upperMinOffset' value='0.0' />
      </Parameters>
      <DataSet type='ParametricTest'>
        <Parameters>
          <Parm name='testNumber' value='1000' />
        </Parameters>
      </DataSet>
      <DataSet type='ParametricTest'>
        <Parameters>
          <Parm name='testNumber' value='1001' />
        </Parameters>
      </DataSet>
      <DataSet type='ParametricTest'>
        <Parameters>
          <Parm name='testNumber' value='1002' />
        </Parameters>
      </DataSet>
      <DataSet type='ParametricTest'>
        <Parameters>
          <Parm name='testNumber' value='2000' />
        </Parameters>
      </DataSet>
      <DataSet type='ParametricTest'>
        <Parameters>
          <Parm name='testNumber' value='2001' />
        </Parameters>
      </DataSet>
      <DataSet type='ParametricTest'>
        <Parameters>
          <Parm name='testNumber' value='2002' />
        </Parameters>
      </DataSet>
      <SoftwareBin number='2'
        description='PATAcceptanceFailSWBin' />
      <RecordAsTest number='99' />
    </WaferMethod>
  </WaferMethodSet>
</Feature>
```

MPR Example

The following example includes MPR (Multiple Parametric Record), which generates a record of one test with a series of sub-tests, with calculated limits for each of the sub-tests.

```
<WaferMethod type='PAT' name='PAT_MPR_2003'
enabled='true'>
  <Parameters>
    <Parm name='lowerMinOffset' value='0.0' />
    <Parm name='type' value='Standard' />
    <Parm name='upperSigma' value='6.0' />
    <Parm name='lowerSigma' value='6.0' />
    <Parm name='upperMinOffset' value='0.0' />
  </Parameters>
  <DataSet type='MPR'>
    <Parameters>
      <Parm name='testNumber' value='2003' />
    </Parameters>
  </DataSet>
  <SoftwareBin number='2'
description='PATAcceptanceFailSWBin' />
  <RecordAsTest number='88' />
</WaferMethod>
```

DURING TEST OUTLIER DETECTION

SwiftTest can perform outlier detection on data collected while a device is being tested, and return information about that detection to the test program for use while testing that device. Normally this is a pass/fail indicator, however it is possible to return non-boolean values to the test program. It is the responsibility of the test engineer to decide what to do with this information.

All real-time methods take an array of measurements, or a pair of measurement arrays and perform some analysis on them. Measurement pairs are computed using either the difference or ratio of the values.

During test, outlier detection requires that a client library be integrated with the test program. The installation and configuration of the client library is beyond the scope of this document.

The test engineer must decide on an identifier, referred to in the profile as the **RealTimeMethod** name attribute. This identifier is provided in two places, when the measurements are collected in the test program, and in the profile. When SwiftTest receives a vector of measurements to perform real-time outlier detection on it looks in the profile for a **RealTimeMethod** entry with a name attribute that matches the identifier that was supplied with the measurement vector.

For example, the following code is from a 93k test program that sends a vector of data to SwiftTest and waits for SwiftTest to return a pass/fail indicator:

```
int swt_outlier_return=0;
/* testsuite_name is a char[] indicating the name of the data */
/* iddq_array is the measurements, stored in an array of double values */
/* count is the number of measurements in the iddq_array */
swt_outlier_return=spc_send_outliers_single(testsuite_name, iddq_array,
count);
/* Test program is blocked until the outlier method completes */
if(swt_outlier_return != 0) {
    /* indicates an outlier was detected in the data. */
}
```

When the **spc_send_outliers_single** function is called it will transmit the parameters to SwiftTest and wait for a response. This means that the test program will block until a response is received. Typically this takes 15-20 milliseconds.

When SwiftTest receives the data from this function call it looks in the device profile for a **RealTimeMethod** that has a name that matches exactly the `testsuite_name` field. Once the method is found SwiftTest will process the measurement data and return a pass/fail result to the test program.

SwiftTest provides traceability into this process by creating reports that detail all the outlier methods that ran, and optionally by inserting PTR records into an STDF data file. The insertion of PTR records happens at lot (wafer) end, and is done by making a copy of the source STDF file and inserting PTR records during the copy process. The testNumber Parm for the **RealTimeMethod** element describes the test number that will be used for this inserted PTR record.

All real-time outlier methods share a common XML schema. An example and the definitions of the elements follows.

Profile Example

```
<RealTimeMethod type="AdjacentRankDelta" name="ard" enabled="true">
  <Parameters>
    <Parm name="testNumber" value="999999999"/>
    <Parm name="numberOfSigma" value="5.6"/>
    <Parm name="estimatedMeanDeltaStdDev" value="0.8"/>
  </Parameters>
</RealTimeMethod>
```

Profile XML Specification

Element	Type	Default	Validation	Description
RealTimeMethod	Tag			
type	Text			
name	Text			The name given to this algorithm. This must match the name transmitted by the client program with the data.
enabled	Bool	False	True/False	Enable or disable this method
baseline	Int	50	1 -2147483648	The required number of devices to establish a statistical baseline
Parameters	Tag			Contains one or more Param elements
Parm	Tag			
name	Text			The name of the parameter
value	Text			The value assigned to the parameter

ADJACENT RANK DELTA

The adjacent rank delta method takes the allowed number of standard deviations and the estimated mean delta standard deviation as parameters.

Profile Example

```
<RealTimeMethod type="AdjacentRankDelta" name="ard" enabled="true">  
  <Parameters>  
    <Parm name="testNumber" value="999999999"/>  
    <Parm name="numberOfSigma" value="5.6"/>  
    <Parm name="estimatedMeanDeltaStdDev" value="0.8"/>  
  </Parameters>  
</RealTimeMethod>
```

Parameters

Name	Type	Description
testNumber	Int	The test number that the results of this method should be assigned to.
numberOfSigma	Real	The number of sigma to use when calculating the mean adjacent rank delta multiplier.
estimatedMeanDeltaStdDev	Real	The threshold mean delta standard deviation. If the calculated mean delta standard deviation exceeds this value then the result is an outlier

INTERPERCENTILE RANGE

The interpercentile range method takes the upper and lower percentile and margin factor as parameters.

Profile Example

```
<RealTimeMethod type="InterPercentileRange" name="ipr" enabled="true">
  <Parameters>
    <Parm name="testNumber" value="99999999"/>
    <Parm name="lowerPercentile" value="20"/>
    <Parm name="upperPercentile" value="80"/>
    <Parm name="marginFactor" value="1.5"/>
  </Parameters>
</RealTimeMethod>
```

Parameters

Name	Type	Description
testNumber	Int	The test number that the results of this method should be assigned to.
lowerPercentile	Int	The lower percentile
upperPercentile	Int	The upper percentile
marginFactor	Real	The margin factor

SINGLE REGRESSION RESIDUAL

The single regression residual method takes the slope, intercept, and margin as parameters.

Profile Example

```
<RealTimeMethod type="SingleRegressionResidual" name="srr" enabled="true">
  <Parameters>
    <Parm name="testNumber" value="999999999"/>
    <Parm name="slope" value="1.1"/>
    <Parm name="intercept" value="3.2"/>
    <Parm name="margin" value="0.02"/>
  </Parameters>
</RealTimeMethod>
```

Parameters

Name	Type	Description
testNumber	Int	The test number that the results of this method should be assigned to.
slope	Real	The slope value of the regression line
intercept	Real	The intercept value of the regression line
margin	Real	The margin padding

TEST-RUN END OUTLIER DETECTION

SwiftTest can perform outlier detection after a touchdown completes, effectively during index time. This allows some calculations that would normally be done at wafer end to be performed while the wafer is testing, thus reducing the amount of time required to perform wafer end outlier detection.

Test-run end methods can take a test number or series of test numbers and perform calculations on the values recorded for those tests during the last test run.

All test-run end methods share a common XML schema. An example and the definitions of the elements follows.

Profile Example

```
< TestRunMethodtype="MethodName" name="name" enabled="true">
  <DataSet type="MPRRRange">
    <Parameters>
      <Parm name="start" value="300001"/>
      <Parm name="end" value="300101"/>
    </Parameters>
  </DataSet>
  <SoftwareBin number="999"/>
  <Parameters>
    <Parm name="parm1" value="999999999"/>
    <Parm name="parm2" value="5.6"/>
  </Parameters>
</ TestRunMethod>
```

Profile XML Specification

Element	Type	Default	Validation	Description
TestRunMethod	Tag			
type	Text			
name	Text			The name given to this algorithm. This must match the name transmitted by the client program with the data.
enabled	Bool	False	True/False	Enable or disable this method
DataSet	Tag			Describes the data set that will be used for this algorithm.
SoftwareBin				
number	Int			The software bin number to re-bin the device to.
Parameters	Tag			Contains one or more Param elements
Parm	Tag			
name	Text			The name of the parameter
value	Text			The value assigned to the parameter

SINGLE REGRESSION RESIDUAL

The single regression residual method takes the slope, intercept, and margin as parameters.

Profile Example

```
<TestRunMethod type="SingleRegressionResidual" name="WQS" enabled="true">
  <DataSet type="MPRRange">
    <Parameters>
      <Parm name="start" value="300001"/>
      <Parm name="end" value="300101"/>
    </Parameters>
  </DataSet>
  <SoftwareBin number="999"/>
  <Parameters>
    <Parm name="slope" value="0.95"/>
    <Parm name="intercept" value="0.1"/>
    <Parm name="margin" value="0.4"/>
  </Parameters>
</TestRunMethod>
```

This example will apply the single regression residual to the 0th element of each MPR record starting at test number 3000001 and ending at test number 3001001.

Parameters

Name	Type	Description
slope	Real	The slope value of the regression line
intercept	Real	The intercept value of the regression line
margin	Real	The margin padding

WAFER END OUTLIER DETECTION

Wafer end outlier detection is also commonly called wafer post processing. SwiftTest can perform wafer outlier detection as soon as the wafer ends, using data and statistics that were collected while the wafer was being probed, thus reducing the amount of time needed to process the wafer, and eliminating possible data management issues associated with getting a complete test data log to an offline system.

Wafer end methods can take a test number or series of test numbers and perform calculations on the values recorded for those tests.

Wafer end methods use the concept of a “DataSet” to decide what data to execute the method against. There are currently two different data sets defined in SwiftTest:

MPRRRange – A range of consecutively numbered MPR records. SwiftTest uses only the 0th item in each MPR.

```
<DataSet type="MPRRRange">
  <Parameters>
    <Parm name="start" value="300001"/>
    <Parm name="end" value="300101"/>
  </Parameters>
</DataSet>
```

ParametricTest – This data set represents a single parametric measurement for each die.

```
<DataSet type="ParametricTest">
  <Parameters>
    <Parm name="testNumber" value="1000"/>
  </Parameters>
</DataSet>
```

All wafer end methods share a common XML schema. An example and the definitions of the elements follows.

Profile Example

```
<WaferMethod type="MethodName" name="name" enabled="true">
  <DataSet type="MPRRRange">
    <Parameters>
      <Parm name="start" value="300001"/>
      <Parm name="end" value="300101"/>
    </Parameters>
  </DataSet>
  <SoftwareBin number="999"/>
  <Parameters>
    <Parm name="parm1" value="999999999"/>
    <Parm name="parm2" value="5.6"/>
  </Parameters>
</WaferMethod>
```

Profile XML Specification

Element	Type	Default	Validation	Description
WaferMethod	Tag			
type	Text			
name	Text			The name given to this algorithm. This must match the name transmitted by the client program with the data.
enabled	Bool	False	True/False	Enable or disable this method
DataSet	Tag			Describes the data set that will be used for this algorithm.
SoftwareBin				
number	Int			The software bin number to re-bin the device to.
Parameters	Tag			Contains one or more Param elements
Parm	Tag			
name	Text			The name of the parameter
value	Text			The value assigned to the parameter

PRINCIPAL COMPONENT ANALYSIS

The principal component analysis algorithm takes an Eigen vector matrix, Eigen value vector, significant number of principal components, and X max value. Note that the dimensions of the Eigen vector matrix, Eigen value vector, and vector of test results must all be compatible.

The PCA wafer method will insert a PTR record into the updated STDF file indicating the results of the computation.

Profile Example

```
<WaferMethod type="PrincipalComponentAnalysis" name="PCA" enabled="true">
  <DataSet type="MPRRange">
    <Parameters>
      <Parm name="start" value="300001"/>
      <Parm name="end" value="300101"/>
    </Parameters>
  </DataSet>
  <SoftwareBin number="555"/>
  <Parameters>
    <Parm name="testNumber" value="99999999"/>
    <Parm name="eigenVectorMatrix" value="[] []"/>
    <Parm name="eigenValueVector" value="[]"/>
    <Parm name="significantPC" value="3"/>
    <Parm name="xMax" value="-1.50"/>
  </Parameters>
</WaferMethod>
```

This example will apply the single regression residual to the 0th element of each MPR record starting at test number 3000001 and ending at test number 3001001.

Parameters

Name	Type	Description
eigenValueMatrix	Real Matrix	An MxM matrix of Eigen vectors computed during characterization
eigenValueVector	Real Vector	An M length vector of Eigen values
significantPC	Int	The number of significant principal components
xMax	Real	The threshold for the computed X value

INTERPERCENTILE RANGE

The interpercentile range (IPR) method takes the upper and lower percentile and margin factor as parameters. The IPR method takes the largest measurement from each die for the defined dataset and uses it in the IPR calculations.

The IPR wafer method will update the execution limits for the range of tests that it ran against. Only tests that actually had an outlier will have the execution limits updated.

Profile Example

```
< WaferMethod type="InterPercentileRange" name="ipr" enabled="true">
  <DataSet type="MPRRange">
    <Parameters>
      <Parm name="start" value="300001"/>
      <Parm name="end" value="300101"/>
    </Parameters>
  </DataSet>
  <SoftwareBin number="555"/>
  <Parameters>
    <Parm name="lowerPercentile" value="20"/>
    <Parm name="upperPercentile" value="80"/>
    <Parm name="marginFactor" value="1.5"/>
  </Parameters>
</RealTimeMethod>
```

Parameters

Name	Type	Description
lowerPercentile	Int	The lower percentile
upperPercentile	Int	The upper percentile
marginFactor	Real	The margin factor

CHAPTER 5

SWIFTEST TTR

OVERVIEW

SwiftTest TTR (Test Time Reduction) allows tests or groups of tests to execute on a sample of parts rather than all parts. By enabling or disabling sampling based on proven statistical measures of process capability, SwiftTest TTR can dramatically reduce test time without any appreciable impact on quality.

The TTR sampling platform allows complete customization of sampling behavior. Sampling can be based on parametric tests having acceptable $\hat{C}_{pk}$ and $\hat{C}_p$ measurements, and on both functional and parametric tests having failure counts below a configured threshold. The sample rate can be fixed at every n^{th} device.

In addition to providing sampling conditions for process capability, TTR also allows new conditions to be created to fit a specific situation.

Each group has a list of conditions that must be satisfied in order for the group to sample, and a strategy that determines what devices will be sample tested and what devices will not.

Conditions decide **if** a group should be sampled.

Strategies decide **when** a group should be sampled.

There are three phases of preparing a device for sampling:

- **analysis**
- **device profile preparation**
- **test program instrumentation**

This chapter covers basic analysis and device profile preparation. The test program instrumentation is specific to the tester platform and is therefore covered in the installation guide for that platform.

TEST GROUPS

Sampling in SwiftTest TTR is based on the concept of a test group. Generally, when sampling tests, it is most beneficial to sample an entire family of tests, including any setup or tear-down steps, rather than specific tests.

A test group in TTR has three key components:

- **list of tests** that belong to this group and will be sampled
- **sampling strategy** that will be used to decide when to sample
- **list of conditions** (zero or more) that must be satisfied in order for the group to sample

The **list of sampled tests** can include both parametric and functional tests. These tests will all sample or skip together, as a group. There must be at least one test defined for the group.

The **sampling strategy** determines when the group should be sampled, provided all conditions are satisfied. The default strategy is to sample test every n^{th} device, however other options are available including customized strategies. Generally, each test group will sample independently, however it is possible to use a sampling strategy that ties the sampling of one group to another.

The **list of conditions** determines if sampling can occur. The conditions are evaluated after every test run. The list can contain zero or more conditions. If zero conditions are specified, sampling will be active as soon as testing starts and will continue unabated until testing stops.

Profile Example

```
<SampledTestGroup name="group1" enabled="true">
  <SampledTests>
    <ParametricTests>
      <ParametricTest testNumber="1000" subTestNumber="0"/>
      <ParametricTest testNumber="1001" subTestNumber="0"/>
    </ParametricTests>
    <FunctionalTests>
      <FunctionalTest testNumber="1000" subTestNumber="0"/>
    </ FunctionalTests >
  </SampledTests>
  <Strategy type="EveryNthTestRun" rate="10"/>
  <Conditions>
    <Condition type="InStatisticalControl" minCp="1.0" minCpk="1.0"
      baseline="50">
      <ParametricTests>
        <ParametricTest testNumber="1001" subTestNumber="0"/>
      </ParametricTests>
    </Condition>
  </Conditions>
</SampledTestGroup>
```

This profile example specifies three tests for sampling. Parametric tests 1000 and 1001, and functional test 1000. The group defines one condition that evaluates test 1001 for being in statistical control ($\hat{C}_p > 1.0$ and $\hat{C}_{pk} > 1.0$). The condition also specifies a baseline of 50 results. The group will sample every 10th device.

Profile XML Specification

Element	Type	Default	Validation	Description
SampledTestGroup	Tag			
name	Text			The name of the test group. Used for reporting purposes
enabled	Bool	true	True/false	Enables or disabled this test group for sampling.
SampledTests	Tag			Contains the tests that are to be sampled
ParametricTests	Tag			Contains the parametric tests to be sampled
ParametricTest	Tag			
FunctionalTests	Tag		0 - 2147483648	Contains the functional tests to be sampled
FunctionalTest	Tag		0 - 2147483648	
Strategy	Tag			Defines the sampling strategy. See below.
Conditions	Tag			Defines the conditions for this group. See below.
testNumber	Int		A valid test number	The unique test number for this test
subTestNumber	Int	0	The sub test number.	Optional. The sub test number
Description	Text		The test name or description	Optional. The description of the test

SAMPLING CONDITIONS

InStatisticalControl

The **InStatisticalControl** condition is used to measure the process capability of one or more parametric tests. The tests do not have to be part of the sampling group, but can come from anywhere in the test program, even another sampling group. This condition is evaluated after a baseline number of test results are received. It can be configured to accept all results for the parametric tests or only results from passing devices.

The process capability measurements taken are $\hat{C}_{pk}$ and optionally $\hat{C}_p$ and those are evaluated against specified minimums. If at any time during testing either measure drifts below the minimum the condition will fail and sampling will stop.

This condition can optionally reset following an exception if the maximum number of resets has not been exceeded. If reset is enabled and the max reset count has not been exceeded then the condition will reset once the configured number of results has been received.

Profile Example

```
<Feature type="Sampling" enabled="true">
  <SampledTestGroups>
    <SampledTestGroup name="Group1" enabled="true">
      <SampledTests>
        .....
      </SampledTests>
      <Condition>
        <Condition type="InStatisticalControl" minCp="2.0"
          minCpk="2.0" baseline="10" resetEnabled="false"
          resetResultCount="50" maxResetCount="3" windowSize="30"
          excludeFailingDevices="false">
          <ParametricTests>
            <ParametricTest testNumber="1002"
              subTestNumber="0" description="test1002"/>
          </ParametricTests>
        </Condition>
      </Condition>
    </SampledTestGroup>
    .....
  </SampledTestGroups>
</Feature>
```

Profile XML Specification

Element	Type	Default	Validation	Description
Condition	Tag			
type	Text			

Element	Type	Default	Validation	Description
minCp	Real	2.0	A positive real number	Optional. The minimum allowed Cp for this test. If omitted Cp will not be used in evaluating this condition.
minCpk	Real	2.0	A positive real number	The minimum allowed Cpk. In the device profile, or E – T Central, if the Cpk drops below the set value, then Sampling will stop for this test.
baseline	Int	50	1 -2147483648	The required number of devices to establish a statistical baseline
resetEnabled	Bool	false	True/False	Enable this condition to reset following an exception
resetResultCount	Int	50	0 - 2147483648	The number of insertions required before the condition will reset
maxResetCount	Int	3	0 - 2147483648	The maximum number of resets allowed for this condition
excludeFailingDevices	Bool	false	True/False	When enabled excludes test results from devices that fail
windowSize	Int		A positive real number	Rolling window calculation size: calculates Cp/Cpk on the last XX devices, so when new devices are logged, then the calculation is adjusted for the latest specified number of devices. E.g. always calculates Cp/Cpk on the latest "windowSize=100" devices.
ParametricTests	Tag			Contains one or more ParametricTest elements.
ParametricTest	Tag			
testNumber	Int		A valid test number	The unique test number for this parametric test
subTestNumber	Int	0	The sub test number.	Optional. The sub test number. This will be the result index in an MPR record
description	Text		The test name or description	Optional. The description of the test

FailureCount

The **FailureCount** condition is used to control sampling based on the number of failures of one or more tests. The number of failures for each test is evaluated independently, not cumulatively. This is the primary sampling mechanism for functional tests that is built into TTR. This condition is evaluated after a baseline number of test results are received.

This condition can optionally reset following an exception, if the maximum number of resets has not been exceeded. If reset is enabled and the max reset count has not been exceeded, then the condition will reset once the configured number of results has been received.

Profile Example

```
<Condition type="FailureCount" maxFailureCount="2" baseline="50"
  resetEnabled="false" resetResultCount="50" maxResetCount="3">
  <FunctionalTests>
    <FunctionalTest testNumber="1"/>
  </FunctionalTests>
</Condition>
```

Profile XML Specification

Element	Type	Default	Validation	Description
Condition	Tag			
type	Text			FailureCount
maxFailureCount	Int	0	1 -2147483648	When type = FailureCount, the maximum failure count that will be allowed before the condition will fail and sampling will stop.
baseline	Int	50	1 -2147483648	The required number of devices to establish a statistical baseline
resetEnabled	Bool	false	True/False	Enable this condition to reset following an exception
resetResultCount	Int	50	0 - 2147483648	The number of results required before the condition will reset
maxResetCount	Int	3	0 - 2147483648	The maximum number of resets allowed for this condition
ParametricTests	Tag			Contains one or more ParametricTest elements.
ParametricTest	Tag			
testNumber	Int		A valid test number	The unique test number for this parametric test
subTestNumber	Int	0	The sub test number.	Optional. The sub test number. This will be the result index in an MPR record
description	Text		The test name or description	Optional. The description of the test

SAMPLING STRATEGIES

EveryNthTestRun

This sampling strategy will cause sampling to happen every nth device. The rate is configurable. The basic algorithm used is:

If the modulo of the (current test run number – sampling start test run number)/rate = zero, then the current test run is sampled.

Profile Example

```
<Strategy type="EveryNthTestRun" rate="10" />
  <Conditions>
    <Condition type="InStatisticalControl" minCp="1.33"
      minCpk="1.33" baseline="10" ...
  </Conditions>
```

Profile XML Specification

Element	Type	Default	Validation	Description
Strategy	Tag			
type	Text		EveryNthTestRun	
rate	Int	blank	1 -2147483648	The sampling rate.*

***Sampling Rate** If **rate=""** is left blank, then the sampling rate will automatically be based on the calculated Cpk value, where SwiftTest will use the following table to set the Automatic Sample Rate:

CpK	Rate	% of Per Device TTR
100 or above	1000	99.90%
10 or above	500	99.80%
4 or above	100	99.0%
3 or above	50	98.0%
2.5 or above	25	95.0%
2 or above	10	90.0%
1.67 or above	8	87.50%
1.33 or above	6	83.33%

EXAMPLES

EXAMPLE 1

This example shows the traditional SwiftTest sampling method of using the $\hat{C}_p$ and $\hat{C}_{pk}$ of all tests in the group to determine if the group should sample. Tests 1000, 1001 and 1002 will all sample at a rate of 1 in 10 after the baseline of 50 devices has been reached and as long as the minimum $\hat{C}_p$ and minimum $\hat{C}_{pk}$ of the tests stays above 2.0.

```
<SampledTestGroup name="group1" enabled="true">
  <SampledTests>
    <ParametricTests>
      <ParametricTest testNumber="1000" subTestNumber="0"/>
      <ParametricTest testNumber="1001" subTestNumber="0"/>
      <ParametricTest testNumber="1002" subTestNumber="0"/>
    </ParametricTests>
  </SampledTests>
  <Strategy type="EveryNthTestRun" rate="10"/>
  <Conditions>
    <Condition type="InStatisticalControl" minCp="2.0" minCpk="2.0"
      baseline="50" excludeFailingDevices="false" resetEnabled="false"
      resetResultCount="50" maxResetCount="3">
      <ParametricTests>
        <ParametricTest testNumber="1000" subTestNumber="0"/>
        <ParametricTest testNumber="1001" subTestNumber="0"/>
        <ParametricTest testNumber="1002" subTestNumber="0"/>
      </ParametricTests>
    </Condition>
  </Conditions>
</SampledTestGroup>
```

EXAMPLE 2

This example shows three tests in the group with a condition that depends on an external test remaining in statistical control. In this case tests 1000, 1001 and 1002 will all sample at a rate of 1 in 10 after the baseline of 50 devices has been reached and as long as the minimum $\hat{C}_p$ and minimum $\hat{C}_{pk}$ of test 2000 stays above 2.0.

```
<SampledTestGroup name="group1" enabled="true">
  <SampledTests>
    <ParametricTests>
      <ParametricTest testNumber="1000" subTestNumber="0"/>
      <ParametricTest testNumber="1001" subTestNumber="0"/>
      <ParametricTest testNumber="1002" subTestNumber="0"/>
    </ParametricTests>
  </SampledTests>
  <Strategy type="EveryNthTestRun" rate="10"/>
  <Conditions>
    <Condition type="InStatisticalControl" minCp="2.0" minCpk="2.0" baseline="50"
      excludeFailingDevices="false" resetEnabled="false" resetResultCount="50"
      maxResetCount="3">
      <ParametricTests>
        <ParametricTest testNumber="2000" subTestNumber="0"/>
      </ParametricTests>
    </Condition>
  </Conditions>
</SampledTestGroup>
```

EXAMPLE 3

This example shows two test groups that will sample together. Both test groups have the same conditions and strategy, therefore they will sample at the same time. In this case, Group1 has one sampling test — 1000 — and a condition that depends on test 2000 maintaining a minimum $\hat{C}_p$ and minimum $\hat{C}_{pk}$ above 2.0. Group2 has one sampling test, 2000, and a condition identical to Group1. Both of these groups will sample together at a rate of 1 in 10.

```
<SampledTestGroup name="group1" enabled="true">
  <SampledTests>
    <ParametricTests>
      <ParametricTest testNumber="1000" subTestNumber="0"/>
    </ParametricTests>
  </SampledTests>
  <Strategy type="EveryNthTestRun" rate="10"/>
  <Conditions>
    <Condition type="InStatisticalControl" minCp="2.0" minCpk="2.0"
      baseline="50" excludeFailingDevices="false" resetEnabled="false"
      resetResultCount="50" maxResetCount="3">
      <ParametricTests>
        <ParametricTest testNumber="2000" subTestNumber="0"/>
      </ParametricTests>
    </Condition>
  </Conditions>
</SampledTestGroup>

<SampledTestGroup name="group2" enabled="true">
  <SampledTests>
    <ParametricTests>
      <ParametricTest testNumber="2000" subTestNumber="0"/>
    </ParametricTests>
  </SampledTests>
  <Strategy type="EveryNthTestRun" rate="10"/>
  <Conditions>
    <Condition type="InStatisticalControl" minCp="2.0" minCpk="2.0"
      baseline="50" excludeFailingDevices="false" resetEnabled="false"
      resetResultCount="50" maxResetCount="3">
      <ParametricTests>
        <ParametricTest testNumber="2000" subTestNumber="0"/>
      </ParametricTests>
    </Condition>
  </Conditions>
</SampledTestGroup>
```

If the TTR (Test Time Reduction) **EveryNthTestRun** > **rate=""** value is left blank, then the sampling rate will automatically be based on the calculated Cpk value, where SwiftTest will use the following table to set the Automatic Sample Rate:

CpK	Rate	% of Per Device TTR
100 or above	1000	99.90%
10 or above	500	99.80%
4 or above	100	99.0%
3 or above	50	98.0%
2.5 or above	25	95.0%
2 or above	10	90.0%
1.67 or above	8	87.50%
1.33 or above	6	83.33%

EXAMPLE 4

The basic configuration is to define a group of tests to be sampled, e.g. **group1**. Then you define the **SampledTests** in that group. Then you define at least one **Condition**, either of type **InStatisticalControl** (based on Cpk), or type **FailureCount** (max number of fails allowed).

Inside of each of the conditions is a list of tests on which to evaluate Cpk or fail count. You are allowed to evaluate the conditions of the same tests you are going to sample, or completely different tests.

An example profile would look like this:

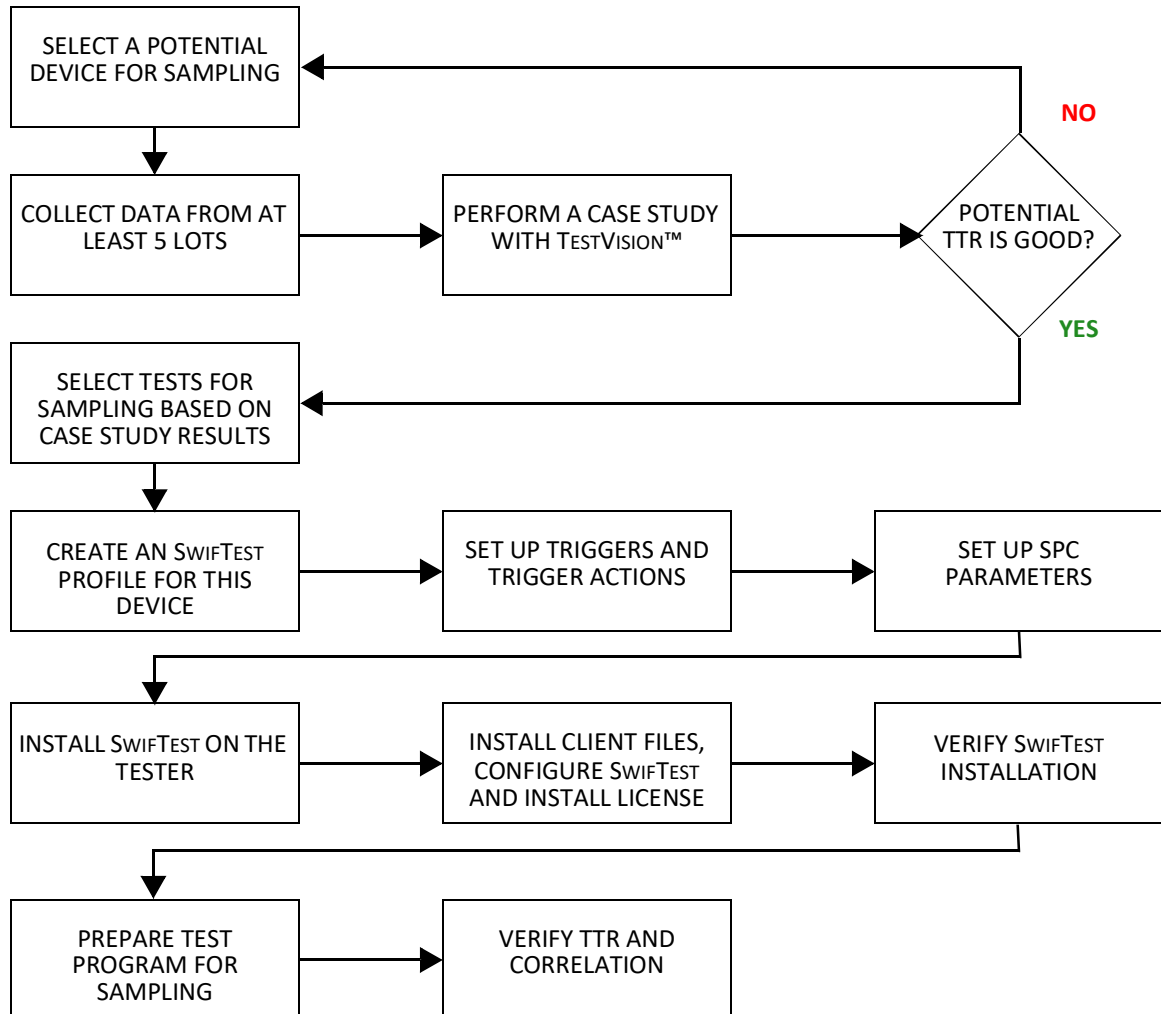
```
<SampledTestGroup name="group1" enabled="true">
  <SampledTests>
    <ParametricTests>
      <ParametricTest testNumber="1000" subTestNumber="0"/>
      <ParametricTest testNumber="1001" subTestNumber="0"/>
      <ParametricTest testNumber="1002" subTestNumber="0"/>
    </ParametricTests>
  </SampledTests>
  <Strategy type="EveryNthTestRun" rate="10"/>
  <Conditions>
    <Condition type="InStatisticalControl" minCp="2.0" minCpk="2.0" baseline="50"
      excludeFailingDevices="false" resetEnabled="false" resetResultCount="50"
      maxResetCount="3">
      <ParametricTests>
        <ParametricTest testNumber="1000" subTestNumber="0"/>  <--These tests are
        usually the same as above, but can be different, or even add more tests that
        the above.
        <ParametricTest testNumber="1001" subTestNumber="0"/>
        <ParametricTest testNumber="1002" subTestNumber="0"/>
      </ParametricTests>
    </Condition>
    <Condition type="FailureCount" maxFailureCount="2" baseline="50"
      resetEnabled="false" resetResultCount="50" maxResetCount="3">
      <FunctionalTests>
        <FunctionalTest testNumber="1"/>
      </FunctionalTests>
    </Condition>
  </Conditions>
</SampledTestGroup>
```

The above profile defines the following:

For Tests 1000, 1001 and 1002 to sample, Tests 1000, 1001 and 1002 MUST have a Cpk ≥ 2 AND Test 50 must not have any more than 2 fails during the lot.

SELECTING TEST GROUPS FOR SAMPLING

The goal of sampling tests is to reduce test time. It is recommended that you use a structured process to determine which tests are candidates for sampling as indicated in the flow chart shown in figure 1. This application note will focus on the selection of candidate tests based on careful review of case study results.



RECOMMENDATIONS – GATHERING DATA AND TEST TIMES

It is recommended that you use using a minimum of five lots of recent STDF production data including 100% of the test results on a high volume product of interest. Each STDF file should contain a minimum of 500 devices from the same package test lot or a full wafer, collected in stop-on-fail mode. Larger lot sizes will provide more accurate TTR predictions. Data should come from previously untested devices and retest data should never be used for a case study.

A Test Time Profile indicating the execution time for each group of tests is required to determine the test time reduction (TTR).

PDF Solutions Applications Engineers utilize the TestVision desktop analysis package to perform the case study analysis and present results in an easy-to-follow format. The analysis reveals a great deal of detailed information on the quality and robustness of your test process. TestVision identifies opportunities for test process improvement and candidate tests for sampling (test time acceleration). And TestVision can simulate the TTR benefits of SwifTest TTR installed in your production environment.

In TestVision, the sample simulation is performed at the test group level. When looking for possible sampling candidate tests, all tests in the test group should be reviewed. If any test in the test group is less than satisfactory, that group should not be selected for the sample simulation. Sampling candidate tests have the following characteristics:

- Tests that are reasonably centered between the upper and lower test limits ($Cpk > 1.66$) with a small range ($Cp > 1.66$).
- Tests that have no failures or a failure can be shown to be a problem with the test process (ex: device was not making proper contact) and retesting the device shows no failures on that test.
- Tests that may have some shift between lots but have consistent distributions on a lot-to-lot basis.
- Tests that are not multi-modal unless it is clear that instrumentation resolution is causing the multi-modal effect and the test limits are greater than 6-sigma from the distribution.
- Tests that are data logged for information only (no binning decision).
- Tests that are non-critical.
- Tests that do not show latent failures during accelerated burn in. Note this is customer internal data and is not associated with the TestVision simulator.

The simulator will reveal whether or not your selections are truly sampling candidate tests. Even if a test group meets all the above criteria, if it is critical for testing device functionality (e.g. specific function not tested elsewhere), you may choose not to sample the test. Carefully reviewing simulator results over multiple lots or wafers will ensure the correct selection of tests for sampling.

BASELINE SETTINGS

It is recommended that you use a minimum baseline of 50 devices or 1% of the average lot size (whichever is greater) not to exceed 200. The baseline is a sample of the lot or wafer and is used to determine baseline statistics which are in turn used to determine whether to sample a candidate test. At wafer test the baseline is calculated for each wafer therefore it is also recommended that you select non-edge die for the baseline. In most cases a simple modification to the wafer touch down map will be necessary.

TEST PROGRAM DEPENDENCIES

Test engineers will frequently use setups from one test for subsequent tests in the flow. For example: voltage settings, instrument connections, etc. The test later in the flow is dependent on the setups made in the earlier test. If the test with the setups is removed from the flow, the dependent test will either fail or the distribution will be shifted.

A complete dependency exercise should be performed before releasing a “sampling ready” test program into the correlation phase. While the test program is being instrumented for sampling, the test program can be evaluated for dependencies using the following:

- If there are known dependencies which are affected by a sampling candidate test: determine if the dependencies can be removed, if the dependent tests can be sampled as well, or if the sampling candidate test must be removed from the list. This is best achieved by direct communication with the product/test engineer.
- With one device in the socket, run the test program 10 times with no tests being skipped, then 10 times for every combination of skipped tests.
- Save the STDF (or similar datalog) and interrogate it for hard fails and significant distribution shifts.
 - For long test programs and/or large numbers of skipped tests, this may not be feasible. If this is the case, choose the combinations which will be the most effective in finding dependencies.
 - It is recommended that you use these three combinations as a minimum:
 - Skip each test one at a time.
 - Skip all tests except for one at a time.
 - Skip the first, then first and second, then first, second and third, etc.
- In most test programs, this can also be scripted to save time.
- Again, determine if the dependencies can be removed, if the dependent groups can be skipped together, or if the affected sampling candidate must be removed from the list.

CORRELATION AND TARGET DPPM

A correlation exercise should be performed before releasing a “sampling ready” test program into production. It is recommended that at least 5,000 normal production packaged devices or five wafers be tested for correlation using the following flow:

- Run the sampling test program on the correlation devices with full data logging turned on.
- Turn sampling off and 100% retest the correlation devices with full data logging turned on.
- Review any binning changes in the resulting data including pass-to-fail, fail-to-pass and fail-to-different-fail on sampled tests. Also note the number of binning changes on non-sampled tests since these will indicate the effective DPM binning change rate of the normal test process.
- Retest any device or die that passed the sampling test program but failed the 100% test program only if a sampled test incurred a binning change. In nearly all cases these parts will pass upon retest since the problem is usually related to poor touch down or device positioning in the contactor.
- If any test can be proven to have a hard binning failure, permanently turn off sampling for that test.
- Document and publish the correlation results.

The key to a successful implementation is to follow the process outlined above. For added assurance, SwifTest can also be setup to shut off sampling immediately on any sampled test that the software detects a fail. SwifTest is always monitoring the test process in real-time and can also shut off sampling on tests where the Cpk falls below a user defined threshold. Using TestVision to analyze test data, many companies have been able to lower the DPPM rate by improving the test process.

For expert help on your SwifTest sampling implementation, contact PDF Solutions applications engineers at support@pdf.com.

CUSTOMIZING SAMPLING

SwiftTest TTR has the ability to create customized modules for controlling test time reduction. These modules are implemented in Java and packaged as a JAR file. There are example conditions and strategies provided in the **swiftest/examples** directory.

CREATING CUSTOM TEST TIME REDUCTION CONDITIONS

Sampling conditions in SwiftTest TTR must implement the **ISamplingCondition** interface. The methods exposed by the **ISamplingCondition** interface are invoked by the sampling framework in response to certain events.

```
public interface ISamplingCondition {
    public boolean evaluate(Lot lot, TestRun testRun);

    public List<SamplingEvent> getSamplingEventList();

    public void read(Element element);
}
```

public boolean evaluate(Lot lot, TestRun testRun)

The **evaluate** method receives the **Lot** object and the current **TestRun** object. It returns true if the method evaluated successfully, false otherwise. The **Lot** object provides access to all information currently collected about the lot such as parametric tests, functional tests, binning information, etc. The **TestRun** object provides information about the current test run, such as devices and individual device test results.

public List<SamplingEvent> getSamplingEventList()

The **getSamplingEventList** method returns a list of events that this condition has recorded. A **SamplingEvent** contains the time-stamp when the event happened, the test run number, the device number, the name of the condition and a description of the event. The sampling event list is used in reporting to record sampling events.

public void read(Element element)

The **read** method is used to read in settings from the XML device profile. The element object provided contains methods to read attributes and child elements.

CREATING CUSTOM TEST TIME REDUCTION STRATEGIES

Sampling strategies in SwiftTest TTR must implement the **ISamplingCondition** interface. The methods exposed by the **ISamplingCondition** interface are invoked by the sampling framework in response to certain events.

```
public interface ISamplingStrategy {  
    public boolean skip(int testRunNumber);  
  
    public void startSampling(int testRunNumber);  
  
    public void read(Element element);  
}
```

public boolean skip(int testRunNumber)

The **skip** method receives the test run number and returns **true** if that test run should be skipped, or **false** otherwise. This method is only called after all conditions have been satisfied, and after the **startSampling(int)** method is called.

public void startSampling(int testRunNumber)

The **startSampling** method is called by the sampling framework the first time that all conditions are satisfied. It is called once per test group, per lot.

public void read(Element element)

The **read** method is used to read in settings from the XML device profile. The element object provided contains methods to read attributes and child elements.

CHAPTER 6

SWIFTEST PART AVERAGE TESTING (PAT)

OVERVIEW

The Automotive Engineering Council (AEC) outlier detection guideline defines PAT, in part, as the following: "... a statistically based method ...for removing parts with abnormal characteristics from the semiconductors supplied"

Standard AEC PAT defines limits as being n standard deviations from the mean. The number of standard deviations on each side of the mean is independently configurable.

SwiftTest's PAT methods implement the essence of this guideline, taking into account process variation from one wafer to the next or one test setup to the next. SwiftTest takes a baseline number of measurements, then calculates and applies new PAT limits based on the current wafer/lot being tested, then assigns "PAT-bins" to the outliers.

SwiftTest allows you to implement PAT in real-time for both packaged parts and wafers, and as a post-process for wafers.

DYNAMIC PAT VS. STATIC PAT

A full-scale implementation of PAT involves using both Static PAT and Dynamic PAT. There is a significant difference between the two. Each is described in the following sections.

Static PAT

Implementing Static PAT involves using historical data to tighten spec limits for tests, based on their individual statistics. These limits should be evaluated regularly (e.g. every six months) to account for process drift. The system allows a user to use historical data to perform PAT simulations and should be used to implement Static PAT. Historical data is only used to establish Static PAT limits per the AEC-Q003 and AEC-Q001 specifications.

Dynamic PAT

The goal of Dynamic PAT is to change the test limits for a particular test by monitoring test data in real-time, and calculating statistics for the lot or wafer being tested. Dynamic PAT only takes into account the statistics of a test for the current lot being processed when establishing new limits. The Dynamic PAT limits are set, and **do not** change after they are established at the baseline. The methodology used for Dynamic PAT by SwiftTest is in accordance with the Automotive Engineering Counsel (AEC) Specification Q001 Rev. C.

PAT METHODOLOGY



***IMPORTANT:** Retesting PAT outlier devices goes against the general PAT methodology. If the devices are tested out of context, i.e. as a separate lot, the results may be meaningless. In addition, if the PAT outlier devices are fed into the handler and retested with the same PAT limits, they may or may not be identified as PAT outliers due to process variability and tester calibration drift.*

SwiftTest establishes and applies Dynamic PAT limits after a configurable number of baseline results is received for a particular test. The AEC-Q001 specification states that only passing test results should be used to determine the Dynamic PAT limits. In addition, SwiftTest can be configured to exclude passing test results from failing devices to provide a higher level of reliability.

AEC Method

When a test has reached its PAT baseline, a histogram is calculated and checked for normalcy. If the histogram indicates a normal distribution, the PAT limits are calculated using the standard formula:

$$\text{Mean} \pm (n * \text{sigma})$$

Where:

- **Mean** is the average or mean of the baseline.
- **n** is a user defined number which can be configured for each individual test.
- **sigma** is the standard deviation of the test results.

Robust Method

The formula for calculating the PAT limits using the Robust method is:

$$\text{Median} \pm (n * \text{RobustSigma})$$

Where:

- **Median** is the median, or second quartile (Q2), value of the test results.
- **n** is a user-defined number which can be configured for each individual test.
- **RobustSigma** is the robust sigma calculated as:
$$\text{RobustSigma} = (Q3 - Q1) / 1.35$$

This method is optional and typically results in tighter limits and, consequently, more outliers than the standard formula.

DEVICE PROFILES

After a baseline number of devices have been tested, SwiftTest will calculate the PAT limits for this wafer or lot and remove outliers from the population.

The device profile should be configured to use only passing devices based on binning or passing devices only (**useOnlyPassingDevices**) when calculating the new PAT limits, thus removing any questionable defective device measurements from the dataset. This means that the device failed “any” test in the flow, not just the test that PAT is being applied to. Failing devices can significantly negatively affect yield loss once the new PAT limits are applied if the baseline sample is small and/or the measured values on failing devices is far from the rest of the passing device measurements.

SwiftTest PAT can be configured to apply PAT to either the upper or lower limit, or both. It can also be configured to have the same or different upper and lower number of standard deviations and upper and lower offsets to account for site-to-site offsets. This, along with the three different methods for calculating the limits (Standard AEC, Robust Sigma and Box Plot), give the engineer the flexibility to account for issues such as single-sided distributions and non-normal distributions. All AEC DPAT outlier methods share a common XML schema.

DEVICE PROFILE EXAMPLE

```
<?xml version='1.0' encoding='UTF-8'?>
<DeviceProfile xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" version="6.0">
  <DeviceInfo jobName="" partType="" company="" organization="" />
  <TestInfo company="" organization="" location="" facility="" />
  <Author name="DynamicProfile" email="support@pdf.com"
generatedBy="RecipeDeliveryService 4.1.0.131" profileRev="" generatedDate="2019-
10-29T01:28:08-07:00" />
  <MonitorSettings />
  <Features>
    <Feature type="PAT" enabled="true" patLimitsBySite="true"
useOnlyPassingDevices="false">
      <Bins>
        <Bin type="Software" number="5"/>
        <Bin type="Hardware" number="7"/>
      </Bins>
      <PATTestSet>
        <PATTest enabled="true" upperLimitEnabled="true" subTestNumber="0"
testNumber="2" lowerLimitEnabled="true" description="Test2">
          <PATMethod type="Standard" baseline="20" lowerSigmaFactor="2"
upperSigmaFactor="2" lowerMinOffset="2" upperMinOffset="3"
ruleGUID="5B2BF57B8A4F4D20AF7A551983838CE7" revision="1" />
        </PATTest>
        <PATTest enabled="true" upperLimitEnabled="true" subTestNumber="0"
testNumber="3" lowerLimitEnabled="true" description="Test3">
          <PATMethod type="Standard" baseline="30" lowerSigmaFactor="5"
upperSigmaFactor="10" lowerMinOffset="3" upperMinOffset="4"
ruleGUID="5B2BF57B8A4F4D20AF7A551983838CE7" revision="1" />
        </PATTest>
      </PATTestSet>
    </Feature>
    <Feature type="Trigger" enabled="true">
      <TriggerMethodSet>
        <TriggerMethod type="LotEndTrigger" enabled="true" ruleID="1021"
ruleGUID="CFC5B153592F45EDB55762E0F83A4875" revision="1">
          <TriggerActions>
            <TriggerAction actionID="2084" type="CreatePATReport"
enabled="true">
              <OutputStrategies>
                <OutputStrategy maxRetryAttempts="100"
destination="reports" type="file"
retryIntervalInMinutes="2" isRetransmittable="true" />
              </OutputStrategies>
            </TriggerAction>
          </TriggerActions>
        </TriggerMethod>
      </TriggerMethodSet>
    </Feature>
  </Features>
</DeviceProfile>
```

Device Profile XML Specification

Element	Type	Default	Validation	Description
Feature	Tag		N/A	
type	Text			The name of the feature.
enabled	Bool	False	true/false	Globally enable or disable this feature.
bySite	bool	false	true/false	(Optional.) Setting this triggers bySite attribute to true will treat each site population independently.
Bins	tag			Begin bins entry. Note that if Bins node is defined then useOnlyPassingDevices will be ignored, even if set to true.
Bin	tag			Define a single watched bin
type	text	"Hardware" / "Software"	case insensitive	Hardware or software bin
number	int			Number of each hardware or software bin to monitor
useOnlyPassingDevices	Bool	False	true/false	The name given to this algorithm. This must match the name transmitted by the client program with the data.
PATTestSet	Tag			Contains one or more PATTest elements.

STANDARD AEC

Standard AEC PAT defines limits as being n standard deviations from the mean. The number of standard deviations on each side of the mean is independently configurable.

ROBUST SIGMA

The robust PAT method uses a robust (quartile) based calculation for standard deviation that is more agreeable to non-normal distributions.

BOX PLOT

Similar to the inter-percentile range method, box-plot calculates limits based on percentiles from the median and a margin multiplier.

BoxPlot PAT limits calculation

$$\text{LowerLimit} = \text{Median} - \text{Max}(\text{LowerMinOffset}, \text{LowerN} * (\text{Median} - \text{Percentile}(\text{LowerPercentile})) / \text{MarginFactor})$$

$$\text{UpperLimit} = \text{Median} + \text{Max}(\text{UpperMinOffset}, \text{UpperN} * (\text{Percentile}(\text{UpperPercentile}) - \text{Median}) / \text{MarginFactor})$$
Device Profile Example

Examples of profiles for Standard (AEC), Robust and BoxPlot PAT methods.

```
<Feature type="PAT" enabled="true" useOnlyPassingDevices="false">
  bySite="true">
    <PATTestSet>

      <Bins>
        <Bin type="Software" number="5"/>
        <Bin type="Hardware" number="7"/>
      </Bins>

      <PATTest testNumber="2003" subTestNumber="0" description="IDDB"
        lowerLimitEnabled="true" upperLimitEnabled="true" enabled="true">
        <PATMethod type="Standard" baseline="50" lowerSigmaFactor="6.0"
          upperSigmaFactor="6.0" lowerMinOffset="0.0" upperMinOffset="0.0"/>
      </PATTest>

      <PATTest testNumber="2004" subTestNumber="0" description="IDDC"
        lowerLimitEnabled="true" upperLimitEnabled="true" enabled="true">
        <PATMethod type="Robust" baseline="50" lowerSigmaFactor="2.0"
          upperSigmaFactor="2.0" lowerMinOffset="0.0" upperMinOffset="0.0"/>
      </PATTest>

      <PATTest testNumber="2005" subTestNumber="0" description="IDDE"
        lowerLimitEnabled="true" upperLimitEnabled="true" enabled="true">
        <PATMethod type="BoxPlot" baseline="50" lowerPercentile="5.0"
          upperPercentile="95.0" lowerN="6.0" upperN="6.0" marginFactor="1.645"
          lowerMinOffset="0.0" upperMinOffset="0.0"/>
      </PATTest>

    </PATTestSet>
  </Feature>
```



NOTE that the *Boxplot* upperPercentile and lowerPercentile **values are configurable to adjust the size of the “box” for included values.**

PATTest Elements

Name	Type	Description
testNumber	Int	DPAT limits will be applied to this test number.
subtestNumber	Int	DPAT limits will be applied to this test number (if applicable).
description	Text	Arbitrary.
lowerLimitEnabled	Bool	If true, a lower DPAT limit will be applied.
upperLimitEnabled	Bool	If true, an upper DPAT limit will be applied.

PATMethod Elements

Name	Type	Description
type	Text	Standard, Robust, BoxPlot
baseline	Int	How many devices of data to collect before computing DPAT limits.
bySite	bool	(Optional.) Setting this triggers bySite attribute to true will treat each site population independently.
lowerSigmaFactor	Real	Set the lower DPAT limit this number of standard deviations from the mean/median (depending on the method).
upperSigmaFactor	Real	Set the upper DPAT limit this number of standard deviations from the mean/median (depending on the method).
lowerPercentile	Real	Defines the bottom of the box.
upperPercentile	Real	Defines the top of the box.
lowerN	Real	Multiplier for [median – lowerPercentile] to set lower DPAT limit.
upperN	Real	Multiplier for [median + upperPercentile] to set upper DPAT limit.
marginFactor	Real	Divisor used to loosen or tighten Box Plot DPAT limits.
lowerMinOffset	Real	Minimum distance lower DPAT limit can be from mean. Note: 0 = ignore lowerMinOffset.
upperMinOffset	Real	Minimum distance upper DPAT limit can be from mean. Note: 0 = ignore upperMinOffset.

HOW DEVICES ARE BINNED

Devices with measured values outside of the test program limits are binned normally, as failures. Devices with measured values outside the PAT limits and inside the test program limits are binned as PAT outliers. Devices with measured values inside the PAT limits are flagged as passing and will continue on to the next test in the flow.

CHAPTER 7

AUTOUPDATE

UPDATESWIFTEST OVERVIEW

This tool is used to automatically update SwiftTest files on each tester. Using a Spotfire template, you have the ability to configure what SwiftTest version you want to update on which specific testers.

This functionality give you the ability to fine-tune what elements of new SwiftTest software updates get distributed to which testers.

3. & 7. Shut-down ports, stop at end of lot

Current SwiftTest /lib

Update SwiftTest

- Spotfire Template - UpdateSwifttest

Command –UpdateSwifttest updatefiles.zip

Reply – OK – from each tester – indicates update has begun

SwiftTest starts UpdateSwifttest

1. Grab all new files via FTP from server
2. Put new files in /lib2.
3. Shutdown ports on /lib Swifttest, stop at end of lot.
4. Start /lib2 Swifttest, if it fails to start, restart /lib.
5. /lib2 Swifttest takes the next lot.
6. Put new files in /lib.
7. Shutdown ports on /lib2 Swifttest, stop at end of lot.
8. Start /lib1 Swifttest, if it fails to start, restart /lib2.
9. /lib Swifttest takes the next lot.
10. UpdateSwifttest removes /lib2 and shuts down.

New SwiftTest JARs, configs, profiles and new version of AutoUpdateSwifttest

UpdateSwifttest Progress Indicators

New SwiftTest /lib2

New SwiftTest /lib

Send first Heartbeat – indicates update midway

Send first Heartbeat – indicates update complete

TDS

HOW UPDATESWIFTEST WORKS

UPDATESWIFTEST COMMANDS

swiftest -updateSwifttest updatefiles.zip

This command tells SwiftTest to fire off the UpdateSwifttest and where to find the new JARs, ext dir, configs and profiles.

The command line is:

swiftest -updateSwifttest updatedir/someUpdate.zip
on a Windows machine

OR

./swiftest -updateSwifttest updatedir/someUpdate.zip
on a Linux/Unix machine.

If you don't provide directory and path, as above, then the system will login to the FTP home directory and look there for the *.zip file.

The command from a template is:

**Command=UpdateSwiftTest;source=update1/
incompleteLib.zip**

- **-updateSwifttest** tells SwiftTest to go into Update mode.
- **updatedir/someUpdate.zip** tells SwiftTest to go into this directory on the FTP site and grab the “someUpdate.zip” file. The ZIP file contains all the files that need to be updated.

UPDATESWIFTEST PROCESS

1. SwiftTest starts the SwifttestUpdate Java program.
2. SwifttestUpdate pulls down the zip file and puts it in **.../swiftest/swiftestUpdate/** working directory.
3. The system installs a second SwiftTest instance.
4. The system tells SwiftTest to stop at the end of the current lot (if one is running).
5. The system starts the second instance in **.../swiftest/lib2**.

6. When the first SwiftTest is done (but still may be processing reports at the end of the lot) the second SwiftTest will start.
So the system does not miss the next lot, there are checks to make sure the second instance starts. If not, it will restart the first SwiftTest.
7. The UpdateSwiftest Java program removes the first instance of SwiftTest.
8. The system installs the new SwiftTest instance.
9. The system tells **Swiftest/lib2** to stop at the end of the current lot (if one is running).
10. The system starts the new instance in **.../swiftest/lib**.
11. When the second SwiftTest is done (but still may be processing reports at the end of the lot) the new SwiftTest will start in **.../swiftest/lib** so the next lot is not missed.

SWIFTESTUPDATE DIRECTORY STRUCTURE

UpdateSwiftest is in **.../swiftest/swiftestUpdate**.

- The **archive** directory is for keeping a copy of what has been updated.
- The **config** directory holds the configuration for swiftestUpdate and for the logging.
- The **lib** directory holds the Java files for the swiftestUpdate.
- The **working** directory holds the *.zip file that was just downloaded from the FTP site that will be used to update Swiftest.

SWIFTESTUPDATE CONFIGURATION

The **swiftestUpdate.xml** file contains an **InputStrategy** of where to FTP the files from. The system can use the current **AutoUpdateSwifttest** FTP site and credentials.

The file is typically located in:

- **c:\swiftest\swiftestUpdate\config** on Windows machines.
- **/home/pdfs/swiftest/swiftestUpdate/config** on Unix/Linux machines

Contents of the file are below:

```
<?xml version='1.0' encoding='UTF-8'?>
<SwiftTestUpdateSettings archivesToKeep="5">
  <UpdateSource>
    <InputStrategy type="FTP" enabled="true" isBinary="true"
      isPassiveMode="true" isRetransmittable="true"
      retryIntervalInMinutes="1" maxRetryAttempts="2">
      <FtpServer host="updateSource.ftpSite.com" user="foo"
        password="bar" directory="sourceDirectory"/>
    </InputStrategy>
  </UpdateSource>
</SwiftTestUpdateSettings>
```

- **archivesToKeep** = number of backups to keep
- **InputStrategy type** = only FTP is supported
- **enabled** = set to **true** to enable this configuration file.
- **isBinary** = set to **true** to .
- **isPassiveMode** = set to **true** to .
- **isRetransmittable** = set to **true** to if you want to attempt to retransmit upon a transmission failure.
- **retryIntervalInMinutes** = the frequency in minutes with which retransmits will be attempted.
- **maxRetryAttempts** = the maximum number of retries to be attempted.
- **FtpServer host** = the FTP server location.
- **user/password** = user name and password for the FTP server.
- **directory** = directory name where.

UPDATE_LOGGING.XML

Update_logging.xml is a typical jog4j configuration file. It can be helpful to set the “File” location to write the swiftestUpdate logging into the standard Swiftest.log like this:

```
<param name="File" value="logs/swiftest-$(HOSTNAME).log"/>
```

This makes the timing much easier to read.

You can also set logging level to debug like this:

```
<priority value="debug"/>
```

“UPDATEFILES.ZIP” FILE CONTENTS

The *.zip file (whatever name you give it) will contain all of the *.jar, *.xml, *.profile.

Additionally, at some customer sites, there will be an **ext** directory containing the **customCUSTOMERNAME.jar** file.

There must a **lib** directory, a **lib/ext** directory, a **config** directory, and a **profiles** directory. The swiftestUpdate Java program will download the *.zip file and open it in the working directory or the new Swiftest directory.

RESTORING THE PREVIOUS VERSION OF SWIFTTEST

The commands and process for restoring the previous version of SwiftTest are similar to running the tool. In this case the tool just gets the files from the previous version of Swifttest that was installed on the tester. (The previous versions are stored in the ...**swifttest/swifttestUpdate/archive** directory.)

The commands to restore the previous version of Swifttest:

```
swifttest -updateSwifttest restore
```

This command tells SwiftTest to revert back to the previous version of SwiftTest, whatever the previous version that was on this system.

The command line is:

```
swifttest -updateSwifttest restore
```

OR

```
./swifttest -updateSwifttest restore
```

The command from a template is:

```
Command=UpdateSwiftTest;source=restore
```


CHAPTER 8

APPENDIX — DEVELOPMENT AND INTEGRATION

This chapter provides an overview of how to integrate SwiftTest with external software, as well as information about how to develop custom components and plug them into SwiftTest. This document assumes a level of familiarity with the following subjects:

- Socket programming
- XML
- Java development for writing extensions to SwiftTest

REMOTE COMMAND INTERFACE

COMMAND CATALOG

SwiftTest ships with a rich set of remote commands that can be used when integrating with external systems.

AbortLot

<i>Description</i>	The AbortLot command will force SwiftTest to abort processing of a running lot. SwiftTest will stop processing additional data from this lot and begin lot end processing.
<i>Syntax</i>	Command=AbortLot;GUID=<The GUID obtained from the OpenLots command>
<i>Response</i>	<Response Command='AbortLot' CommandStatus='Success' ReportTime='2012-03-06T16:22:27-06:00'/>

AddRemoteSystemEventListener

Description SwiftTest allows systems to register to be notified via network of events. This requires a server socket to be running on the listener that can accept “callbacks” from SwiftTest in response to events. The event messages are:

```
Event=LotMonitorCreated;GUID=<The GUID>
Event=LotMonitorClosed;GUID=<The GUID>
Event=DateProcessingStarted;GUID=<The GUID>
Event=DateProcessingEnded;GUID=<The GUID>
Event=LotIdentified;GUID=<The GUID>
Event=FinalStatisticsStarted;GUID=<The GUID>
Event=FinalStatisticsEnded;GUID=<The GUID>
Event=LotEndProcessingStarted;GUID=<The GUID>
Event=LotEndProcessingEnded;GUID=<The GUID>
Event=LotMonitorIdleEventStarted;GUID=<The GUID>
Event=LotMonitorIdleEventEnded;GUID=<The GUID>
```

Syntax Command=AddRemoteSystemEventListener;host=<The listening host name or IP address>;port=<The listening port number>

Response <Response Command='AddRemoteSystemEventListener'
CommandStatus='Success' ReportTime='2012-08-07T13:34:28-05:00'><AddRemoteSystemEventListener
instanceId='fb6c0ba46a341f44_31e0cd05_1390259087c_-8000'/></Response>

BeginLotMonitorSession

Description The BeginLotMonitorSession begins a lot monitor session.

Syntax Command= BeginLotMonitorSession;DataFilePath=<The path to the STDF file>;ProfileFilePath=<The path to the device profile>

Response <Response Command='BeginLotMonitorSession' CommandStatus='Success' ReportTime='2012-08-07T13:49:35-05:00'/>

<Response Command='BeginLotMonitorSession' CommandStatus='Error' ReportTime='2012-08-07T13:48:12-05:00' Message='Unable to locate data file C:\swiftest\data\archive\MAX3997_F_SV1ZAU003C_FT_ROOM_20120103201630.stdf'/>

BeginSamplingSession

Description The BeginSamplingSession begins a sampling session and returns the port of the data pump.

Syntax Command=
BeginSamplingSession;LotID=MyLotID;SubLotID=MySubLotID;NodeName=
MyNodeNameJobName=MyJobName;PartType=MyPartType;HeadNumber=1

Response Status=Success;Port=60003

BeginSocketLotMonitorSession

Description The BeginSocketLotMonitorSession returns an XML document with the port for the requested socket connection.

Syntax Command= BeginLotMonitorSession;ProfileFilePath=<The path to the device profile>

Response <Response Command='BeginSocketLotMonitorSession'
CommandStatus='Success' ReportTime='2012-08-07T14:49:27-05:00'
port='1386'/>

BeginSwiftTestProxyConnection

Description The BeginSwiftTestProxyConnection returns an XML document with the connection status of the current lot.

Syntax Command=BeginSwiftTestProxyConnection; GUID=550247a84a9220b7_-
23c804d4_135ea16f27b_-7fff;Time=Hello

Response <Response Command='BeginSwiftTestProxyConnection' CommandStatus='Error'
ReportTime='2012-08-07T14:58:38-05:00' Message='Fatal error'/>

BinSummary

Description The BinSummary returns an XML document with the current bin counts for the specified lot.

Syntax Command=BinSummary;GUID=<The GUID obtained from the OpenLots command>

DisableTestRunTriggers

Description The DisableTestRunTriggers disables the SPC triggers for a given GUID.

Syntax Command=DisableTestRunTriggers;GUID=550247a84a9220b7_-23c804d4_135ea16f27b_-7fff

Response <Response Command='DisableTestRunTriggers' CommandStatus='Success' ReportTime='2012-03-06T16:18:44-06:00'/>

EndSamplingSession

Description The EndSamplingSession command will end a sampling session.

Syntax Command=EndSamplingSession;SessionID=2352553

Response Status=Success

EndSwifttestProxyConnection

Description The EndSwiftTestProxyConnection command will close a SwiftTest Proxy connection.

Syntax Command= EndSwiftTestProxyConnection; GUID=550247a84a9220b7_-23c804d4_135ea16f27b_-7fff;Time=;Csum=927364

ForwardMessage

Description The ForwardMessage command will forward a message string to TestScape if the TestScapeIntegration feature in SwiftTest is enabled.

Syntax Command=ForwardMessage;Message=<A text message>

Response <Response Command='ForwardMessage' CommandStatus='Error' ReportTime='2012-03-06T15:17:11-06:00' Message='Message forwarding is disabled.'/>

GetDeviceProfile

Description The GetDeviceProfile command will return the device profile in use for the specified lot.

Syntax Command=GetDeviceProfile;GUID=<The GUID obtained from the OpenLots command>

GetHistory

Description The GetHistory command will return an XML document listing all SwiftTest startups, shutdowns, lot starts, and lot ends for the specified time period.

Syntax Command=GetHistory;Days=<The number of days worth of history, or 'ALL'>

Syntax <Response Command='GetHistory' CommandStatus='Success' ReportTime='2012-08-07T17:21:24-05:00'><EventList><Event type='STARTUP' date='2012-08-07T13:28:51-05:00' version='6.3.5'><Event type='STARTUP' date='2012-08-07T13:29:14-05:00' version='6.3.5'><Event type='LOT_START' date='2012-08-07T13:49:37-05:00' guid='fb6c0ba46a341f44_31e0cd05_1390259087c_-7ffe' lotId='SV1ZAU003C' subLotId='' jobName='MAX3997_F' partType='myPartType' nodeName='myNodeName' headNumber='1' lotClass='Unknown' profileName='myProfile.profile.xml'><Event type='LOT_END' date='2012-08-07T13:49:54-05:00' guid='fb6c0ba46a341f44_31e0cd05_1390259087c_-7ffe'></EventList></Response>

GetLogFile

- Description** The GetLogFile command will return the requested log file (zipped).
- Syntax** Command=GetLogFile;Filename=<The log file name obtained from the GetLogFileList command>

GetLogFileList

- Description** The GetLogFileList command will return an XML document listing all available log files. Each entry contains the log file name, current size, and a flag to indicate if it is a live log file or an archive log file.
- Syntax** Command=GetLogFileList
- Response** <Response Command='GetLogFileList' CommandStatus='Success' ReportTime='2012-08-07T17:23:40-05:00'><LogFileList><LogFile filename='swiftest-kevinxp.log' date='2012-08-07T17:23:40-05:00' size='83985' live='true'/><LogFile filename='swiftest-kevinxp-2012-07-05.log.zip' date='2012-07-05T00:00:00-05:00' size='6637' live='false'/></LogFileList></Response>

GetLotDetails

- Description** The GetLotDetails command will return an XML document containing the full set of known lot metadata. This includes all the information contained in the STDF MIR and SDR records.
- Syntax** Command=GetLotDetails;GUID=<The GUID obtained from the OpenLots command>

GetOpenDataSourceList

- Description** The GetOpenDataSourceList command will return an XML document containing a list of all open data sources.
- Syntax** Command= GetOpenDataSourceList

Response <Response Command='GetOpenDataSourceList' CommandStatus='Success' ReportTime='2012-08-10T11:42:07-05:00'><GetOpenDataSourceList><DataSource Type='StdfRecordFactory' URI='file:/C:/swifttest/data/archive/MAX3997_F_SV1ZAU003C_FT_ROOM_20120103201630.std' GUID='ee2cc9635ce5f055_78b8e802_13911698a1a_-7ffc'/><DataSource Type='StdfRecordFactory' URI='file:/C:/swifttest/data/archive/ftxwea013ac_QA_ROOM_01142012_042446_wafer19.std' GUID='ee2cc9635ce5f055_78b8e802_13911698a1a_-7ffe'/><DataSource Type='StdfRecordFactory' URI='file:/C:/swifttest/data/archive/N8C057-04_20120503_231512.STDF' GUID='ee2cc9635ce5f055_78b8e802_13911698a1a_-7ffa'/></GetOpenDataSourceList></Response>

GetSettings

Description The GetSettings command will return an XML document containing the current *swifttest.xml* configuration file embedded as CData.

Syntax Command=GetSettings

Response <Response Command='GetSettings' CommandStatus='Success' ReportTime='2012-08-07T17:17:36-05:00'><GetSettings><Settings source='config\swifttest.xml'><![CDATA[<?xml version='1.0' encoding='UTF-8'?>

<SwiftTestSettings>

<ServerSettings commandPort='60000' maxLotMonitorCount='5' captureEnabled='false' captureDir='data/capture'>

.

.

.

</SwiftTestSettings>]]></Settings></GetSettings></Response>

LicenseInfo

Description The LicenseInfo command will return an XML document listing all checked out licenses.

Syntax Command=LicenseInfo

Response <Response Command='LicenseInfo' CommandStatus='Success'
ReportTime='2012-08-07T17:30:42-05:00'><LicenseInfo><MonitorLicenses/
><SamplingLicenses/><PATLicenses/><OutlierLicenses/
><DynamicTestLicenses/></LicenseInfo></Response>

OpenLots

Description The OpenLots command will return an XML document listing all currently running lots.

Syntax Command=OpenLots

Response <Response Command='OpenLots' CommandStatus='Success' ReportTime='2012-08-10T11:14:56-05:00'><OpenLots><LotInfo JobName='MyJobName'
PartType='MyPartType' NodeName='Tester052' HeadNumber='1'
LotID='MyLotID' SubLotID="" StartTime='2012-01-14T04:31:58-06:00'
WaferID='19' DeviceCount='10' LotClass='FIRSTPASS' LastRecordTime='2012-08-10T11:14:56-05:00' State='Lot Identified'
GUID='ee2cc9635ce5f055_1b4f9f05_139114f2dd5_-7ffe' URI='file:/C:/swiftest/
data/archive/MySTDFFile.std'/><LotInfo JobName='MyJobName'
PartType='MyPartType' NodeName='Tester9328' HeadNumber='1'
LotID='MyLotID' SubLotID="" StartTime='2012-05-03T23:15:12-05:00'
WaferID='04' DeviceCount='12' LotClass='FIRSTPASS' LastRecordTime='2012-08-10T11:14:56-05:00' State='Lot Identified'
GUID='ee2cc9635ce5f055_1b4f9f05_139114f2dd5_-7ffe' URI='file:/C:/swiftest/
data/archive/<MySTDFFile.STDF'/><LotInfo JobName='MyJobName'
PartType='MyPartType' NodeName='Tester040' HeadNumber='1'
LotID='MyLotID' SubLotID="" StartTime='2012-01-03T20:16:30-06:00'
WaferID="" DeviceCount='61' LotClass='UNKNOWN' LastRecordTime='2012-08-10T11:14:56-05:00' State='Lot Identified'
GUID='ee2cc9635ce5f055_1b4f9f05_139114f2dd5_-7ffe' URI='file:/C:/swiftest/
data/archive/MySTDFFile.std'/></OpenLots></Response>

OpenNewStdFDataConduitConnection

Description The OpenNewStdFDataConduitConnection command will return a port value where a client can initiate an STDF data stream through a socket connection.

Syntax Command= OpenNewStdFDataConduitConnection

Response Status=Success;Port=60001

RemoveRemoteSystemEventListener

Description The RemoveRemoteSystemEventListener command will remove a listener that previously registered to receive events with the AddRemoteSystemEventListener command.

Syntax Command=RemoveRemoteSystemEventListener;host=<The listening host name or IP address>;port=<The listening port number>

Response <Response Command='RemoveRemoteSystemEventListener'
CommandStatus='Success' ReportTime='2012-08-10T11:25:27-05:00'/>

SetCapture

Description The SetCapture command will turn on or off the capture feature of SwiftTest. Note that this has no effect on already processing lots. The capture setting will revert to what is configured in *swifttest.xml* the next time SwiftTest is restarted.

Syntax Command=SetCapture;Value=<true or false>

Response <Response Command='SetCapture' CommandStatus='Success'
ReportTime='2012-08-10T11:26:48-05:00'/>

SetDebug

Description The SetDebug command will turn on or off debug logging. The log level setting will revert to what is configured in *log4j.xml* the next time SwiftTest is restarted.

Syntax Command=SetDebug;Value=<true or false>

Syntax <Response Command='SetDebug' CommandStatus='Success' ReportTime='2012-08-10T11:28:01-05:00'/>

SetLotClass

Description The SetLotClass command will turn set the lot class of the running lot. This will not affect the lot class as reported in the ilog. It is used to disable triggers and alerts if for example the failing parts are retested at the end of a lot.

Syntax Command= SetLotClass;NewLotClass=QA

Syntax <Response Command='SetLotClass' CommandStatus='Success'
ReportTime='2012-08-10T11:28:01-05:00'/>

Shutdown

Description The Shutdown command will cause SwifTest to begin shutting down. SwifTest will stop accepting new lots but will complete processing of any running lots when issued the Shutdown command. The When parameter is not currently used and should always be zero.

Syntax Command=Shutdown;When=0

Response <Response Command='Shutdown' CommandStatus='Success' ReportTime='2012-08-10T11:29:50-05:00'/>

ShutdownNow

Description The ShutdownNow command will cause SwifTest to shut down immediately. All running lots will be aborted and no lot end processing will occur.

Syntax Command=ShutdownNow

Response <Response Command='ShutdownNow' CommandStatus='Success'
ReportTime='2012-08-10T11:29:50-05:00'/>

SiteSummary

Description The SiteSummary command will return an XML document describing the counts and yields for each site for the specified lot.

Syntax Command=SiteSummary;GUID=<The GUID obtained from the OpenLots command>

Response <Response Command='SiteSummary' CommandStatus='Success' ReportTime='2012-03-06T16:03:38-06:00'><SiteSummary><Site Number='0' Count='35' Pass='35' Fail='0' YieldPercentage='100.00'></SiteSummary></Response>

Stop

Description The Stop command will cause SwiftTest to shut down immediately. All running lots will be finished and all lot end processing will occur.

Syntax Command=Stop

Syntax <Response Command='Stop' CommandStatus='Success' ReportTime='2012-08-10T11:35:10-05:00'>

SystemStatus

Description The SystemStatus command will return an XML document containing information about the running SwiftTest instance, including the version, last startup date, running lots, and total lots processed since last started.

Syntax Command=SystemStatus

Response <Response Command='SystemStatus' CommandStatus='Success' ReportTime='2012-08-10T11:33:46-05:00'><ServerStatus><SystemInfo SwiftTestVersion='6.3.5' SwiftTestBuild='2206' SwiftTestRevision='4842' SwiftTestBuildDate='2012-07-05T14:05:37-05:00' RunningSince='2012-08-10T11:31:38-05:00'><ActivityStatus CurrentMemUsage='337551360' LotsProcessedSinceStartup='3' LotsCurrentlyInProcess='2'></ServerStatus></Response>

